

CT3307: Network Security

Week 6 : BGP & DNS Security

Chua Zong Fu





Module 1 : Foundation of DNS



Internet Routing Fundamentals and the Trust Model

1. **Autonomous Systems (AS):** Role of AS numbers, transit, peering, and customer relationships.
2. **Economic incentives:** Understanding the business logic behind routing decisions.
3. **How BGP actually moves traffic:** Distinction between control plane decisions vs. data plane forwarding.
4. **Indirect and distributed decisions:** Why routing is eventually consistent" and lacks a central authority.
5. **Route propagation and path selection:** Mechanics of how routes spread and stabilize.
6. **The default trust assumption:** Why BGP trusts announcements by design and lacks verification.
7. **Basic connectivity models:** Single upstream vs. multi-homed resiliency and exposure trade-offs.
8. **Where control ends:** What you can enforce inside your AS vs. what you cannot enforce once routes leave.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Before we talk about hacks, hijacks, or firewalls, we have to understand how the internet actually works. It is not a single cloud; it is a network of networks held together by a protocol from 1989 called BGP.

"First, we define the players. The internet isn't made of routers; it is made of **Autonomous Systems (AS)**. We will discuss what an ASN is, and the difference between paying for 'Transit' and agreeing to 'Peer' for free.

BGP is weird because it doesn't choose the *fastest* path; it chooses the *cheapest* path. We will look at the **Business Logic** behind routing decisions, why your traffic might go through Europe to reach Asia if it saves your ISP money.

"We need to separate the **Control Plane** (The Map) from the **Data Plane** (The Traffic). We'll explain why you can have a perfect map but still have dropped packets if the forwarding tables aren't in sync.

There is no central brain. The internet is **Eventually Consistent**. We will discuss how routing decisions are made via a global game of 'Telephone' (rumors), and why there is no central authority to call when things break.

How does a route spread? We will look at the mechanics of **Propagation**, how an announcement in Singapore ripples out to London and New York, and how routers choose the 'Best Path' when they hear two different stories.

The Default Trust Assumption is the root cause of every attack we will study. BGP was built on a handshake. We will discuss why routers believe whatever they are told, and why BGP lacks built-in verification.

"We'll cover architecture. Should you have one ISP (Single Upstream) or two (Multi-Homed)? We will weigh the **Resiliency** of redundancy against the **Exposure** trade-offs, more connections mean more doors to lock.

Finally, we define the boundary. We will discuss what you can enforce *inside* your AS versus what you are powerless to control once the packet leaves your border. This is the definition of **Internet Risk**.



Autonomous Systems (AS) & The Routing Hierarchy

- **Definition:** An Autonomous System (AS) is a collection of IP networks under the control of a single administrative entity (like an ISP, University, or Enterprise) that presents a common routing policy to the internet.
- **The Identity:** Every AS is identified by a unique **ASN** (Autonomous System Number).
 - Example: Google is AS15169. Your University is likely its own AS.
- **The Relationships:**
 - **Transit:** A paid relationship where a smaller network pays a larger network (Upstream) to carry its traffic to the rest of the world.
 - **Peering:** A barter" relationship where two networks of similar size connect directly to exchange traffic between *only their own customers* for free.

Think of an AS as an independent country. Transit is like paying for a flight to travel anywhere in the world. Peering is like building a bridge to your neighbor's house so you can walk over for free.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

To understand how the internet functions at a macro level, we must move beyond individual IP addresses and look at **Autonomous Systems**. An AS is essentially a collection of IP networks and routers under the control of a single administrative entity, such as an ISP, a university, or a large enterprise, that presents a common routing policy to the rest of the internet,. You can think of an AS as a sovereign territory in the digital world. Just as a country manages its own internal roads but has specific checkpoints for crossing borders, an AS manages its own internal routing but uses a unified policy when exchanging traffic with the rest of the world.

2. The Identity: ASN Every one of these administrative domains is identified by a globally unique **Autonomous System Number (ASN)**. These numbers are allocated by the **Internet Assigned Numbers Authority (IANA)** and managed regionally by Regional Internet Registries (RIRs) like ARIN or RIPE,. For example, when we look at network traffic, we don't just see a random IP; we can map that IP back to its 'owner' via the ASN, such as identifying traffic belonging to Google, Amazon EC2, or a specific university. This is crucial for threat intelligence because it allows us to identify the reputation of the network hosting a specific IP address,.

3. The Protocol of the Internet: BGP The relationships described on this slide, Transit and Peering, are technically implemented using the **Border Gateway Protocol (BGP)**,. BGP is the routing protocol that allows these independent 'countries' to talk to one another. Unlike internal protocols that calculate routes based on speed or bandwidth, BGP makes routing decisions based on policies and paths, specifically, the shortest AS path,.

Transit: When we discuss 'Transit' as a paid relationship, we are talking about connecting to an upstream provider. In BGP terms, we are asking a larger network to advertise our routes to the rest of the world.

Peering: 'Peering' is a direct connection between two networks to exchange traffic only between themselves. In BGP configurations, these peers are explicitly defined neighbors.

4. Security Implications Finally, understanding this hierarchy is vital for security. BGP was designed in an era of trust, meaning routers inherently trust the route advertisements they receive from their neighbors,. This makes the distinction between Transit and Peering critical; if we misconfigure a peering relationship, we might accidentally become a 'transit' center for others, routing traffic we shouldn't be handling. Furthermore, attackers can exploit this trust to redirect traffic (BGP hijacking), so monitoring AS paths and relationships is a key part of network defense.



Economic Incentives in Routing

- **The Logic:** BGP is not a performance protocol; it is a policy protocol driven by money.
 - **Valley-Free Routing:**
 - Routers generally prefer sending traffic to a **Customer** (because the customer pays them).
 - If that fails, they send to a **Peer** (Free).
 - As a last resort, they send to a **Provider** (because that costs money).
 - **Security Implication:** Attackers can exploit this. If an attacker peers with you and announces a route, your router might prefer the attacker's path over a valid path simply because the attacker is a "cheaper" or "closer" hop in the BGP decision algorithm.
- The "Best Path" in BGP usually means the Cheapest Path," not necessarily the fastest or safest.**

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. Defining the Autonomous System (AS) To understand how the internet functions at a macro level, we must move beyond individual IP addresses and look at **Autonomous Systems**. An AS is essentially a collection of IP networks and routers under the control of a single administrative entity, such as an ISP, a university, or a large enterprise, that presents a common routing policy to the rest of the internet,. You can think of an AS as a sovereign territory in the digital world. Just as a country manages its own internal roads but has specific checkpoints for crossing borders, an AS manages its own internal routing but uses a unified policy when exchanging traffic with the rest of the world,.

2. The Identity: ASN Every one of these administrative domains is identified by a globally unique **Autonomous System Number (ASN)**. These numbers are allocated by the **Internet Assigned Numbers Authority (IANA)** and managed regionally by Regional Internet Registries (RIRs) like ARIN or RIPE,. For example, when we look at network traffic, we don't just see a random IP; we can map that IP back to its 'owner' via the ASN, such as identifying traffic belonging to Google, Amazon EC2, or a specific university. This is crucial for threat intelligence because it allows us to identify the reputation of the network hosting a specific IP address,.

3. The Protocol of the Internet: BGP The relationships described on this slide,

Transit and Peering, are technically implemented using the **Border Gateway Protocol (BGP)**,. BGP is the routing protocol that allows these independent 'countries' to talk to one another. Unlike internal protocols that calculate routes based on speed or bandwidth, BGP makes routing decisions based on policies and paths, specifically, the shortest AS path,.

Transit: When we discuss 'Transit' as a paid relationship, we are talking about connecting to an upstream provider. In BGP terms, we are asking a larger network to advertise our routes to the rest of the world.

Peering: 'Peering' is a direct connection between two networks to exchange traffic only between themselves. In BGP configurations, these peers are explicitly defined neighbors.

4. Security Implications Finally, understanding this hierarchy is vital for security. BGP was designed in an era of trust, meaning routers inherently trust the route advertisements they receive from their neighbors,. This makes the distinction between Transit and Peering critical; if we misconfigure a peering relationship, we might accidentally become a 'transit' center for others, routing traffic we shouldn't be handling. Furthermore, attackers can exploit this trust to redirect traffic (BGP hijacking), so monitoring AS paths and relationships is a key part of network defense.



Control Plane vs Data Plane

- **The Distinction:**

- **Control Plane (The Brain):** The BGP process running on the router's CPU. It exchanges routing updates (IP prefixes) and builds the map of the internet.
- **Data Plane (The Muscle):** The ASIC chips that actually forward the user packets based on the map built by the Control Plane.

- **Why it Matters:**

- Routing decisions are distinct from packet forwarding. You can have a perfectly functioning Data Plane but a confused Control Plane (e.g a Route Leak).
- **Security Risk:** Denial of Service attacks often target the *Control Plane* (flooding the CPU with updates) to crash the router, even if the Data Plane has plenty of bandwidth.

1. The Brain vs. The Muscle: Understanding the Separation To secure a network, we must understand the fundamental architectural split within a router: the **Control Plane** and the **Data Plane**. Think of the Control Plane as the 'brain' of the device. This is where protocols like **BGP** (Border Gateway Protocol) and **OSPF** operate, running on the router's general-purpose CPU. Its sole job is to exchange routing updates, learn about the network topology, and calculate the best path to every destination, effectively building the 'map' of the internet. In contrast, the Data Plane acts as the 'muscle.' It takes the map built by the Control Plane and uses specialized hardware, such as **ASICs** (Application-Specific Integrated Circuits), to forward user traffic at line speed. The Data Plane doesn't 'think'; it simply executes the forwarding instructions it was given.

2. Why the Distinction Matters: Logical vs. Physical Failure This separation is critical because it means the health of your packet forwarding capability is distinct from the accuracy of your routing logic. You can have a perfectly functioning Data Plane with massive bandwidth available, yet suffer a catastrophic outage because the Control Plane is confused. A classic example is a **Route Leak** or **BGP Hijack**. In these scenarios, a router might receive and accept a malicious or erroneous routing update, such as a more specific prefix advertised by an attacker. The Data Plane will dutifully forward traffic

according to this corrupted map, sending sensitive data into a black hole or through an adversary's network, even though the physical infrastructure is operating perfectly ``.

3. Security Risks: Targeting the CPU From a security perspective, this architecture reveals a specific vulnerability: the Control Plane is a resource-constrained bottleneck compared to the high-speed Data Plane. Attackers exploit this via **Denial of Service (DoS)** attacks that specifically target the router's CPU rather than its bandwidth . For instance, by flooding a router with BGP updates or control packets (like the 'data to control plane saturation' attacks seen in SDN environments), an attacker can exhaust the CPU. If the CPU hits 100%, the router can no longer process keepalive messages, causing BGP sessions to drop and the network to de-converge . This crashes the network logic without ever saturating the physical links, proving that protecting the Control Plane requires different safeguards, like strict filtering of management traffic, than protecting the Data Plane.



Indirect and Distributed Decisions

- **No Central Brain:** The internet has no central server or database of Truth. There is no "God View".
- **Distributed Consensus:** Every router makes decisions based solely on what its immediate neighbors tell it.
- **Eventual Consistency:** When a route changes (e.g. a cable cut in Tokyo), it takes time for that "rumor" to propagate to New York. During this convergence time, different parts of the internet have different maps.
- **Security Implication:** You cannot turn off a bad route instantly. If someone hijacks your IP, it takes time for your mitigation (announcing a more specific prefix) to spread globally.

1. No Central Brain: The Absence of God View" To understand internet routing security, we must accept a fundamental truth: there is no single server or database that defines the 'state' of the internet. Unlike a centralized application where a master database holds the absolute truth, the internet relies on **Distributed Consensus**. Routing protocols, particularly BGP, operate on a 'trust your neighbor' model. A router does not have a global view of every link; it makes decisions based solely on the reachability information advertised to it by its immediate peers. Consequently, a router's 'view' of the internet is entirely dependent on what its neighbors tell it, which makes the system inherently reliant on trust rather than verification.

2. Eventual Consistency: The Propagation Delay Because decisions are distributed, the internet operates on **Eventual Consistency**. When a significant change occurs, such as a physical cable cut in Tokyo or a router failure, this information must propagate hop-by-hop across the globe. This is not instantaneous; it takes time for the 'rumor' of the route change to travel from Asia to New York. During this convergence time, different parts of the internet hold different maps of the topology. A router in London might still be sending traffic to the failed link in Tokyo because the update hasn't reached it yet, while a router in Singapore has already updated its table. The internet is rarely in a state of perfect

synchronization.

3. Security Implication: The Latency of Defense This architecture creates a critical security challenge: we cannot instantly 'turn off' a malicious route. If an attacker hijacks your IP address, perhaps by advertising a more specific prefix, which routers prefer over shorter prefixes, your traffic is immediately redirected to them. To fix this, you must announce a new, legitimate route (mitigation). However, just like the initial attack, your defensive announcement is subject to propagation delays. It takes time for your correction to spread globally and overwrite the attacker's false map in every router's table. During this lag time, the attacker maintains access to your traffic, emphasizing that BGP defense is often a race against propagation latency rather than just a configuration switch.



Route Propagation and Path Selection

- **The Mechanism:** How does a router choose a path?
 - **Longest Prefix Match:** (e.g /24 wins over /22).
 - **Local Preference:** (Policy/Economics).
 - **AS_PATH Length:** (Shortest number of hops).
- **Stabilization:** BGP routers are programmed to dampen flapping" routes (routes that go up and down quickly) to prevent CPU exhaustion.

Millions of routes enter the router, pass through a series of filters (Path Selection), and only the single Best Path" is installed in the forwarding table.

1. The BGP Decision Mechanism: A Filter for Millions When we consider how a router handles the global routing table, we are talking about ingesting millions of potential routes and distilling them down to a single 'Best Path' for each destination. Unlike internal protocols that might prioritize pure speed, BGP is a **policy protocol** used to build a map of the internet based on administrative rules and relationships. The router runs these millions of routes through a rigorous selection algorithm, often remembered by the mnemonic 'Wise Lip Lovers Apply Oral Medication Every Night' (Weight, Local Preference, Locally Originated, AS_PATH, Origin, MED, eBGP, NextHop), to determine which single path is installed in the forwarding table.

2. The Supreme Rule: Longest Prefix Match Before we even get to attributes like AS Path, there is a fundamental rule that overrides everything else: **Longest Prefix Match**. If a router receives two advertisements for IP addresses that overlap, it will *always* prefer the most specific route. For example, if one neighbor advertises a broad /19 network and another advertises a specific /24 subnet within that /19, the router chooses the /24 path for traffic destined to that specific range. This mechanism is powerful but dangerous; it is frequently the vector used in route hijacking attacks (like the Rostelecom incident) because a malicious actor advertising a more specific prefix can steal traffic even if their

path is longer or less preferred by policy.

3. Policy and Metrics: Local Pref and AS_PATH Once prefix lengths are equal, the router evaluates BGP attributes. **Local Preference** allows network administrators to enforce policy and economic decisions, such as preferring a route through a private peer (which is free) over a transit provider (which costs money),. Following this, the router looks at the **AS_PATH Length**. BGP generally assumes that fewer hops between Autonomous Systems equate to a better path, making the 'shortest AS path' a primary metric for selection,.

4. Stabilization: Preventing Control Plane Exhaustion Finally, we must address stability. In large service provider networks, stability is often preferred over rapid convergence. If a route is 'flapping', constantly advertising as up and then down, it forces the router's Control Plane CPU to repeatedly recalculate the topology. This can lead to CPU exhaustion, where the router becomes so busy processing updates that it drops keepalive messages and crashes the BGP session. To prevent this, BGP routers employ dampening mechanisms (like holding off on sending notifications) to stabilize the network and protect the Control Plane from being overwhelmed by routing noise.



The Default Trust Assumption

- **The Flaw:** BGP was designed in 1989 for a trusted environment (Universities and Gov).
- **Implicit Trust:** By design, BGP assumes that if a neighbor announces a route (e.g I am Google"), they are telling the truth.
- **Lack of Verification:** The protocol has no native mechanism to verify if the neighbor *actually owns* the IP address they are announcing.
- **The Result:** This is the root cause of **Prefix Hijacking**. Security features like RPKI are bolt-on" patches to fix this foundational design choice.

1. The Flaw: Architecture from a Bygone Era The core vulnerability of the Border Gateway Protocol (BGP) stems from its origin story. BGP was architected in the late 1980s, a time when the internet was a smaller, closed loop of universities and government agencies. In that context, the protocol was designed with a 'trust your neighbor' philosophy suitable for a trusted WAN environment where all participants played by the rules. The designers prioritized connectivity and policy propagation over security, never anticipating a modern landscape filled with malicious actors and complex commercial relationships.

2. Implicit Trust and Lack of Verification Because of this legacy design, BGP operates on **Implicit Trust**. When a neighboring router sends an update saying, 'I know how to get to this network,' your router believes it by default. There is no native mechanism within the original protocol to challenge this assertion or verify that the neighbor actually owns or is authorized to advertise that specific IP space. This means that a malicious actor, or simply a fatigued administrator making a typo, can announce routes they do not control, and the BGP mesh will blindly accept and propagate them.

3. The Result: Prefix Hijacking and Bolt-on Security This blind trust is the root cause of **Prefix Hijacking**. Attackers exploit this by advertising a 'more specific' route (a longer prefix) than the legitimate owner, knowing that BGP routers will

automatically prefer the more specific path, redirecting traffic through the attacker's infrastructure. To fix this foundational flaw, we have had to implement 'bolt-on' patches like **Resource Public Key Infrastructure (RPKI)** and BGPSEC. These solutions attempt to retrofit the protocol with digital certificates and cryptographic verification to validate route origins, adding a layer of authentication that was conspicuously absent from the original design.



Basic Connectivity Models

- **Single Upstream:**
 - *Pros:* Simple configuration, cheaper.
 - *Cons:* Single point of failure. If the fiber is cut, you are offline.
- **Multi-Homed:**
 - *Definition:* Connecting to two or more ISPs (e.g Verizon and Level3).
 - *Pros:* Resiliency and redundancy.
 - *Cons:* Complexity. You must now run BGP yourself, manage your own ASN, and handle load balancing.
- **Exposure Trade-off:** Multi-homing increases your attack surface. You are now reachable via multiple paths, and a misconfiguration on your end (Route Leak) affects multiple ISPs.

1. The Single Upstream Model: Simplicity vs. Fragility We begin with the most common configuration for small to mid-sized entities: the **Single Upstream** model. Here, your network relies on a single Internet Service Provider (ISP). The primary advantage is simplicity; you typically do not need complex routing protocols like BGP because a simple static default route to the ISP handles all traffic. However, this simplicity creates a critical **Single Point of Failure**.

Defenses against Physical Layer attacks or environmental hazards rely primarily on redundancy; if your single fiber line is cut or your ISP suffers an outage, your organization goes completely offline.

2. Multi-Homing: Complexity and the BGP Requirement To solve the availability problem, organizations shift to **Multi-Homing**, connecting to two or more upstream providers (e.g Verizon and Level3). While this provides essential resiliency and failover capabilities, it introduces significant operational complexity. You can no longer rely on a default route; you must now participate in the internet's routing decision-making process. This requires you to run **Border Gateway Protocol (BGP)** yourself and obtain a unique **Autonomous System Number (ASN)** to identify your network to the global routing table. You also take on the responsibility of load balancing, which in BGP is driven by policy and path attributes rather than simple bandwidth metrics.

3. The Exposure Trade-off: Route Leaks and Attack Surface Finally, we must address the security trade-off. Multi-homing expands your **attack surface** because your infrastructure is now reachable via multiple disparate paths, complicating access control. More dangerously, BGP was designed in an era of trust, meaning routers often accept route advertisements from neighbors by default. If you misconfigure your BGP routers, you risk causing a **Route Leak**. This happens when you accidentally advertise routes learned from one ISP to another ISP. This effectively turns your corporate network into a transit hub for global traffic, potentially overwhelming your bandwidth and disrupting the ISPs. To mitigate this, organizations must implement strict route filtering and utilize monitoring services (like BGPMON) to detect route hijacking or configuration errors immediately.



Control Boundary

- **The Boundary:** You have absolute control over routing *inside* your Autonomous System (iBGP/OSPF). You can dictate exactly how traffic flows.
- **The Void:** Once a packet leaves your border router and enters your ISP (eBGP), you have **zero control** over the path it takes.
- **Scenario:** You send traffic to a server in London. Your ISP might route it through Russia or China due to their own economic agreements (or a hijack). You cannot prevent this at the routing layer.
- **Security Takeaway:** This is why **Encryption (TLS/VPN)** is non-negotiable. You must assume the transport path is untrusted and potentially hostile.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Control Boundary: Absolute Authority vs. The Void We must recognize the distinct dichotomy between our internal network and the public internet. Inside our **Autonomous System (AS)**, we maintain absolute control. We utilize Interior Gateway Protocols (IGPs) like **OSPF** or **iBGP** to engineer traffic flow precisely. In this environment, we dictate the path, performance, and security policy. However, the moment a packet traverses our border router and is handed off to an Internet Service Provider (ISP) via **eBGP**, it enters 'The Void.' We surrender all control over the physical and logical path that packet takes to reach its destination. The internet operates on a mesh of distributed trust, where no single entity dictates the global map.

2. The Scenario: Geographic and Political Routing Risks The slide presents a scenario where traffic destined for London is routed through Russia or China. This is not hypothetical; it is a feature of how **Border Gateway Protocol (BGP)** functions. BGP makes routing decisions based on policy, AS path length, and economic relationships rather than geographic logic or security. Consequently, your traffic may traverse hostile infrastructure simply because an upstream provider found it to be the 'cheaper' path. Furthermore, this can happen maliciously through **BGP Hijacking**, where an adversary advertises a more specific route to attract traffic through their network, as seen in the Rostelecom

incident where financial traffic was redirected. Crucially, we cannot technically prevent this at the routing layer; features like **Source Routing** (which would allow a sender to dictate the path) are almost universally blocked by ISPs today due to security risks.

3. Security Takeaway: The Necessity of Encryption Because we cannot control the path or trust the intermediaries, we must adopt a defensive posture that assumes the transport network is hostile. **Encryption in transit** is non-negotiable. We must rely on upper-layer security protocols like **TLS (Transport Layer Security)** or **VPNs (using IPsec)** to protect the data. Encryption ensures that even if our traffic is routed through a hostile nation or intercepted by a compromised router, the confidentiality and integrity of the payload remain intact. In the modern network, the physical path is irrelevant; the cryptographic tunnel is the only boundary that matters.



DNS Architecture

1. **DNS roles and hierarchy:** Recursive resolvers, authoritative servers, Root, TLDs, and delegation.
2. **How resolution really works:** Query flow, caching behavior, TTLs, and propagation delays.
3. **Operational DNS models:** Centralized, Distributed, and Anycast DNS architectures.
4. **Split-horizon DNS:** Managing internal vs. external views.
5. **DNS as a dependency:** Why almost every service fails without DNS (single point of systemic failure).
6. **Implicit trust in DNS:** Why resolvers trust responses and where integrity is assumed.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

To understand how the internet functions, we must view the Domain Name System (DNS) not just as a 'phone book' mapping names to numbers, but as a distributed, hierarchical database. The architecture is strictly tiered: it begins at the **Root** (managed by entities like ICANN), flows down to **Top Level Domains (TLDs)** like .com or .org, and finally delegates authority to organizations that manage specific domains. In this ecosystem, the **Recursive Resolver** is the workhorse. When a client needs to find a website, it queries the recursive resolver, which, if it doesn't already know the answer, traverses the hierarchy, asking the Root, then the TLD, and finally the **Authoritative Name Server** to locate the specific IP address.

The efficiency of DNS relies heavily on **Caching**. When a recursive resolver learns an IP address, it stores that answer for a specific duration defined by the **Time to Live (TTL)**. This mechanism drastically reduces network traffic and latency for subsequent requests. However, this creates the challenge of **Propagation Delay**. If an administrator changes an IP address at the authoritative source, the rest of the internet will not know until the old TTL expires in the various caches worldwide. This latency is why we cannot instantly mitigate attacks by simply changing an IP address; we are at the mercy of the cache.

From a security architecture perspective, we must implement **Split-Horizon** (or Split Namespace) DNS. This model dictates that we maintain two distinct views of our network: an **External** view for the public internet and an **Internal** view for corporate employees. This segregation is critical. If we use a single DNS infrastructure for both, an external attacker can query our public name server to enumerate internal assets, such as development servers or intranets, effectively building a map of our internal network without ever breaching the perimeter. Split-Horizon ensures that internal records remain invisible to the outside world.

Finally, we must address the fragility of this system. DNS is a single point of systemic failure; if DNS goes down, services like Active Directory, web applications, and email simply stop working, even if the underlying network connectivity is fine. This risk is compounded by the protocol's **Implicit Trust**. Traditionally, DNS operates on a 'first response wins' basis over UDP. A resolver does not inherently verify the identity of the sender; it trusts the first response packet that matches its query transaction ID. This architectural flaw allows for **Cache Poisoning** and spoofing, where attackers inject malicious records to redirect traffic, necessitating the adoption of integrity layers like **DNSSEC** to cryptographically validate that an answer actually came from the authoritative source.



DNS Roles and Hierarchy

- **The Tree Structure:** DNS is a distributed database organized in a hierarchical tree.
 - **Root (.):** The top of the tree, managed by ICANN/IANA. It knows where the Top Level Domains live.
 - **TLD (Top Level Domain):** The extensions like .com, .org, .sg. They know who owns the specific domains.
 - **Authoritative Nameserver:** The server that holds the actual zone file" for a domain (e.g ns1.google.com). It provides the final answer.
 - **Recursive Resolver:** The librarian" that searches the tree on behalf of the client (e.g your ISP's DNS or 8.8.8.8).

Just like a Phonebook. The Root is the country code (+1), the TLD is the area code (212), and the Authoritative server is the local switchboard that connects you to the specific person.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Tree Structure: A Distributed Hierarchy To understand how the internet functions, we must view the Domain Name System (DNS) not as a single list, but as a massive, distributed database organized into a hierarchical tree structure. At the very top of this hierarchy sits the **Root**, often represented technically by a trailing dot (.). This root zone is managed by the Internet Corporation for Assigned Names and Numbers (ICANN) and IANA. The Root servers do not know the IP address of every website on earth; their specific job is to know where the Top Level Domains live and direct traffic to them.

2. TLDs and Authoritative Servers Below the root, we have the **Top Level Domains (TLDs)**, which include generic extensions like .com and .org, as well as country codes like .sg or .us. These registries serve as the next layer of delegation. They don't hold the final answers but point to the **Authoritative Nameservers**. The authoritative server is the definitive source of truth for a specific domain. It holds the actual 'zone file', the database containing the IP addresses (A records) and mail servers (MX records) for that specific domain,. When a query reaches this level, the authoritative server provides the final answer.

3. The Recursive Resolver: The Librarian Standing between the client and this complex hierarchy is the **Recursive Resolver**. You can think of this as a reference

librarian. Your computer does not typically contact the Root or TLD servers directly. Instead, it asks the Recursive Resolver (often provided by your ISP, or a public service like Google's 8.8.8.8) to find the answer,. This resolver performs the legwork of traversing the tree, asking the Root, then the TLD, and finally the Authoritative server, to retrieve the IP address and send it back to you,.

4. The Phonebook Analogy To visualize this, use the phonebook analogy provided on the slide. The **Root** acts like a country code (+1), directing you to a broad geographic region. The **TLD** acts like an area code (212), narrowing the search to a specific network or location,. Finally, the **Authoritative Server** acts like a local switchboard or a specific phone line that connects you directly to the person you are trying to reach. This hierarchical division of labor allows the system to be decentralized and redundant.



How Resolution Really Works

- **Recursive vs. Iterative:**

- **The Client (Stub Resolver):** Sends a **Recursive Query** ("Find me the IP for google.com, I don't care how you do it").
- **The Server (Recursive Resolver):** Performs **Iterative Queries** ("Hey Root, where is .com?" -> Hey .com, where is google?").

- **Performance Mechanics:**

- **Caching:** Resolvers save answers to avoid repeating work. Security risks like Cache Poisoning target this mechanism.
- **TTL (Time To Live):** How long a record stays in the cache. Short TTL = agility but high load; Long TTL = stability but slow updates.

1. The Mechanics of Resolution: Recursive vs. Iterative To understand how we get from a hostname to an IP address, we must distinguish between the two types of queries involved. The client, your laptop or phone, acts as a **Stub Resolver**. It issues a **Recursive Query** to its configured DNS server, essentially saying, 'Find me the IP for google.com, and I will wait until you have the final answer'. The burden of discovery lies entirely on the server. The server, acting as a **Recursive Resolver**, accepts this task and performs a series of **Iterative Queries**. It doesn't get the answer in one shot; instead, it navigates the hierarchy. It asks the Root servers, which refer it to the Top-Level Domain (TLD) servers (like .com), which in turn refer it to the authoritative name servers for the specific domain,. The recursive resolver follows this chain of referrals until it locates the specific server that holds the answer.

2. Performance Mechanics: The Critical Role of Caching If every single page load required traversing the entire global DNS hierarchy, the internet would be unacceptably slow. To solve this, resolvers rely heavily on **Caching**. Once a recursive resolver hunts down an answer, it saves that result. The next time a client asks for 'google.com,' the resolver skips the iterative process and answers immediately from its local memory,. However, this efficiency creates a major attack surface: **DNS Cache Poisoning**. If an attacker can race the legitimate

name server and inject a spoofed response into the resolver's cache, the resolver will unknowingly serve that malicious IP to every client that queries it subsequently,. This allows attackers to redirect traffic without ever compromising the client machine itself.

3. Time To Live (TTL): The Trade-off The lifespan of these cached records is governed by the **Time To Live (TTL)** value. This is a setting configured by the domain administrator that dictates how many seconds a record remains valid in a cache before it must be discarded and re-fetched,. TTL represents a strategic trade-off. **Long TTLs** (such as 24 hours) enhance stability and reduce the load on authoritative name servers because they are queried less frequently. **Short TTLs** (such as 60 seconds) provide agility, allowing administrators to change IP addresses rapidly for load balancing or disaster recovery,. However, attackers also leverage extremely short TTLs in 'Fast-Flux' networks to rapidly rotate the IP addresses of command-and-control servers, making them harder to block.



Operational DNS Models

- **Centralized:** Using a single pair of massive servers (e.g Active Directory Domain Controllers) for the whole organization. Easy to manage, but a single point of failure.
- **Distributed:** Placing DNS servers at every branch office. improved latency for local users, but harder to patch and secure.
- **Anycast:** The modern standard for resilience. The same IP address is announced from multiple global locations via BGP.
 - *Benefit:* If the London node goes down, BGP automatically routes traffic to the Paris node. It provides native" DDoS mitigation.

1. The Centralized Model: Simplicity vs. Fragility We begin with the **Centralized** model, which is the traditional architecture for many enterprises using Active Directory. In this setup, the Domain Controllers act as the focal point for the environment, hosting authentication and DNS services in a central datacenter. The primary advantage here is administrative simplicity; you have fewer servers to patch, monitor, and secure. However, this architecture creates a critical **Single Point of Failure**. If the central link is severed or the primary datacenter suffers an outage, the entire organization loses name resolution capabilities. Furthermore, relying on a central location can introduce latency for remote users who must traverse the WAN for every query.

2. The Distributed Model: Performance vs. Security To address latency, organizations may adopt a **Distributed** model. This approach places DNS servers at every branch office, adhering to the principle that placing data and services closer to the user minimizes network latency and improves performance. While this improves the user experience, it drastically expands the attack surface. We know that infrastructure is often built for ease of deployment rather than security, and remote branch servers are frequently managed by less-skilled staff or left unpatched for years. Maintaining consistency across hundreds of distributed nodes is difficult, and synchronization mechanisms like

Zone Transfers (AXFR) can become liabilities if not strictly secured.

3. Anycast: Resilience via BGP The modern standard for resilience is **Anycast**. Unlike unicast, where an IP addresses a single specific host, Anycast allows the same IP address to be announced from multiple different endpoints or data centers simultaneously. This is made possible by **BGP (Border Gateway Protocol)**. BGP views these multiple announcements and simply routes the client's traffic to the 'nearest' node based on routing metrics.

4. Native DDoS Mitigation The 'London to Paris' failover scenario on the slide illustrates Anycast's native ability to handle **Denial of Service (DoS)**. In a traditional setup, a flood of traffic can easily overwhelm a specific target IP. In an Anycast environment, because the IP exists in multiple locations, the attack traffic is distributed across the entire global network infrastructure rather than hitting a single server. If the London node becomes overwhelmed or goes offline, the route is withdrawn, and BGP automatically redirects traffic to the next closest healthy node (e.g Paris), effectively spreading the load and mitigating the impact without manual intervention.



Split-Horizon DNS

- **The Concept:** A security technique where the DNS server gives different answers depending on *who* is asking.
- **Internal View (Trusted):**
 - Query: intranet.corp.com
 - Answer: 10.10.10.5 (Private IP).
 - Access: Employees can reach the internal dashboard.
- **External View (Untrusted):**
 - Query: intranet.corp.com
 - Answer: NXDOMAIN (Domain does not exist).
- **Security Value:** Prevents external attackers from mapping your internal network topology (Reconnaissance) simply by querying your public DNS.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Concept: Splitting the Knowledge To secure our network architecture, we must implement **Split-Horizon DNS** (also referred to as Split Namespace or Split DNS). This is a security technique where we effectively 'split the knowledge' of our DNS zone into two distinct views: one for the public internet and one for our trusted internal network,. Rather than having a single DNS server that answers all queries for everyone, we configure our infrastructure so that the answer a client receives depends entirely on *who* is asking, specifically, whether the request originates from a trusted internal IP address or an untrusted external source.

2. Internal vs. External Views For our employees inside the corporate network, the **Internal View** is authoritative. When they query a resource like intranet.corp.com, the internal DNS server resolves this to the private IP address (e.g 10.10.10.5), facilitating access to internal dashboards and resources. However, for the **External View**, we strictly limit information. The external DNS server contains records *only* for public-facing systems, such as your main website or mail server,. If an outsider on the internet queries intranet.corp.com, the external server checks its limited public records, finds nothing, and returns an error like **NXDOMAIN** (Non-Existent Domain) or simply refuses to resolve it, effectively hiding the resource.

3. Security Value: Countering Reconnaissance The primary security value of this configuration is the prevention of **Reconnaissance**. If an organization uses a single DNS infrastructure for both internal and external clients, an external attacker can query the public name server to enumerate internal hosts. Tools like theHarvester or simple brute-force dictionary attacks can reveal the existence and IP addresses of internal assets, such as development servers or financial databases, allowing the attacker to map your internal network topology without ever breaching the firewall,. By employing Split DNS, we ensure that internal records remain invisible to the outside world, forcing attackers to operate blindly regarding our internal geography.



DNS as a Dependency

- **The Hidden Foundation:** DNS is often called the Achilles' Heel" of the internet.
- **Systemic Failure:** If DNS fails, *everything* fails.
 - Web browsing stops (can't resolve URLs).
 - Email stops (can't find MX records).
 - Active Directory fails (can't find Domain Controllers).
 - Even physical security (badge readers) may fail if they rely on DNS to find the central auth server.

When troubleshooting a network outage," 90% of the time the pipes are fine, but the directory (DNS) is broken.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Hidden Foundation: The Achilles' Heel We often refer to the Domain Name System (DNS) as the 'Achilles' Heel' of the internet because, while it acts as a silent background utility, it is a single point of failure for nearly every digital interaction. At its core, DNS is the directory service that translates human-readable hostnames into the IP addresses required for machines to communicate. Because this protocol is a precursor to almost every other network communication, its failure does not just degrade performance; it renders the infrastructure unusable.

2. Systemic Failure: From Browsing to Identity When DNS fails, the outage is systemic. Web browsing stops immediately because the browser cannot resolve the Uniform Resource Locator (URL) to the server's IP address. Email traffic also halts; mail servers rely on Mail Exchange (MX) records to identify authorized mail servers and route messages. Without these records, inbound and outbound mail flow ceases completely.

"Critically, this dependency extends deep into the corporate environment via **Active Directory (AD)**. AD is not just a user database; it relies on DNS to function. Workstations and servers use DNS Service (SRV) records to locate Domain Controllers for authentication, replication, and Group Policy updates. If DNS is unavailable, users cannot authenticate, and the 'focal point' of the

enterprise security architecture becomes inaccessible, effectively locking the organization out of its own network.

3. Physical Security and Troubleshooting The risk even bleeds into the physical realm. Modern physical security systems, such as badge readers and IP cameras, are increasingly network-connected IoT devices. If these devices rely on hostnames to locate their central controllers for authorization, a DNS failure can result in physical lockouts or a loss of surveillance capabilities.

"This reality drives the common troubleshooting adage that 'it's always DNS.' During major outages, such as the 2016 attack on DynDNS, the physical 'pipes' of the internet (the fiber and routers) were functioning perfectly with ample bandwidth. However, because the directory was broken, major global services were unreachable. This proves that availability is not just about connectivity; it is about resolvability.



Implicit Trust in DNS

- **The Design Flaw:** DNS was built in 1983 without security.
- **No Authentication:** By default, a resolver accepts the first response it receives that matches the transaction ID. It assumes the responder is telling the truth.
- **No Integrity:** There is no built-in way to verify if the IP address you received is actually the one the domain owner intended.
- **The Fix:** This vulnerability necessitates **DNSSEC** (cryptographic signatures) to prove identity and **Source Port Randomization** to make spoofing harder.

1. The Design Flaw: A Legacy of Trust We must recognize that the Domain Name System (DNS) was architected in the early 1980s, a time when the internet was a small, cooperative network of researchers rather than a hostile global battlefield. Consequently, the protocol was built with a fundamental 'default trust' assumption. It prioritizes the efficient dissemination of data over security, operating largely as an unauthenticated protocol. This legacy design means that if a device receives a DNS packet that looks correct on the surface, it processes that packet as absolute truth without inherently verifying the identity of the sender.

2. No Authentication: The First Response Wins" Vulnerability The mechanic that makes this dangerous is the lack of authentication. When a recursive resolver sends a query, it opens a UDP socket and waits for a reply. It identifies the correct response primarily by matching a 16-bit **Transaction ID** (or Query ID). The critical vulnerability here is that the resolver accepts the **first** response it receives that matches this ID. Once that first response is accepted, the socket closes or the application stops listening. If an attacker can 'race' the legitimate authoritative server and inject a spoofed response with the correct ID before the real answer arrives, the resolver accepts the attacker's lie as fact.

3. No Integrity: Blindly Accepting the Map Because there is no built-in integrity

check in the standard DNS protocol, there is no way for the resolver to verify that the IP address inside that packet is actually the one the domain owner intended. This allows for attacks like **DNS Cache Poisoning** (or spoofing), where an attacker injects a malicious IP address into the resolver's cache. For example, if an attacker successfully spoofs a response for 'google.com,' the resolver will unknowingly direct all subsequent users to the attacker's server until that cache entry expires.

4. The Fix: DNSSEC and Randomization To mitigate this, we must layer security on top of the protocol. **DNSSEC (DNS Security Extensions)** is the solution for integrity and authentication. It uses digital signatures (RRSIG records) to create a cryptographic chain of trust. This allows the resolver to mathematically verify that the response originated from the authoritative server and was not altered in transit. Additionally, to make the 'race' harder for attackers, we use **Source Port Randomization**. By randomizing the source port of the query in addition to the Transaction ID, we force the attacker to guess a much larger combination of variables (approx. 32 bits of entropy instead of 16) to successfully spoof a response, significantly raising the difficulty of cache poisoning.



Module 2 : Threats and Failure Modes



BGP Threat Landscape

1. **Prefix hijacking:** Full prefix vs. sub-prefix hijacks (and why specific routes win).
2. **Route leaks and misconfiguration:** Policy errors leading to accidental global impact.
3. **AS path manipulation:** How path attributes are altered to influence routing.
4. **Impact types:** Distinguishing between traffic interception, blackholing, silent failures, and noisy failures.
5. **Intent vs. Outcome:** Analyzing accidental incidents vs. malicious attacks (impact matters more than intent).
6. **Historical incidents:** High-level analysis focusing on blast radius rather than blame.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We have established that BGP connects the world. But because it connects everything, a failure here breaks everything. In this section, we are going to categorize exactly *how* BGP fails, whether by accident or by malice.

1. Prefix Hijacking (The Theft):

"First, we look at the most famous attack: **Prefix Hijacking**. This is when someone else announces your IP address space. We will explain the 'Longest Prefix Match' rule and why a more specific route (a /24) always beats a broader route (a /16), effectively stealing the traffic.

2. Route Leaks and Misconfiguration (The Accident):

"Not all outages are attacks. We will discuss **Route Leaks**, often called 'Fat Finger' errors. This is when an ISP accidentally tells the world it can route traffic to Google, crushing their small network under global load. We'll show you how policy errors cause global impact.

3. AS Path Manipulation (The Detour):

"Next, we cover subtle manipulation. **AS Path Manipulation** is when an attacker alters path attributes to silently influence traffic flow without announcing new prefixes. This is how you redirect traffic through a specific surveillance point.

4. Impact Types (Interception vs. Blackholing):

"We need to distinguish the *Goal*. Is the attacker trying to destroy you

(**Blackholing** / DoS) or spy on you (**Interception**)? We'll discuss why 'Silent Failures' (Interception) are much harder to detect than 'Noisy Failures' (Outages).

5. Intent vs. Outcome (Attribution):

"How do you tell the difference between a spy and a typo? We will analyze **Intent vs. Outcome**. We'll learn why, from a defensive perspective, the *Impact* matters more than the *Intent*, and why attribution in BGP is notoriously difficult.

6. Historical Incidents (Case Studies):

"Finally, we will look at the history books. We'll analyze famous incidents like **YouTube vs. Pakistan Telecom**. We study these not to blame, but to understand the 'Blast Radius', how a config error in one country can take down a platform on the other side of the world.



Prefix Hijacking (The More Specific" Attack)

- **The Mechanism:** BGP routers are programmed to prefer the most specific" path to a destination.
 - *Scenario:* You own 1.2.0.0/16. An attacker announces 1.2.3.0/24.
 - *Result:* All traffic destined for that /24 subnet will route to the attacker, because /24 is more specific than /16.
- **Types of Hijacks:**
 - **Full Prefix Hijack:** Announcing the exact same prefix. Often fails if paths are longer, but works if the attacker is closer.
 - **Sub-Prefix Hijack:** Announcing a smaller subnet. This almost always wins globally.

Just like Zip Code": If the post office has a bin for New York" (Broad) and a bin for Manhattan, New York" (Specific), the letter always goes to the specific bin.

1. The Mechanism: Precision Overrules Proximity To understand how prefix hijacking works, we must understand the 'Prime Directive' of internet routing: the **Longest Prefix Match**. While BGP normally uses metrics like the shortest AS Path to choose the best route, that rule only applies when comparing identical prefixes. If a router receives two advertisements for the same IP space, one broad and one specific, it is programmed to *always* prefer the most specific path. This is not a bug; it is a fundamental feature designed to ensure traffic gets to the precise destination. However, attackers exploit this by announcing a 'longer prefix' (a more specific subnet) than the legitimate owner. For example, if you own a broad /16 network, and an attacker announces a specific /24 within that range, routers will view the attacker's path as 'better' because it is more precise, regardless of where the attacker is located.

2. The Scenario: Sub-Prefix vs. Full Prefix Hijacks We categorize these attacks into two types based on the prefix length. In a **Full Prefix Hijack**, the attacker announces the exact same prefix as the victim. In this case, the internet is split; routers will route traffic based on who is 'closer' in terms of AS Path length. This often fails to capture global traffic if the attacker is far away. However, in a **Sub-Prefix Hijack**, the attacker announces a smaller, more specific subnet. Because the longest prefix match overrides AS path length, this attack almost always wins

globally. Even if the legitimate owner is one hop away and the attacker is ten hops away, the router chooses the attacker.

3. Real-World Context and Limitations This technique is highly effective and has been seen in major incidents. For instance, in 2017, the Russian ISP Rostelecom announced more specific prefixes for financial subnets, effectively redirecting traffic meant for major financial institutions through their network. A similar mechanic affected YouTube when a Pakistani ISP attempted to block it by announcing a more specific route, which inadvertently propagated globally. It is important to note, however, that there is a floor to this 'specificity' war; the smallest subnet typically propagated on the public internet is a /24. Attacks using prefixes longer than /24 (like /25) generally won't traverse the backbone carriers. To visualize this, use the 'Zip Code' analogy: if the post office has a general bin for 'New York' and a specific bin for 'Manhattan,' the letter always goes to the specific bin, regardless of which bin is closer to the sorter.



Route Leaks and Misconfiguration

- **The Fat Finger" Problem:** Most routing incidents are not malicious; they are mistakes.
- **Definition:** A Route Leak occurs when an AS accidentally propagates routes it learned from one neighbor to another neighbor in violation of policy.
 - *Example:* A small ISP buys service from Level3 and Verizon. It accidentally bridges them, telling Level3 I can reach Verizon. Suddenly, terabits of Tier 1 traffic try to squeeze through the small ISP's router.
- **Result:** Massive packet loss, high latency, and eventual hardware crash due to overload.
- **Key Defense:** Route Filters (Prefix Lists) and Max-Prefix Limits.

1. The Fat Finger" Reality: Malice vs. Mistake While we spend a great deal of time analyzing malicious threat actors, we must acknowledge that in the realm of BGP, the 'Fat Finger' problem, simple human error, is often more disruptive. Routing incidents frequently stem from configuration mistakes rather than intentional hijacking. A prime example occurred in 2021 when Vodafone Idea Ltd. mistakenly announced over 30,000 BGP prefixes, causing a massive spike in traffic. These incidents occur because BGP was designed in an era of trust, where peers largely accepted the reachability information sent by their neighbors without inherent verification of intent,.

2. Anatomy of a Route Leak To define it precisely, a **Route Leak** happens when an Autonomous System (AS) propagates routing information learned from one neighbor to another in a way that violates the intended routing policy. In the scenario on the slide, a small ISP is multi-homed to two Tier 1 providers (Level3 and Verizon). Without proper filtering, the small ISP might accidentally re-advertise the routes it learns from Verizon to Level3. Logically, BGP sees a valid path; physically, however, you have created a bridge between two massive oceans of data using a garden hose. If Level3 prefers this new path, perhaps because it appears logically shorter or more specific, it will attempt to shove terabits of global traffic through the small ISP's limited infrastructure,.

3. The Consequence: Volumetric Overload The result is a catastrophic denial of service, not just for the small ISP but potentially for the upstream providers as well. The small ISP's routers and links are instantly overwhelmed by the sheer volume of traffic (volumetric exhaustion). This leads to massive packet loss, extreme latency, and often a hardware crash as the routers run out of memory or CPU trying to process the flood of traffic,.

4. Defensive Posture: Filters and Limits The only defense against this is a 'verify, then trust' approach. You must implement **Route Filters** using **Prefix Lists**. These allow you to define explicit routing policies that whitelist only the specific IP blocks you own and intend to advertise, rejecting everything else by default,. Furthermore, you should implement **Max-Prefix Limits** on your BGP sessions. This acts as a safety mechanism (a circuit breaker) that tears down the BGP session if a neighbor attempts to dump more routes than your router is configured to handle, preventing resource exhaustion before it crashes your network.



AS Path Manipulation

- **The Concept:** Attackers can alter BGP attributes to influence the Best Path" decision without announcing new prefixes.
- **Techniques:**
 - **AS Prepending:** Adding your own ASN multiple times to make the path look longer (usually used for traffic engineering, but can be used maliciously to shift traffic).
 - **Origin Manipulation:** Changing the Origin" attribute to make a route look like it originated internally (IGP) rather than externally (EGP), which is preferred by routers.
- **Goal:** To silently redirect traffic through a specific surveillance point (Man-in-the-Middle) while maintaining connectivity.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Concept: Attribute-Based Engineering We must understand that BGP does not simply select a route based on who announces it; it selects the 'Best Path' based on a rigorous algorithm comparing specific attributes,. While we often focus on Prefix Hijacking (announcing a more specific IP block), **AS Path Manipulation** is a subtler tradecraft. Here, the attacker does not necessarily announce a new prefix but instead alters BGP path attributes, such as the AS Path length or Origin type, to trick the global routing table into preferring their route over the legitimate one. This exploits the deterministic nature of the BGP decision process, often summarized by the mnemonic 'Wise Lip Lovers Apply Oral Medication Every Night,' where 'Apply' stands for AS Path and 'Oral' stands for Origin.

2. Techniques: Prepending and Origin Codes The slide highlights two specific manipulation techniques. First is **AS Prepending**. In BGP, the shortest AS Path is generally preferred. Legitimate network operators frequently use prepending, intentionally adding their own Autonomous System Number (ASN) multiple times to a route advertisement, to make a path look longer and artificially 'drain' traffic away from a specific link for maintenance,. Attackers can invert this logic or strip prepending to make their malicious path appear shorter and more attractive than the valid path.

"Second is **Origin Manipulation**. BGP prefers routes that originate internally (IGP) over those learned externally (EGP) or those marked as incomplete. By fraudulently marking a route with an 'IGP' origin code, an attacker can win the routing decision if higher-priority metrics like Local Preference are equal, effectively overriding the legitimate route that might be marked as 'EGP' or 'Incomplete.'

3. The Goal: Silent Interception The objective here is not Denial of Service; it is surveillance. The attacker aims to establish a **Man-in-the-Middle (MITM)** position,. By influencing the best path decision, they force traffic to flow through a node they control. Crucially, to remain undetected, they must maintain connectivity. Once they capture the traffic, they inspect or copy it and then forward it to the legitimate destination,. This allows them to harvest credentials or sensitive data without disrupting the user experience, making the attack extremely difficult to detect without deep monitoring of routing anomalies.



Impact Types (Interception vs. Blackholing)

- **Traffic Blackholing (DoS):**
 - *Goal:* Destruction.
 - *Method:* Attacker announces the victim's IP, attracts the traffic, and then drops it (Null Route). The victim goes offline.
- **Traffic Interception (MitM):**
 - *Goal:* Espionage.
 - *Method:* Attacker attracts the traffic, inspects/copies it, and then forwards it to the legitimate destination.
 - *Detection:* Extremely difficult because the user still reaches the website, just with slightly higher latency.
- **Silent Failures:** Users don't notice interception until sensitive data is stolen.
- **Noisy Failures:** Blackholing is immediately obvious (site down).

1. Traffic Blackholing: The Noisy" Destruction When an attacker successfully hijacks a routing path, whether through BGP manipulation or local protocols, they must decide how to handle the stolen traffic. In a **Traffic Blackholing** scenario, the objective is destruction via Denial of Service (DoS). The attacker announces ownership of the victim's IP address range to attract the global traffic and then directs it to a 'null route' or nonexistent interface, effectively dropping the packets into a void,. Because the traffic never reaches its intended destination, the victim's services go offline immediately. This is a 'noisy' failure; it is instantly obvious to administrators and users that something is wrong because the site is down, making it easier to identify that an attack is occurring,.

2. Traffic Interception: The Silent" Espionage In contrast, **Traffic Interception** (Man-in-the-Middle) is a far more insidious threat where the goal is espionage and data theft,. Here, the attacker inserts themselves into the communication path, attracts the traffic, and inspects or copies sensitive data, such as credentials or proprietary information,. Crucially, to maintain stealth, the attacker **forwards** the traffic to the legitimate destination after inspecting it,.

3. The Detection Challenge Because the attacker forwards the traffic, the connection remains intact, and the user successfully reaches the website or service they intended to visit. This results in a 'silent failure' where the victim is

completely unaware that their communications are being monitored. Detection is extremely difficult because the only tangible symptom is often a negligible increase in latency, a difference of a few milliseconds that is imperceptible to the average human user. This allows attackers to harvest data for extended periods without raising alarms, whereas blackholing forces an immediate incident response.



Intent vs. Outcome

- **The Analyst's Dilemma:** When you see a weird route, is it a hack or a typo?
- **Accidental Incidents:** Usually characterized by massive scale (leaking 100,000 routes) and indiscriminateness.
- **Malicious Attacks:** Usually surgical (hijacking exactly one specific IP block, like a DNS server or a cryptocurrency exchange) and often timed with other events.

In Incident Response, we treat *impact* as the priority. Whether it's incompetence or malice, the mitigation (Route Filtering) is the same. Don't waste time guessing *Why*" until you have fixed the *What*.

1. The Analyst's Dilemma: Incompetence or Malice? When analyzing routing anomalies, we face a fundamental difficulty: identifying the intent behind the event. In the realm of routing protocols like BGP, the technical manifestation of a 'fat finger' configuration error often looks identical to an intentional attack. Because BGP was designed in an era of trust, it lacks inherent mechanisms to verify the intent of a neighbor's announcement, meaning a router will process a misconfiguration just as validly as a malicious hijack. The analyst must look at the scope and scale of the routing change to infer intent, rather than relying on the protocol itself to flag it.

2. Accidental Incidents: The Noise of Mass Leaks Accidental incidents are typically characterized by their massive scale and lack of discrimination. We often refer to these as 'Route Leaks,' where an Autonomous System (AS) mistakenly propagates routes it learned from one neighbor to another. A prime example occurred in 2021 when Vodafone Idea Ltd. accidentally announced over 30,000 BGP prefixes,. This wasn't a targeted strike against a specific bank or government; it was a configuration error that caused a 13x spike in inbound traffic, overwhelming their network. When you see thousands of unrelated prefixes from all over the world suddenly routing through a small ISP, you are likely looking at incompetence rather than a surgical strike.

3. Malicious Attacks: The Precision of Prefix Hijacking In contrast, malicious attacks are defined by precision. Attackers leverage the BGP rule that the 'longest prefix' (most specific route) always wins over a shorter AS path. This allows them to be surgical. Instead of leaking 30,000 routes, an attacker might announce exactly one specific /24 subnet belonging to a high-value target, such as a cryptocurrency exchange or a financial institution. For example, in 2017, the Russian ISP Rostelecom announced specific prefixes belonging to financial institutions, effectively redirecting that sensitive traffic through their network,. This type of 'more specific' route injection indicates a calculated effort to intercept traffic or perform a Man-in-the-Middle attack, rather than a simple configuration error.

4. Incident Response: Impact Over Attribution From an Incident Response (IR) perspective, we must prioritize impact over attribution. Whether the outage is caused by a malicious state actor or a typo by a junior administrator, the immediate mitigation strategy remains the same: you must filter the bad routes. IR teams should deploy access lists and route filters at the edge to block the illegitimate announcements,. It is dangerous to waste critical response time trying to guess the 'Why' (attribution) before fixing the 'What' (availability),. Recover the service first by filtering the route or contacting the upstream ISP, and save the forensic analysis of intent for the post-mortem phase.



Historical Incidents (Blast Radius Analysis)

- **YouTube vs. Pakistan Telecom (2008):**
 - *Intent:* Local censorship of a specific YouTube video.
 - *Mechanism:* Sub-prefix hijack (/24) leaked to upstream provider PCCW.
 - *Blast Radius:* Global YouTube outage for 2 hours.
- **The Lesson:** BGP has no borders. A configuration error in a small regional ISP can propagate globally in seconds.
- **Focus:** We study these not to blame, but to understand the *Fragility* of the trust model. One single point of failure can cascade into a systemic collapse.

1. The Incident: Censorship Gone Wrong To understand the real-world stakes of routing security, we look to the 2008 incident between YouTube and Pakistan Telecom. The intent here was purely local: the ISP attempted to censor a specific video by blocking access to it within Pakistan. However, to achieve this, they created a 'blackhole' route for YouTube's IP addresses within their internal network. The critical failure occurred when this internal configuration leaked out to their upstream provider (PCCW). Because the Border Gateway Protocol (BGP) was designed in an era of implicit trust between devices, the upstream provider accepted this advertisement without verifying the ISP's authority to announce those prefixes.

2. The Mechanism: Precision Beats Power The technical reason this local mistake became a global crisis lies in the BGP decision algorithm, specifically the **Longest Prefix Match**. BGP routers are programmed to prefer the most specific path to a destination. While YouTube was advertising a broader block of IP addresses to the internet, Pakistan Telecom announced a smaller, more specific /24 subnet. In the world of routing, precision overrules proximity. Consequently, routers around the world saw the /24 path as the 'better' route and immediately swerved traffic destined for YouTube into Pakistan's network, which was incapable of handling the global load.

3. The Lesson: A Borderless Blast Radius This incident illustrates that in BGP, there are no geographic borders; a configuration error in a regional ISP can propagate globally in seconds. The 'blast radius' here was a two-hour global outage for one of the world's largest platforms. We study this not to assign blame, but to highlight the fragility of the trust model we rely on. When a single entity can accidentally announce a route they do not own, and the rest of the world accepts it as fact, we face a systemic vulnerability where a single point of failure can cascade into a total collapse of service availability.



DNS Threat Landscape

1. **Integrity attacks:** DNS spoofing, cache poisoning, and control-plane compromise (Registrar takeover).
2. **Availability attacks:** DDoS campaigns against authoritative and recursive servers.
3. **DNS as an attacker tool:** Tunneling, covert channels, and infrastructure abuse.
4. **Domain shadowing and Fast-flux:** Techniques used to hide Command and Control (C2) infrastructure.
5. **Why DNS is attractive for malware:** High availability and trusted status in firewalls.

We just established that DNS is the 'Phonebook' of the internet. Now, imagine if someone could erase the phonebook, change the numbers inside it, or hide secret messages between the pages. That is the DNS Threat Landscape.

1. Integrity Attacks (The Lie):

"First, we look at **Integrity**. This is when the user asks for 'Bank of America' and the DNS server lies and sends them to a phishing site. We will cover **Cache Poisoning** (the technical exploit) and **Registrar Takeover** (the administrative exploit).

2. Availability Attacks (The Silence):

"Second, we cover **Availability**. If an attacker cannot trick you, they will try to silence you. We'll discuss how **DDoS** attacks target Authoritative servers to make domains vanish from the internet.

3. DNS as an Attacker Tool (The Tunnel):

"Here, DNS isn't the victim; it's the weapon. We will show you how attackers use **DNS Tunneling** to sneak credit card data out of your network right past your firewall, disguised as innocent DNS queries.

4. Domain Shadowing and Fast-Flux (The Camouflage):

"We will discuss advanced evasion techniques. **Domain Shadowing** is when attackers hack your GoDaddy account to create malware.your-legit-

business.com. **Fast-Flux** is when they rotate IP addresses every few seconds to outrun your blocklists.

5. Why DNS is Attractive for Malware:

"Finally, we answer the question: 'Why is it always DNS?' The answer is simple: It is the only protocol that is allowed through every firewall, monitored by few, and trusted by everyone.



Integrity Attacks (The Lie)

- **The Concept:** Integrity attacks compromise the accuracy of the DNS resolution. The goal is to redirect a user to a malicious site while they think they are visiting a legitimate one.
- **Mechanisms:**
 - **Cache Poisoning (Spoofing):** Injecting fake records into a recursive resolver's cache. The attacker wins the race condition" by replying faster than the real nameserver.
 - **Control-Plane Compromise (Registrar Takeover):** The attacker hacks the GoDaddy or Namecheap account of the victim and changes the nameservers (NS records) to point to their own infrastructure.

Poisoning is like pasting a fake sticker over a phonebook entry; Takeover is owning the printing press.

1. The Concept: Compromising the Map When we discuss **Integrity Attacks**, we are addressing a fundamental breakdown in trust. In the CIA triad (Confidentiality, Integrity, Availability), integrity ensures that data is correct and trustworthy. In the context of DNS, an integrity attack means the system is lying to the user. The user types a legitimate URL, but the DNS infrastructure returns a malicious IP address. This is particularly dangerous because the user's browser and behavior remain correct; the betrayal happens in the directory service itself, redirecting them to a malicious site or a 'cousin domain' designed to harvest credentials or distribute malware.

2. Mechanism 1: Cache Poisoning (The Race Condition) The first mechanism is **Cache Poisoning**, often referred to as DNS Spoofing. This attack exploits the stateless, unauthenticated nature of traditional DNS (Do53). When a recursive resolver sends a query, it opens a port and waits for an answer. It generally accepts the **first** response that matches the query's Transaction ID and port. Attackers exploit this by flooding the resolver with spoofed responses, attempting to guess the Transaction ID and win the 'race condition' against the legitimate authoritative server. If the attacker wins, the resolver caches the malicious IP address. For the duration of the Time To Live (TTL), every client relying on that resolver will be silently redirected to the attacker's infrastructure.

3. Mechanism 2: Control-Plane Compromise (The Takeover) The second mechanism is **Control-Plane Compromise**, or Registrar Takeover. This is not a protocol exploit but an administrative breach. If an attacker gains access to the domain owner's account at a registrar (like GoDaddy or Namecheap), they can modify the authoritative Name Server (NS) records. By changing these NS records, the attacker doesn't just inject a temporary lie; they officially designate their own server as the source of truth for the domain. As noted in the sources, 'If I own your DNS, I own you', this allows the attacker to redirect all web, email, and application traffic globally without needing to race packets.

4. The Analogy: Temporary vs. Absolute Control To summarize using the analogy on the slide: **Cache Poisoning** is like pasting a fake sticker over a phonebook entry. It affects only the people looking at that specific phonebook (the specific recursive resolver) and only until the sticker is removed (TTL expires). **Registrar Takeover**, however, is like owning the printing press. Because the attacker controls the authoritative source (the NS records), every phonebook printed thereafter includes the fake entry, granting the attacker total, persistent control over the domain's traffic.



Availability Attacks (The Silence)

- **The Goal:** Denial of Service (DoS). Stopping the resolution process so users cannot find the service.
- **Targets:**
 - **Against Authoritative Servers:** Flooding the specific nameservers hosting a domain (e.g DynDNS attack 2016). If the Authoritative server is down, the domain effectively disappears from the internet.
 - **Against Recursive Servers:** Flooding ISP resolvers to disrupt connectivity for a specific population of users.
- **Technique: DNS Amplification.** Attackers send small queries (with a spoofed victim IP) to Open Resolvers, which reply with massive responses to the victim, overwhelming their bandwidth.

1. The Goal: Creating The Silence" When we discuss availability attacks in the context of DNS, we are describing a scenario often referred to as 'The Silence.' In a traditional Denial of Service (DoS) attack, an adversary might crash a web server directly. However, in a DNS availability attack, the web server remains perfectly functional, but it becomes invisible. By disrupting the resolution process, the attacker ensures that users cannot translate the human-readable hostname into an IP address. The infrastructure is online, but for the end-user, the service effectively ceases to exist because the 'directory' is broken.

2. Targeting Authoritative Servers: The DynDNS Incident The slide highlights attacks against **Authoritative Nameservers**. If an attacker can overwhelm the specific nameservers that hold the 'zone file' for a domain, that domain disappears from the global internet. A prime example of this is the **2016 DynDNS attack**. This attack was orchestrated using the **Mirai botnet**, which comprised hundreds of thousands of compromised IoT devices like cameras and DVRs. The botnet hurled traffic peaking at **1.2 Terabits per second** at Dyn's infrastructure. Because major companies like Twitter, Netflix, and Reddit relied on Dyn for authoritative DNS, they became unreachable to millions of users, even though their own datacenters were operational. This proves that if you take down the directory assistance, you effectively take down the phone network.

3. Targeting Recursive Servers: Disrupting the ISP Alternatively, attackers may target **Recursive Servers**, such as those managed by an ISP. By flooding these resolvers, the attacker disrupts connectivity for a specific population of users, the ISP's customers. If a user's configured recursive resolver is overwhelmed and cannot process queries, the user cannot browse the web, send email, or access cloud services, effectively severing their link to the internet.

4. The Technique: DNS Amplification The mechanic driving these volumetric attacks is **DNS Amplification**. This technique exploits the connectionless nature of **UDP**, which allows an attacker to easily spoof their source IP address. The attacker sends a small DNS query (like a request for 'ANY' record or a large TXT record) to an open recursive resolver, but they falsify the return address to be the victim's IP. The open resolver processes the request and sends the response, which can be significantly larger than the query (e.g a 60-byte query resulting in a 512-byte or larger response), directly to the victim. This creates a multiplier effect; the attacker expends little bandwidth to generate a massive flood of traffic that overwhelms the victim's network capacity, resulting in a denial of service.



DNS as an Attacker Tool (The Tunnel)

- **The Shift:** Here, DNS is not the *target* or the *victim*; it is the **transport mechanism** for the attack.
- **Tunneling:** Encapsulating non-DNS traffic (like credit card data or SSH commands) inside DNS queries and responses.
 - **Exfiltration:** credit-card-data.evil.com -> The authoritative server logs the query and steals the data.
 - **C2 (Command & Control):** The response to the query contains instructions for the malware.
- **Why it works:** Firewalls usually block Port 80/443/22 for unknown destinations, but they almost always allow UDP/53 because blocking DNS breaks the network.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Shift: DNS as Transport, Not Target In this scenario, we must shift our perspective. We are not discussing attacks *against* the DNS infrastructure, such as cache poisoning or denial of service. Instead, we are looking at DNS as the **transport mechanism** for the attack itself. Attackers leverage the ubiquity of DNS to create a covert communication channel, effectively turning a core networking utility into a vehicle for malicious activity.

2. Tunneling: Encapsulating the Forbidden Technically, this is achieved through **Tunneling**. This process involves encapsulating non-DNS traffic, such as SSH sessions, credit card data, or shell commands, inside standard DNS queries and responses,. Tools like dnscat2 and Iodine allow adversaries to tunnel any data or protocol (even IPv4 traffic) over UDP port 53, bypassing standard network inspections,.

3. Exfiltration and C2: The Bi-Directional Flow This tunnel serves two primary functions: **Exfiltration** and **Command & Control (C2)**.

Exfiltration: To steal data, the malware on the compromised endpoint encodes sensitive information (using Base32 or Hex) and places it into the subdomain of a query, such as credit-card-data.evil.com,,. Because the attacker controls the authoritative name server for that domain, they receive the query, log it, and reconstruct the stolen data from the subdomain strings.

Command & Control: The communication is bidirectional. The attacker sends instructions back to the malware via the **DNS response** record (often using TXT, CNAME, or MX records),. This allows the adversary to issue commands to the victim machine without ever establishing a direct TCP connection,.

4. Why It Works: The Firewall Blind Spot This technique is incredibly effective because of how we configure our **Firewalls**. Security teams typically lock down egress traffic, blocking ports like TCP 80, 443, and 22 to unknown destinations to prevent direct internet access,. However, almost every network must allow **UDP port 53** outbound to function, as blocking DNS would break all name resolution and effectively halt network operations,. Furthermore, the compromised host does not even need direct internet access; as long as it can reach an internal recursive resolver that forwards queries to the internet, the tunnel will function, allowing the attacker to bypass even strict network segmentation.



Domain Shadowing and Fast-Flux (The Camouflage)

- **Domain Shadowing:**

- *Technique:* Attackers compromise a legitimate user's registrar account but don't change the main website. Instead, they create thousands of malicious subdomains (e.g login.pizza.com, update.pizza.com).
- *Benefit:* Leveraging the good reputation of the parent domain (pizza.com) to bypass reputation filters.

- **Fast-Flux:**

- *Technique:* Rapidly changing the A-record (IP address) of a malicious domain every few minutes.
- *Benefit:* It creates a moving target. By the time defenders block IP 1.2.3.4, the domain is already resolving to 5.6.7.8.

1. Domain Shadowing: Stealth Through Legitimacy First, we examine **Domain Shadowing**, a sophisticated evasion technique often categorized alongside advanced Domain Generation Algorithms (DGA++). In this scenario, attackers compromise the registrar credentials of a legitimate organization but, crucially, they do not disrupt the primary website. Instead, they create a multitude of malicious subdomains, such as login.pizza.com, piggybacking on the trust established by the parent domain.

"The security value here is camouflage. Security controls often allow traffic based on the reputation of the parent domain. By hiding within the subdomains of a reputable entity, attackers can bypass security infrastructure like Web Application Firewalls (WAFs) or reputation filters that would otherwise block a newly registered or suspicious domain,. This allows them to host phishing pages or command-and-control infrastructure behind a 'trusted' facade, making detection significantly harder for defenders who might overlook subdomains during routine analysis.

2. Fast-Flux: The Moving Target Next, we look at **Fast-Flux**, a technique designed to make malicious infrastructure resilient against blocking and identification. Fast-Flux works by associating a constantly changing set of IP addresses with a single domain name. This is technically achieved by setting

extremely low **Time-to-Live (TTL)** values for DNS 'A' records, often five minutes or less, and sometimes as low as 90 seconds,,.

"When a defender identifies and blocks a malicious IP (e.g 1.2.3.4), the attack infrastructure has likely already rotated to a new IP address (e.g 5.6.7.8) by the time the firewall rule is implemented. This rotation is often powered by a botnet of compromised hosts acting as proxies, relaying traffic to the actual backend Command and Control (C2) server, which remains hidden,. This 'moving target' dynamic forces defenders into a reactive game of 'whack-a-mole,' rendering static IP blocklisting largely ineffective.



Why DNS is Attractive for Malware

- **Universal Reach:** Every device, from a smartwatch to a mainframe, must speak DNS to function. It is the most ubiquitous protocol.
- **Trusted Status:** Firewalls treat DNS as "infrastructure traffic" rather than user traffic. It is rarely inspected as deeply as HTTP.
- **High Availability:** The DNS system is designed to be incredibly resilient. Malware authors piggyback on this resilience to ensure their Command & Control infrastructure stays online even when individual servers are taken down.

This is the ultimate "Living off the Land" attack: Attackers use the tools and protocols that are already present and essential, making their activity blend in with legitimate noise.

1. Universal Reach: The Ubiquitous Glue" First, we must understand that DNS is the 'glue' that holds the internet together. It is the most ubiquitous protocol because it serves as a precursor to almost every other form of network communication. From the smallest IoT smart device to the largest mainframe, nearly every networked asset must speak DNS to locate resources. This universality means that malware authors do not need to install custom communication tools; they can rely on the fact that the victim machine already has a built-in mechanism to query the outside world. Even in environments where we have locked down internal subnets to prevent direct internet access, we almost always allow those systems to resolve public domains to function, inadvertently providing a direct, bidirectional path for attackers.

2. Trusted Status: Infrastructure vs. User Traffic Security architectures often suffer from a blind spot regarding DNS. Firewalls and security appliances traditionally treat DNS as 'infrastructure traffic', essential utility data, rather than 'user traffic' that requires deep inspection. While organizations might rigorously filter HTTP or block outbound connections on non-standard ports, DNS traffic (typically UDP port 53) is often allowed outbound by default and is rarely ignored or inspected deeply. Attackers exploit this implicit trust. Because DNS requests are proxied through internal resolvers to the internet, a 'bucket chain' is formed

that bypasses firewall rules, allowing malware to tunnel data out of the network under the guise of simple address lookups.

3. High Availability: Weaponizing Resilience The Domain Name System was architected for extreme resilience and distributed availability. Malware authors piggyback on this robust infrastructure to ensure their Command and Control (C2) networks remain operational. By utilizing techniques like **Fast-Flux DNS**, attackers can rapidly rotate the IP addresses associated with their C2 domains, swapping them out every few minutes to evade IP-based blocklists. Furthermore, they employ **Domain Generation Algorithms (DGAs)** to programmatically generate thousands of domain names daily. As long as the attacker registers just one of these domains, the malware can successfully 'phone home,' making the infrastructure incredibly difficult to take down.

4. The Ultimate 'Living off the Land' Attack Finally, abusing DNS is the ultimate form of 'Living off the Land' (LOL). In the same way attackers use built-in Windows binaries (LOLBins) to execute code without triggering antivirus alarms, using DNS for C2 allows them to hide in plain sight using a protocol that is already present and essential for business operations. This traffic blends in with the millions of legitimate queries generated daily, making detection extremely difficult without specialized monitoring of query volume, entropy, or frequency. By using the network's own essential plumbing against it, attackers ensure their malicious activity looks just like legitimate noise.



Module 3 : Risk Reduction and Engineering Controls



Routing Security Controls and Standards

1. **Basic hygiene:** Prefix filtering (ingress/egress), IRR-based controls, and filter generation.
2. **RPKI fundamentals:** Resource certificates, trust anchors, and what RPKI actually validates.
3. **ROA operations:** Creation, maintenance, and understanding Valid/Invalid/Not-Found states.
4. **Operational risks:** Handling validation failures and dependency on external infrastructure.
5. **BGPsec reality:** The difficulty of path validation and barriers to adoption.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Now that we understand how fragile BGP is, how easily a 'fat finger' or a malicious actor can redirect traffic, we need to talk about how to fix it. This module covers the defense-in-depth strategy for routing.

1. Basic Hygiene (The First Line of Defense):

"We start with the basics: **Prefix Filtering**. This is the 'firewall' for your router. We will discuss the absolute minimum requirement: filtering what you accept from neighbors (Ingress) and, more importantly, what you send to them (Egress) to prevent accidental leaks.

"We'll also touch on **IRR (Internet Routing Registry)**, the legacy phonebook of the internet, and why it's useful but insufficient.

2. RPKI Fundamentals (The Gold Standard):

"Next, we move to the modern standard: **RPKI (Resource Public Key Infrastructure)**. This is 'SSL for BGP.' We will explain how we use digital certificates to cryptographically prove who owns an IP prefix.

"We'll break down the architecture: Trust Anchors (the RIRs like ARIN/RIPE) and the repositories that hold the data.

3. ROA Operations (The 'Signed Contract'):

"We will get hands-on with **ROAs (Route Origin Authorizations)**. This is the actual signed document that says 'ASN 65001 is allowed to announce my IP'.

"We'll learn the three validation states: **Valid** (Good), **Invalid** (Bad/Hijack), and **Not Found** (Unknown). Our goal is to move the internet from 'Unknown' to 'Valid'.

4. Operational Risks (The 'Fail Open' Debate):

"Security always carries risk. What happens if the RPKI repository goes down? Do you drop all traffic? We will discuss the **'Fail Open'** vs. **'Fail Closed'** trade-off and how to handle 'Invalid' routes without breaking legitimate connectivity.

5. BGPsec Reality (The Future?):

"Finally, we address the elephant in the room: **BGPsec**. RPKI validates the *Origin*, but BGPsec validates the *Path*. We will discuss why BGPsec is theoretically great but operationally difficult (hardware cost, complexity) and why it hasn't seen widespread adoption yet.



Basic Hygiene (Filtering & Sanitation)

- **The Concept:** Before we get to cryptography, we must do the basics. This is "Input Validation" for routers.
- **Prefix Filtering:**
 - **Rule:** Deny All" by default. Explicitly permit only what is known.
 - **Ingress:** Filter what neighbors send to you (Don't accept Google" from a small customer).
 - **Egress:** Filter what you send out (Don't leak your internal lab routes).
- **Max-Prefix Limits:**
 - **The Safety Valve:** A mechanism to shut down a BGP session if a neighbor sends too many routes (e.g >100).
 - **Why:** Prevents memory exhaustion and crashes caused by accidental route leaks.
- **Bogon Filtering:** Blocking private IPs (RFC 1918) and reserved space from entering the public routing table.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Concept: Input Validation for Routing Before we attempt to implement complex cryptographic solutions like BGPSEC, we must first establish a foundation of basic hygiene. Think of this as 'Input Validation' for our network infrastructure. Just as a developer sanitizes user input to prevent SQL injection, network engineers must sanitize routing advertisements. We are operating in an environment where we cannot blindly trust the data packets or routing updates sent to us, even from known peers. By enforcing strict hygiene, we prevent our routers from processing malicious or malformed data that could compromise the integrity of our routing table or the availability of our network.

2. Prefix Filtering: The Whitelist Approach To secure the control plane, we adopt a 'Deny All' posture by default. This means we explicitly permit only the specific IP prefixes we know are valid and block everything else.

Ingress Filtering: We must filter what our neighbors send to us. For example, a small downstream customer should never be advertising reachability to major networks like Google or Level3. If we do not filter these inbound announcements, we risk accepting a 'route hijack' or a leak, which could redirect global traffic through our limited infrastructure.

Egress Filtering: Equally important is filtering what we send out. We must ensure we are not leaking internal infrastructure routes, such as our lab networks

or private management subnets, to the public internet. Implementing strict egress filters prevents us from becoming the source of a route leak or accidentally exposing internal topology.

3. Max-Prefix Limits: The Circuit Breaker We implement **Max-Prefix Limits** as a critical safety valve against accidental misconfiguration. This mechanism acts like a circuit breaker: if a BGP peer that typically sends 50 routes suddenly attempts to send 5,000, the router automatically tears down the session. This is vital because accidental route leaks, such as the 2021 incident where a provider mistakenly announced over 30,000 prefixes, can cause massive spikes in traffic. Without this limit, a sudden flood of routes can exhaust the router's memory, causing the hardware to crash and resulting in a denial of service.

4. Bogon Filtering: Blocking the Unroutable Finally, we must apply **Bogon Filtering**. Bogons are IP prefixes that should never be routable on the public internet. This includes the private address spaces defined in **RFC 1918** (such as 10.0.0.0/8 and 192.168.0.0/16) and reserved subnets like 127.0.0.0/8,. Since these addresses have no validity on the public web, any packet claiming to originate from or destined to a bogon address at the network perimeter is illegitimate. Filtering these prevents private traffic from leaking out and stops attackers from using spoofed private addresses to bypass internal security controls.



RPKI Fundamentals (The Root of Trust)

- **The Problem:** BGP has no built-in authentication. It runs on a "handshake" trust model from 1989.
- **The Solution:** RPKI (Resource Public Key Infrastructure).
 - It applies Public Key Cryptography (PKI) to routing.
 - **Resource Certificates:** Digital certificates that prove permission to announce specific IP prefixes.
- **Mechanism:**
 - The **IP Owner** (e.g. Bank of America) cryptographically signs a statement authorizing an ASN to route their traffic.
 - **Trust Anchors:** The Regional Internet Registries (RIRs like ARIN, RIPE) act as the Root CAs.

This is SSL for BGP. Just as a browser checks the lock icon (Certificate) for a website, a router checks the RPKI signature for a route.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Problem: A Legacy of Implicit Trust To understand the necessity of RPKI, we must first look at the history of our routing protocols. The initial design of routing architectures, specifically BGP, was established decades ago during an era of implicit trust between devices. In this 'handshake' model, routers accepted route advertisements from peers without a rigorous, built-in mechanism to cryptographically verify the sender's authority to announce those specific IP prefixes. This lack of authentication makes it relatively simple for malicious actors or misconfigured devices to abuse the routing protocol, redirecting or intercepting traffic that was intended for a different destination.

2. The Solution: Applying PKI to Routing The industry's answer to this vulnerability is **Resource Public Key Infrastructure (RPKI)**. Conceptually, this solution applies the standards of Public Key Cryptography, which we rely on for secure communications, directly to internet routing. Just as we use digital signatures to prove that data has not been altered and originated from a specific key holder, RPKI uses cryptographic signatures to validate BGP announcements. This transition moves us from a model based on reputation and trust to one based on cryptographic verification.

3. Mechanism: Resource Certificates and Trust Anchors In this framework, we utilize **Resource Certificates**, which function similarly to the X.509 certificates

used in other PKI implementations. The IP Owner (such as a large financial institution) acts as a holder of a private key and cryptographically signs a statement authorizing a specific Autonomous System (ASN) to route their traffic. This establishes a verifiable chain of trust. At the top of this hierarchy are the **Trust Anchors**. In standard PKI, we have Root CAs that sign self-issued certificates to establish the root of trust. In the context of RPKI, the Regional Internet Registries (RIRs like ARIN and RIPE) fulfill this Root CA role, issuing certificates that validate the resources allocated to organizations.

4. The Analogy: SSL for BGP The slide uses the analogy 'SSL for BGP,' which is an accurate way to visualize the process. When a web browser connects to a secure site, it validates the server's identity by checking its certificate against a list of trusted authorities. RPKI performs this exact function for routers. Instead of blindly accepting a route, the router checks the RPKI signature (the 'lock icon') to verify that the route advertisement is authorized by the legitimate owner of the IP space, ensuring that the path we take through the internet is valid and secure.



ROA Operations (The States of Validity)

- **The Object: Route Origin Authorization (ROA).** This is the actual signed document.
 - Statement: ASN 65001 is allowed to announce 10.1.0.0/16.
- **Validation States:** When a router receives a BGP update, it checks the ROA and assigns one of three states:
 - **Valid:** The signature matches. (Allow the route) .
 - **Invalid:** The signature exists, but fails (Wrong ASN or Wrong Prefix Length). DROP IT.
 - **Not Found:** No signature exists. (Accept it, but with lower preference).
- **Operational Goal:** Moving the internet from "Not Found" to "Valid," and strictly enforcing "Drop Invalid".

1. The Object: Cryptographic Authority over Routing We begin with the fundamental object of this security architecture: the **Route Origin Authorization (ROA)**. In the context of the Resource Public Key Infrastructure (RPKI), the ROA serves as the digital 'title deed' for IP address space. As we discussed regarding BGPSEC and RPKI, this system uses certificates to extend a trust model to routing, moving us away from the legacy 'implicit trust' model where routers accept advertisements without verification. The ROA is a digitally signed document that explicitly authorizes a specific Autonomous System (AS) to announce a specific block of IP addresses, such as 'ASN 65001 is allowed to announce 10.1.0.0/16.'

2. Validation States: The Router's Decision Logic When a router receives a BGP update, it performs Route Origin Validation (ROV) by comparing the update against the cached ROAs. This process results in one of three deterministic states:

Valid: The update matches a valid ROA. The cryptographic signature is correct, the ASN matches the authorized origin, and the prefix length is within the permitted range. The router can safely **allow** this route, confident it is authorized.

Invalid: This is the critical defense mechanism against the hijacking incidents we see in the wild, such as the Vodafone or Rostelecom events. An 'Invalid' state

means a ROA exists for this prefix, but the BGP update contradicts it, either the announcing ASN is wrong (a potential hijack) or the prefix is too specific (violating the Max Length). The operational guidance here is strict: **DROP IT**. If we do not drop invalid routes, we remain vulnerable to route leaks and malicious redirection.

Not Found: This state reflects the current transitional reality of the internet. It means no ROA exists for the prefix. Because RPKI deployment is not yet universal, we cannot simply drop these packets without breaking connectivity to a large portion of the internet. We accept these routes, but often with a lower local preference than 'Valid' routes, treating them as 'unverified.'

3. The Operational Goal: Enforcing Trust Our ultimate operational goal is to shift the global routing table from a state of 'Not Found' to 'Valid.' Currently, reliance on manual access lists or passive monitoring services like BGPMON is necessary but reactive. By populating ROAs and strictly enforcing 'Drop Invalid,' we proactively prevent configuration errors and malicious hijacks from propagating, securing the control plane against the fragility inherent in BGP's original design.



Operational Risks & MANRS

- **The Dependency:** RPKI introduces a dependency on external infrastructure (the RPKI repositories).
 - Risk: If the repository goes down, do you fail open or fail closed? (Standard practice is Fail Open" / revert to Not Found" state).
- **Coordination (MANRS):**
 - **Mutually Agreed Norms for Routing Security:** A global initiative to force operators to talk to each other.
 - **Key Requirement:** Maintain accurate contact info in PeeringDB so network engineers can call each other during a hijack.
- **"The Herd Immunity" Concept:** Routing security only works if everyone does it. One secure network in a sea of insecure networks is still vulnerable to leaks.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The RPKI Dependency: Balancing Security and Availability As we move toward securing BGP with Resource Public Key Infrastructure (RPKI), we introduce a critical dependency on external infrastructure: the RPKI repositories. BGPSEC and RPKI aim to replace the legacy 'trust your neighbor' model with a cryptographic trust model using certificates,. However, this introduces a new operational risk. If the repository hosting these certificates goes offline, the network operator faces a binary choice: fail open or fail closed. Standard practice dictates we 'Fail Open' (reverting to a 'Not Found' state). If we failed closed, a repository outage would result in a self-inflicted Denial of Service, dropping legitimate traffic because the validator couldn't be reached. We must prioritize availability alongside integrity in this architecture.

2. Coordination and the Human" Protocol Technology alone cannot solve routing security; it requires human coordination. The slide highlights MANRS (Mutually Agreed Norms for Routing Security) and PeeringDB as essential tools. This aligns with the fundamental defense strategy of involving your ISP in incident response planning. When a routing incident occurs, automation often falls short, and you need to contact a human engineer immediately. Maintaining accurate contact information in databases like PeeringDB ensures that when a route hijack is detected, the victim can quickly reach the upstream provider or the

offending network to mitigate the damage. Without this coordination, the 'trust your neighbor' design of routing protocols becomes a liability rather than an asset.

3. Herd Immunity: The Collective Defense Finally, we must understand that routing security operates on the concept of 'Herd Immunity.' BGP incidents often have a blast radius that extends far beyond the source. For example, when Vodafone Idea Ltd. accidentally announced over 30,000 prefixes, it caused a massive spike in inbound traffic that affected networks globally. This illustrates that a single insecure network can compromise the stability of the entire ecosystem. One secure network cannot protect itself entirely if its neighbors accept and propagate invalid routes. Therefore, preventing route leaks and hijacks requires collective adoption of security norms; we are only as secure as the weakest network in our routing path.



BGPsec Reality (Path Validation vs. Origin Validation)

- **The Limitation of RPKI:** RPKI only validates the *Origin* (who started the route). It does *not* validate the *Path* (the hop-by-hop journey).
 - *Attack Vector:* A malicious AS can still perform an "AS-Path Replay" attack even with RPKI.
- **The Solution (BGPsec):** A standard to cryptographically sign *every single hop* in the path.
- **The Reality:**
 - **Adoption Barrier:** It is computationally expensive. Verifying signatures at every hop slows down convergence and requires hardware upgrades.
 - **Current State:** RPKI (Origin Validation) is the current industry standard. BGPsec (Path Validation) is the future goal, but deployment is slow due to complexity.

Distinguish clearly between "Who sent this?" (RPKI - Solved) and "How did it get here?" (BGPsec - Hard).

1. The Limitation of RPKI: Origin vs. Path As we examine the current state of routing security, it is critical to distinguish between validating the *source* and validating the *journey*. Currently, the industry relies heavily on Resource Public Key Infrastructure (RPKI). While beneficial, RPKI has a distinct limitation: it only validates the Origin, verifying that a specific Autonomous System (AS) is authorized to announce a specific IP prefix. It does not validate the AS Path, or the hop-by-hop chain of custody that the route advertisement takes across the internet. This leaves the network vulnerable to 'AS-Path Replay' attacks, where a malicious actor propagates a valid origin announcement but manipulates the path attributes to redirect traffic through their own infrastructure.

2. The Solution: Cryptographic Path Assurance The proposed solution to close this gap is **BGPsec**. BGPsec is designed to extend a Certificate Authority (CA) trust model directly into the routing protocol. In a BGPsec-enabled world, routers would use certificates to cryptographically authenticate the peering relationship at every hop. This means that as a route advertisement moves from AS to AS, each router adds a digital signature, creating an unbroken, verifiable chain of custody. This answers the critical question, 'How did this packet get here?' rather than just 'Who sent it?'

3. The Reality: Deployment Barriers Despite the theoretical security benefits,

the reality is that BGPsec has not yet been widely deployed. The primary barrier is cost, both computational and financial. Verifying digital signatures at every single hop requires significant processing power, which slows down route convergence and often necessitates expensive hardware upgrades for routers to handle the load. Furthermore, the rise of automated certificate issuance (such as Let's Encrypt) has introduced new concerns; if a malicious router can easily automate the acquisition of valid certificates, the trust model BGPsec relies upon may be undermined. Consequently, while RPKI remains the current standard for Origin Validation, BGPsec and full Path Validation remain a complex, future goal that the industry is struggling to implement at scale.



Secure Network Architecture for Routing

1. **Multi-ISP design:** Path diversity and failure isolation strategies.
2. **Anycast vs. Unicast:** Benefits, risks, and operational implications.
3. **Internet edge design:** Segmentation and blast radius containment.
4. **Control plane protection:** Rate limiting and Router CPU defense mechanisms.
5. **Infrastructure hardening:** Access control, configuration management, and plane separation (Management/Control/Forwarding).

We've learned the commands to secure a BGP session. Now, we need to zoom out. How do we arrange these routers physically and logically so that a single failure doesn't take down the entire enterprise? That is **Network Architecture**.

1. Multi-ISP Design (Diversity & Resiliency):

"First, we tackle the single point of failure. We'll discuss **Multi-Homing**, connecting to multiple upstream providers (like Level3 and Verizon) to ensure that if one ISP has a meltdown, your business stays online.

2. Anycast vs. Unicast (The DDoS Shield):

"Next, we look at **Anycast**. This is how the giants (Google, Cloudflare) survive. We will explain how announcing the same IP address from 50 different cities simultaneously creates a natural 'Distributed Scrubbing Center' that absorbs DDoS attacks globally.

3. Internet Edge Design (Segmentation):

"We will apply the 'Blast Radius' concept to hardware. We'll discuss why your **Edge Routers** (the wild west) must be segmented from your **Core Routers** (the internal sanctuary), ensuring that a compromise at the border doesn't spread laterally.

4. Control Plane Protection (CoPP):

"This is a critical defense mechanism: Protecting the Brain. We will show you

how to configure **Control Plane Policing**. This ensures that even if your router is flooded with gigabits of trash traffic (the Data Plane), the CPU (the Control Plane) stays alive to process SSH and BGP.

5. Infrastructure Hardening (The Management Plane):

"Finally, we lock down the keys to the kingdom. We'll cover **Infrastructure Hardening**, separating the Management Plane (SSH) from the User Plane, using Out-of-Band (OOB) networks, and treating router configuration files like software code.



Multi-ISP Design (Diversity & Resiliency)

- **The Concept:** Avoiding Single Points of Failure (SPoF) by connecting to multiple upstream providers (e.g Level3 and Verizon).
- **Path Diversity:**
 - **Physical:** Ensure fibers leave the building via different conduits (North/South) to prevent a single backhoe fade from killing both lines.
 - **Logical:** Ensure your providers don't share the same upstream dependencies (e.g both relying on the same sub-sea cable).
- **Failure Isolation:**
 - If ISP A has a routing meltdown (Route Leak), BGP automatically shifts traffic to ISP B.

Resilience is a security feature. Availability is one of the three pillars of the CIA Triad. A network that is down" is indistinguishable from a network that is DoS'd.

1. Resilience as a Security Function When we discuss Multi-ISP Design, we are fundamentally addressing the 'Availability' pillar of the CIA Triad, Confidentiality, Integrity, and Availability. As security professionals, we must treat resilience as a core security feature because, to an end-user, a network taken offline by an accidental fiber cut is indistinguishable from one taken offline by a malicious Denial-of-Service (DoS) attack; both result in the inability to access resources. Therefore, redundancy designs are a cornerstone of how network infrastructure delivers on the availability requirement.

2. Physical and Logical Diversity To mitigate Single Points of Failure (SPoF), we must employ redundancy at the WAN and internet access levels. This requires two forms of diversity:

Physical Diversity: We must ensure cables leave the building via different physical paths (e.g North and South conduits). Defenses against Layer 1 (Physical) attacks or failures rely heavily on redundancy to maintain operations during power loss, natural disasters, or physical line destruction.

Logical Diversity: We must verify that our upstream providers do not share the same underlying dependencies, such as a single sub-sea cable. Best practices dictate planning failover connectivity using different network providers to ensure that a failure in one provider's infrastructure does not cascade into a total

outage.

3. Failure Isolation via Routing Finally, we rely on routing protocols like BGP to handle **Failure Isolation**. In a resilient network, routing protocols are designed to dynamically discover the next best path during a network failure. If ISP A experiences a routing meltdown or a route leak, BGP detects the failure and automatically shifts traffic to ISP B, routing around the trouble spot to ensure the internet continues to deliver reliable connectivity.



Anycast vs. Unicast (The DDoS Shield)

- **Unicast (Traditional):** One IP address = One physical server.
 - Risk: If 1 million attackers target that server, it dies.
- **Anycast (Modern):** One IP address = Hundreds of physical servers globally.
 - Mechanism: BGP announces the same /24 prefix from Singapore, London, and NY.
 - Routing: The internet routes users to the topologically closest node.
- **Security Benefit:** It acts as a Distributed Scrubbing Center. Attack traffic is diluted across the global fleet rather than overwhelming a single data center.
- **Operational Trade-off:** Troubleshooting is harder because you don't always know which server a user is hitting.

1. Unicast: The Fragility of a Single Target To appreciate the resilience of modern routing, we must first look at the traditional **Unicast** model. In this setup, a specific IP address corresponds to a single, specific interface or destination host on the network,. While this simple one-to-one relationship facilitates established connections like TCP, it introduces a critical point of fragility. If a malicious actor creates a volumetric attack, such as a botnet flooding the target with traffic, that single server's bandwidth or processing capacity is easily overwhelmed, resulting in a complete service outage,.

2. Anycast: The Global Mirror Anycast fundamentally changes this topology. Instead of an IP address representing one box in one rack, an anycast IP address is announced by multiple endpoints simultaneously, potentially hundreds of physical servers across global data centers. The mechanism driving this is the Border Gateway Protocol (BGP). By announcing the same IP prefix (such as a /24) from diverse locations like Singapore, London, and New York, we essentially 'clone' the destination address across the map,.

3. Routing and the Distributed Scrubbing Center" When a user (or an attacker) sends a packet to this Anycast IP, the internet's routing infrastructure automatically directs that packet to the topologically closest node offering the service,. This architecture provides a massive security benefit during a DDoS

attack. Instead of a 'tsunami' of traffic crashing into a single data center, the attack traffic is logically split and absorbed by the closest regional nodes. This effectively turns the entire global network into a **Distributed Scrubbing Center**, diluting the attack volume to manageable levels at each site and preventing a total collapse of the service,.

4. The Operational Trade-off However, this resilience comes with an operational cost: complexity in troubleshooting. Because the destination is determined dynamically by routing protocols, different endpoints serve different users based on their network location. This makes it difficult for administrators to pinpoint exactly which physical server a specific user is communicating with when errors occur, complicating forensic analysis and debugging efforts compared to the deterministic nature of Unicast.



Internet Edge Design (Segmentation)

- **The Architecture:** Defining the Demilitarized Zone" (DMZ) for routing.
- **Blast Radius Containment:**
 - The **Edge Routers** (that talk to the ISP) should be physically separate from the **Core Routers** (that talk to the Database).
 - **Why:** If an attacker compromises the Edge Router via a BGP exploit, they are still stuck in the DMZ.
- **Segmentation Strategy:**
 - Use Virtual Routing and Forwarding (VRFs) to keep Management traffic (SSH/SNMP) completely isolated from User traffic.
 - Never allow the Public Internet table to leak into the Internal LAN.

1. The Architecture: Defining the DMZ We begin by establishing the architecture of the Internet Edge through the 'Demilitarized Zone' or DMZ. This network segment functions as a buffer zone between the untrusted public internet and our trusted internal network. By placing external-facing systems, such as web servers, email relays, and DNS servers, within the DMZ, we ensure that incoming traffic is terminated and inspected at a perimeter 'choke point' before it can interact with internal assets,. This architecture is critical because it forces an adversary to breach multiple distinct layers of security controls, external firewalls, the DMZ itself, and internal firewalls, before they can reach the organization's most sensitive data.

2. Blast Radius Containment: Edge vs. Core A primary goal of this design is 'Blast Radius Containment.' To achieve this, we must enforce strict separation between Edge Routers, which speak BGP to the ISP, and Core Routers, which handle sensitive internal traffic like database connections,. If an attacker compromises an Edge Router, perhaps through a BGP hijack or a vulnerability in the routing service, this segmentation ensures they remain contained within the DMZ. By designing the network this way, we minimize the impact of a security control failure; the compromised edge device has no direct route or permission to pivot into the internal core to access the 'crown jewels,' effectively trapping the

attacker in a segment where they can do the least amount of damage,.

3. Segmentation Strategy: VRFs and Route Isolation To implement this isolation technically, we utilize **Virtual Routing and Forwarding (VRF)**. VRF technology allows a single router to maintain multiple, independent routing tables that are completely isolated from one another, functioning for Layer 3 traffic much like VLANs do for Layer 2. We use VRFs to segregate **Management traffic** (such as SSH and SNMP) from standard **User traffic**. This ensures that administrative interfaces are not accessible from the user data plane; even if the user network is flooded or compromised, the management channel remains secure and accessible to administrators,. Finally, we must strictly enforce a rule to never allow the Public Internet routing table to leak into the Internal LAN. We accomplish this by implementing strict route filtering and ingress/egress filtering at the border, ensuring that internal private addresses are never routable or visible to the public internet.



Control Plane Protection (CoPP)

- **The Brain vs. The Muscle:**
 - **Data Plane:** Fast ASICs forwarding user packets.
 - **Control Plane:** The slow CPU processing BGP updates and SSH logins.
- **The Attack:** A volumetric DDoS (ICMP flood) targeting the router's interface IP. This overwhelms the CPU, causing BGP sessions to drop.
- **The Defense:** Control Plane Policing (CoPP):
 - A QoS policy applied to the CPU interface.
 - Rule: Rate limit ICMP to 10 packets/sec. Rate limit SSH to 5 packets/sec. Guarantee bandwidth for BGP (Port 179).
- **Analogy:** CoPP is the VIP rope at a club. It ensures the Bouncer (CPU) doesn't get mobbed by the crowd, so he can keep doing his job.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Brain vs. The Muscle: CPU vs. ASIC To understand Control Plane Protection, we must first distinguish between the two primary functions of a network device: the **Data Plane** and the **Control Plane**. The Data Plane is the 'muscle,' responsible for forwarding user data passing *through* the router. This is handled by specialized hardware (ASICs) designed for high-speed throughput. In contrast, the Control Plane is the 'brain,' it handles traffic destined *to* the router itself, such as SSH management sessions, logging, and routing updates (like BGP or OSPF). The problem is that while ASICs are fast, the general-purpose CPU managing the Control Plane is relatively slow. As noted in the sources, forcing high-throughput interfaces into 'process mode', where the CPU must inspect packet headers, can significantly impact performance.

2. The Attack: Overwhelming the CPU Attackers exploit this architectural difference by launching volumetric Denial of Service (DoS) attacks targeting the router's interface directly. While the ASICs could easily forward a flood of ICMP or UDP packets, the router's CPU cannot process them if they are destined for its own IP address. By flooding the Control Plane with junk traffic (like an ICMP flood), the attacker exhausts the CPU's resources. When the CPU is overwhelmed, it fails to process critical keepalive messages for routing protocols. This causes BGP sessions to time out and drop, disconnecting the

router from the network even though the physical links are not saturated.

3. The Defense: Control Plane Policing (QoS for the CPU) The defense against this is **Control Plane Policing (CoPP)**. CoPP applies a Quality of Service (QoS) policy to the virtual interface feeding the CPU. It acts as a filter that strictly regulates how much traffic can reach the processor. We configure rules to rate-limit non-essential traffic, for example, capping ICMP traffic at 10 packets per second and SSH login attempts at 5 packets per second. Crucially, we explicitly guarantee bandwidth for essential infrastructure traffic, such as BGP (TCP port 179), ensuring routing updates are always processed even during an attack,. This is similar to the 'Input Validation' concepts we apply to routing updates; we must validate and limit the volume of traffic interacting with the network brain,.

4. The Analogy: The VIP Rope To visualize this, think of CoPP as the VIP rope at a busy nightclub. The CPU is the Bouncer. If the crowd (data traffic) is allowed to mob the bouncer all at once, he cannot perform his job of checking IDs (processing routing updates), and the club shuts down. CoPP places a velvet rope in front of the bouncer, allowing only a manageable stream of people to approach him at one time. This ensures he stays functional and secure, regardless of how chaotic the street outside becomes.



Infrastructure Hardening (The Management Plane)

- **Access Control:**
 - **MFA:** Require Multi-Factor Authentication for all SSH access to routers.
 - **Jump Box:** No direct SSH from the internet. Engineers must VPN into a Bastion Host first.
- **Configuration Management:**
 - Infrastructure as Code (IaC): Treat router configs like software code (Git).
 - **Benefit:** Prevents "Fat Finger" errors (Route Leaks) by requiring peer review before deployment.
- **Plane Separation:**
 - Use a dedicated Out-of-Band (OOB) management network (physically separate cables) to access console ports.
 - **Why:** If the production network melts down (DDoS), you need a "Back Door" to get in and fix it.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. Access Control: Fortifying the Entry Points We begin by locking down the 'front door' of our infrastructure. Allowing direct SSH or RDP access from the public internet is a critical vulnerability that exposes management interfaces to brute-force attacks and exploitation. Instead, we must mandate the use of a **Bastion Host** (or Jump Box). This serves as a fortified, single point of entry that acts as a buffer between the untrusted internet and our private management subnet. By funneling administrative traffic through this gateway, we can enforce stricter access policies and logging. Furthermore, relying on passwords alone is no longer sufficient. We must enforce **Multi-Factor Authentication (MFA)** for all privileged access. MFA mitigates the risk of credential theft by requiring a second form of verification, ensuring that even if a password is compromised, the attacker cannot gain access to the router or switch.

2. Configuration Management: Eliminating the 'Fat Finger' Next, we shift our operational mindset by adopting **Infrastructure as Code (IaC)**. Instead of manually typing commands into a CLI, where a single typo can take down a network, we treat configurations as software code stored in version control systems like Git. This approach allows us to define the desired state of our infrastructure programmatically, preventing 'configuration drift' where manual, ad-hoc changes accumulate and create inconsistencies. The primary security

benefit here is the ability to enforce **peer review** and automated testing before any change is deployed. By validating configurations in a test environment and requiring approval, we significantly reduce the risk of deploying misconfigurations or 'fat finger' errors that could lead to route leaks or security exposures.

3. Plane Separation: The Hail Mary" Network Finally, we must implement **Plane Separation** through **Out-of-Band (OOB) Management**. This involves creating a dedicated management network that is physically or logically isolated from the production data plane. The necessity of this becomes clear during a crisis: if your production network is saturated by a **Distributed Denial of Service (DDoS)** attack, in-band management protocols like SSH will fail because the bandwidth is consumed by malicious traffic. An OOB network acts as a 'back door' or 'spy network,' ensuring that administrators retain console access to devices even when the production network is melting down, allowing them to apply filters and remediate the attack without being locked out.



Secure DNS Operations

1. **Secure resolver architecture:** Design principles for resilience.
2. **Rate limiting and RPZ:** Using Response Policy Zones for filtering.
3. **Sinkholing and intelligence feeds:** diverting malicious traffic for analysis.
4. **Zone transfer protection:** Securing the replication of zone data.
5. **Registrar and domain lifecycle security:** Protecting the ownership of the domain.
6. **Logging and forensic visibility:** What to capture for post-incident analysis.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We know how DNS works, and we know how attackers break it. Now, we are going to learn how to *run* it. This is the difference between designing a secure building and actually guarding the doors.

1. Secure Resolver Architecture (The Golden Rule):

"First, we start with the architecture. We will discuss the 'Golden Rule' of DNS operations: **Never mix roles**. We'll explain why Authoritative servers and Recursive Resolvers must be on separate hardware to prevent creating 'Open Resolvers' that fuel DDoS attacks.

2. Rate Limiting and RPZ (Active Defense):

"Next, we move to **Active Defense**. We will introduce **RPZ (Response Policy Zones)**. This is where we stop treating DNS as a phonebook and start treating it as a **Firewall**. We'll show you how to block phishing domains before the connection is even made.

3. Sinkholing and Intelligence Feeds (turning the tables):

"Blocking is good; identifying is better. We'll discuss **Sinkholing**. Instead of just dropping a malicious query, we lie to the malware and redirect it to a sensor. This allows you to instantly identify exactly which laptop in your office is infected.

4. Zone Transfer Protection (Locking the Map):

"We'll look at **AXFR** (Zone Transfers). Your Zone file is a map of your entire

network. We will show you how to use **TSIG keys** and ACLs to ensure you aren't handing that map to attackers for reconnaissance.

5. Registrar and Domain Lifecycle Security:

"It doesn't matter how secure your servers are if someone guesses your GoDaddy password. We will discuss **Domain Shadowing** and why securing the administrative 'Control Panel' at your Registrar is just as critical as patching your BIND server.

6. Logging and Forensic Visibility:

"Finally, we end with **Visibility**. When an incident happens, you need data. We'll discuss why standard logs aren't enough and how to capture the specific metadata (like Query Types and Responses) needed for forensics.



Secure Resolver Architecture (Resilience by Design)

- **The Golden Rule:** Never mix roles.
 - Authoritative Servers: Answer for your domains (e.g mycompany.com) to the world.
 - Recursive Resolvers: Answer for others' domains (e.g google.com) for your employees.
 - The Risk: If you mix them, you become an **Open Resolver**, enabling DDoS Amplification attacks against third parties.
- Access Control Lists (ACLs):
 - Configure your Recursive Resolvers to accept queries only from internal subnets (e.g allow-recursion { 10.0.0.0/8; };
 - Any query from the public internet should be dropped or refused.

Treat DNS servers like mail servers. You wouldn't let the entire internet use your Exchange server as an open relay. Don't let them use your DNS server as an open resolver.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Golden Rule: Separation of Roles We start with the fundamental principle of DNS security architecture: the separation of duties. We must strictly distinguish between **Authoritative Servers** and **Recursive Resolvers**. An authoritative server acts as the source of truth for your own domains, providing answers to the world about resources you own, such as 'mycompany.com'. In contrast, a recursive resolver functions like a librarian for your internal employees; it performs the legwork of querying the internet's hierarchy (Root, TLD, etc.) to find the IP addresses for external domains like 'google.com'. Best practice dictates that these two roles should be dedicated to separate servers to maintain a 'Split Namespace,' ensuring internal and external data remain distinct.

2. The Risk: The Open Resolver Threat Why is mixing these roles dangerous? Because if a server is configured to answer recursive queries for anyone on the internet, it becomes an **Open Resolver**. Attackers exploit this configuration to launch **DDoS Amplification attacks**. In this scenario, an attacker spoofs the IP address of a victim and sends a small DNS query to your open resolver. Your server then processes the request and sends a significantly larger response to the victim's IP address. By recruiting thousands of such open resolvers, attackers can generate massive volumes of traffic to overwhelm a target, making your

infrastructure an unwitting accomplice in the attack.

3. Access Control Lists (ACLs): Enforcing Boundaries To prevent this abuse, we must implement strict **Access Control Lists (ACLs)**. We must configure our recursive resolvers to accept queries *only* from trusted internal subnets, such as those within the RFC 1918 private address space (e.g 10.0.0.0/8). Any query originating from the public internet attempting to use your recursive service should be explicitly dropped or refused. This is often part of a 'Split DNS' strategy, which also prevents external attackers from mapping your internal network by querying your internal resolvers.

4. The Analogy: DNS as an Open Relay Finally, treat your DNS infrastructure with the same rigor you apply to your email infrastructure. In the world of SMTP, we learned long ago that leaving a mail server configured as an '**open relay**' allows spammers to hijack your resources to send junk mail, leading to blacklisting and reputational damage. We must apply that same logic here: just as you wouldn't allow the entire internet to route spam through your Exchange server, you must not allow the internet to route denial-of-service traffic through your DNS servers.



Rate Limiting & Response Policy Zones (RPZ)

- **The Concept:** Turning the DNS server into a Firewall.
- **Response Policy Zones (RPZ):** An open standard that allows you to subscribe to Blocklists" just like you subscribe to antivirus updates.
- **Mechanism:**
 - The server checks every query against the RPZ zone file.
 - **Triggers:** QNAME (Domain Name), Client-IP (Who is asking?), or RPZ-IP (The resolved malicious IP).
 - **Actions:** Return NXDOMAIN (Block) or CNAME (Redirect to Sinkhole).
- **Rate Limiting:** Protects the server itself from being overwhelmed by internal infected hosts performing DoS or brute-force scanning.

1. The Concept: Transforming DNS into a Security Control We begin by shifting our perspective of the DNS server. Traditionally, it is viewed merely as a utility for directory resolution, but with **Response Policy Zones (RPZ)**, we transform the DNS server into a proactive **DNS Firewall**. This approach allows us to block access to malicious infrastructure, such as phishing sites or Command and Control (C2) servers, at the earliest possible stage of the kill chain: the lookup request. By filtering at this layer, we can mitigate threats even for encrypted traffic or devices where we cannot install endpoint agents.

2. Response Policy Zones (RPZ): The Firewall" Mechanism RPZ functions as an open standard that allows DNS administrators to overlay custom information on top of the global DNS to provide alternate responses, essentially, 'lying' to the client for their own protection. Think of an RPZ zone file as a subscription feed, similar to how you subscribe to antivirus signatures or reputation blocklists. These feeds contain known malicious global identifiers, such as hostnames, domain names, or IP addresses associated with cybercrime. When a server configured with RPZ receives a query, it checks the request against these subscribed 'blocklists' in near-real-time.

3. Triggers and Actions: How the Server Responds The mechanism works by evaluating specific triggers within the query. The server checks the **QNAME** (the

domain being requested), the **Client-IP** (identifying the infected internal host), or the **RPZ-IP** (the resolved IP address of the malicious destination),. Based on these triggers, the server takes a defined action:

NXDOMAIN: The server returns a 'Non-Existent Domain' response, effectively blocking the connection by telling the client the target does not exist.

Sinkhole (CNAME/A Record): Instead of a simple block, we can redirect the client to a **Sinkhole**, a server controlled by the security team,. This allows the traffic to leave the network to a safe destination, enabling defenders to identify exactly which internal machine is infected and attempting to phone home.

4. Rate Limiting: Protecting Availability Finally, we must implement **Rate Limiting** to protect the DNS infrastructure itself. Infected internal hosts often perform noisy brute-force scanning or participate in Denial of Service (DoS) attacks, which can exhaust the server's CPU and memory resources,. By enforcing limits on the number of requests a single client IP can make per second, we prevent a single compromised host from overwhelming the DNS service, ensuring availability for legitimate users while flagging the anomalous volume for investigation.



Sinkholing & Intelligence Feeds (Active Defense)

- The Architecture: Instead of just blocking a malicious domain (which gives the attacker a silent fail), we redirect it.
 - Malware Query: beacon.evil.com
 - DNS Response: 192.168.100.100 (Your Sinkhole Server).
- The Value:
 - The infected machine connects to your Sinkhole.
 - You capture the source IP (Attribution) and the payload (Intent).
- Intelligence Feeds:
 - Use external feeds (Spamhaus, SURBL) to populate your RPZ automatically via Zone Transfers.
 - This provides "Herd Immunity", if a new C2 domain is spotted in Europe, your network in Asia blocks it instantly.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Architecture: From Silent Failure to Active Deception We must shift our mindset from passive blocking to active defense. When a firewall simply blocks a connection or a DNS server returns an 'NXDOMAIN' (Non-Existent Domain) error, we force the attacker's malware into a 'silent fail' state. The malware knows it failed, but we often gain little insight into *why* it was calling out or *what* it was trying to do. By implementing **Sinkholing**, we change the architecture to one of deception. Instead of telling the malware the door is locked, we open a fake door, redirecting the traffic to infrastructure we control, often referred to as a sinkhole server or honeypot,.

2. The Mechanism: Lying for Attribution and Intent This process relies on the DNS server 'lying' to the client. When an infected machine queries a malicious domain like beacon.evil.com, our DNS server, configured with a sinkhole list, intercepts the request. Instead of resolving to the real Command and Control (C2) server, it returns the IP address of our internal Sinkhole Server (e.g 192.168.100.100),. The value here is immediate visibility. The infected machine, believing it has found its home, connects to our sinkhole. This connection provides us with two critical pieces of intelligence:

Attribution (Source IP): We instantly identify exactly which internal machine is infected because it is the one establishing the connection to our sinkhole.

Intent (Payload): By accepting the connection (e.g completing the TCP handshake or accepting an HTTP request), we can capture the payload. This allows us to analyze the 'callbacks' to diagnose the malware family and understand its objectives, rather than just guessing based on a blocked DNS log.

3. Intelligence Feeds and RPZ: Automating Herd Immunity To scale this defense, we utilize **Response Policy Zones (RPZ)**, which effectively act as a 'DNS Firewall.' RPZ allows us to overlay custom information, our 'lies', on top of the global DNS data. We populate these zones automatically using external intelligence feeds from trusted providers like **Spamhaus** or SURBL,. This architecture creates 'Herd Immunity.' We are no longer defending in isolation; we are leveraging the collective experience of the security community. If a new C2 domain is detected and added to a feed in Europe, your DNS servers updates via a zone transfer mechanism, similar to how secondary DNS servers sync with a primary, and your network in Asia begins blocking or sinkholing that domain instantly,,. This moves us from a reactive posture to a proactive, globally informed defense.



Zone Transfer Protection (Locking the Map)

- The Vulnerability: AXFR (Full Zone Transfer).
 - By default, many DNS servers will give their entire database to anyone who asks.
 - Attacker Command: `dig axfr target.com @ns1.target.com`.
- The Impact (Reconnaissance):
 - The attacker gets a complete list of every server, subnet, and internal resource (e.g `secret-project.target.com`, `dev-db.target.com`).
- The Fix:
 - Restrict Zone Transfers strictly to trusted Secondary Nameserver IPs only.
 - Use TSIG (Transaction Signature) keys to cryptographically authenticate the transfer.

1. The Vulnerability: The Open Book" of AXFR We must address a critical misconfiguration in DNS known as the full zone transfer, or AXFR. Fundamentally, a zone transfer is a necessary mechanism designed to allow a primary DNS server to replicate its entire database of records to backup or secondary servers to ensure redundancy. The vulnerability arises when administrators fail to restrict *who* can request this data. By default or through misconfiguration, many nameservers will honor an AXFR request from any source IP address on the internet, effectively handing over their entire database to anyone who asks. As shown on the slide, an attacker can exploit this using standard client tools found in Linux distributions, such as the `dig` utility, by simply issuing the command `dig axfr` against the target's nameserver.

2. The Impact: A Reconnaissance Gold Mine The impact of a successful zone transfer is devastating from an operational security (OPSEC) perspective because it allows an attacker to bypass the tedious work of guessing hostnames. Instead of probing for common public servers like 'www' or 'mail,' the attacker receives a complete list of every DNS record in the zone. This often reveals sensitive information that organizations intend to keep private, such as internal IP addressing schemes, naming conventions, and the location of non-public resources like development databases or 'secret' project servers. In a split-DNS

scenario where internal and external records are co-mingled or not properly separated, this single command can instantly provide a network map of the organization's entire internal infrastructure without triggering intrusion detection systems that would normally catch active scanning.

3. The Fix: Authentication and IP Restriction To lock this map, we must implement strict access controls. The operational fix is to ensure that zone transfers are restricted strictly to defined, trusted peers, specifically, only your backup DNS servers. Any request originating from an unauthorized IP should be met with a 'Refused' error message. Furthermore, as indicated on the slide, rely on cryptographic authentication such as Transaction Signatures (TSIG). This ensures that even if an attacker attempts to spoof the IP address of a trusted secondary server, the transfer will fail because they lack the cryptographic key required to authenticate the transaction, thereby securing the replication process against unauthorized reconnaissance.



Registrar & Domain Lifecycle Security

- **The Threat: Domain Shadowing.**
 - Attackers don't hack your server; they hack your GoDaddy/Namecheap account.
 - They create thousands of malicious subdomains (e.g login.pizza.com) pointing to their servers.
- **Defense:**
 - MFA: Enforce Multi-Factor Authentication on all Registrar accounts.
 - Locking: Enable Registry Lock" to prevent unauthorized changes to NS records.
 - Monitoring: Alert on any new subdomain creation via passive DNS monitoring or Certificate Transparency logs.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Threat: Domain Shadowing We must recognize that the 'front door' to our digital presence is not just the web server itself, but the registrar account that controls it. In a **Domain Shadowing** attack, the adversary does not need to exploit a vulnerability in your web application code or server operating system. Instead, they compromise the administrative credentials for your domain registrar, providers like GoDaddy, Namecheap, or Route 53. Once inside, they do not take down your main website, which would trigger immediate alarms. Instead, they act as parasites, creating thousands of malicious subdomains (e.g login.pizza.com) that piggyback on your domain's good reputation but resolve to their malicious servers. Because these subdomains rely on your legitimate NS (Name Server) records, they appear trusted to users and security filters, effectively turning your brand into a launchpad for phishing or malware distribution.

2. Defense Layer 1: Identity Hardening (MFA) The primary defense against this account compromise is rigorous **Multi-Factor Authentication (MFA)**. Since credentials can be stolen through phishing or credential stuffing, relying solely on a username and password is insufficient. We must enforce MFA on all registrar accounts, requiring a second factor, 'something you have,' such as a hardware token or authenticator app, before any login is permitted. This ensures

that even if an attacker harvests the password, they cannot access the control panel to modify DNS records.

3. Defense Layer 2: Registry Locking For critical domains, we must go beyond basic login security and implement **Registry Lock** (often referred to as domain transfer lock). This feature prevents unauthorized changes to the domain's registration details, such as transferring the domain to another registrar or modifying the critical Name Server (NS) records that control where traffic is routed. By enabling this, we add a layer of friction that requires out-of-band verification to remove, preventing an attacker from easily hijacking the domain's routing logic even if they gain console access.

4. Defense Layer 3: Visibility and Monitoring Finally, because prevention can fail, we must have detection in place through **Monitoring**. We can leverage **Certificate Transparency (CT) logs**, a public ledger where Certificate Authorities record every SSL/TLS certificate they issue. Attackers creating malicious subdomains (like login.pizza.com) will almost certainly request SSL certificates to make their phishing sites appear legitimate with a 'secure' padlock. By monitoring feeds like **CertStream** or querying databases like **crt.sh**, we can receive real-time alerts whenever a certificate is issued for any variation of our domain. Additionally, utilizing **Passive DNS** allows us to track historical resolution data, helping us spot when new, unauthorized subdomains suddenly appear and resolve to unknown IP addresses, signaling immediate investigation.



Logging & Forensic Visibility

- **What to Log:** Pass/Fail" is not enough. You need the metadata.
 - Query Name: What are they looking for? (Detects DGA) .
 - Query Type: Are they asking for TXT records? (Detects Tunneling) .
 - Query Length: Is the domain 200 characters long? (Detects Exfiltration) .
 - Answer: What IP was returned? (Detects Fast Flux) .
- **Data Source:** Zeek dns.log is the gold standard for this analysis.

Logs are time machines. When a breach is announced today, you need logs from 3 months ago to answer: When was Patient Zero?"

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Necessity of Metadata: Beyond Pass/Fail When we discuss forensic visibility, we must accept that simple 'Pass/Fail' logging is insufficient for modern incident response. Knowing a connection occurred is useless without knowing the *intent* of that connection. We need granular metadata to understand the transaction. As the sources indicate, transaction data, specifically from DNS, provides a 'pulse' of network activity that allows us to characterize specific malicious behaviors that flow logs alone would miss. We need to capture the specific attributes of the query and the response to differentiate between legitimate user behavior and covert channel activity.

2. Analyzing the Query: DGA, Tunneling, and Exfiltration By breaking down the specific fields of a DNS log, we can map them to specific attack techniques:

Query Name (DGA): We analyze the query name to detect **Domain Generation Algorithms (DGA)**. Malware uses these algorithms to randomly generate rendezvous points, creating domains with high entropy or unusual character distributions,. Tools like freq.py can score these names for randomness to identify infected hosts.

Query Type (Tunneling): We scrutinize the Query Type to detect **Tunneling**. While 'A' records are standard, an excessive volume of **TXT**, **CNAME**, or **NULL** records often indicates an attacker is using the protocol to carry payloads or

Command and Control (C2) instructions,. For example, iodine tunneling tools utilize NULL records, which are rare in normal traffic.

Query Length (Exfiltration): We monitor Query Length to detect **Exfiltration**.

Attackers often encode sensitive data (using Base32 or Hex) into long subdomains to smuggle it out of the network,. Legitimate queries are rarely 200+ characters long; consistently long queries to a single domain are a high-fidelity indicator of data theft,.

3. The Answer and the Data Source We also look at the **Answer** section, the IP address returned, to detect **Fast Flux** networks. This is where attackers rapidly rotate the IP addresses behind a domain to evade IP-based blocking. By logging the answer, we can see if a single domain resolves to a constantly changing set of IPs with short Time-To-Live (TTL) values,. To capture all of this, **Zeek's dns.log** is the gold standard. Unlike standard server logs, Zeek creates a structured, protocol-aware log that links the query, the response, and the client in a format perfect for ingestion into SIEMs,.

4. Logs as Time Machines Finally, remember that logs are your **time machine**. The average 'dwell time', the time between compromise and detection, can be weeks or months. When an incident is announced today, the initial infection likely happened long ago. You need a robust retention policy that allows you to look back 90 days or more to answer the critical question: 'When was Patient Zero?'. Without historical logs, you are forced to start your investigation from the middle, often making it impossible to determine the root cause.



DNSSEC

1. **DNSSEC threat model:** What it protects against vs. what it doesn't.
2. **Key hierarchy:** Understanding KSK (Key Signing Key) and ZSK (Zone Signing Key).
3. **Chain of trust:** How validation flows from the root to the zone.
4. **Validation failures:** Troubleshooting breakage and outage risks.
5. **Operational complexity:** Managing key rollovers and zone signing.
6. **Strategic deployment decisions:** Weighing the costs vs. benefits.

In the last section, we saw how easy it is to lie to a DNS resolver. The 'Kaminsky Attack' proved that speed alone isn't a defense. Now, we introduce the solution: **DNSSEC**. We are going to stop trusting the *location* of the packet and start verifying the *signature* of the data.

1. DNSSEC Threat Model (What it is, What it isn't):

"First, we bust the biggest myth in networking. DNSSEC does **not** hide your browsing history; it is not a VPN. We will use the 'Envelope Analogy', DNSSEC is the wax seal that proves the letter wasn't opened (Integrity), but it is *not* a locked briefcase (Privacy). Everyone can still read the postcard.

2. Key Hierarchy (KSK vs. ZSK):

"Why do we need two keys? We'll explain the engineering trade-off between the **Zone Signing Key** (fast, small, changes often) and the **Key Signing Key** (heavy, strong, changes rarely). This is the foundation of managing crypto at scale.

3. Chain of Trust (The Digital Notary):

"How does your browser know to trust my-small-blog.com? We trace the chain of trust all the way up to the **Root Zone (.)**. We will visualize how the trust flows down from ICANN to the TLD and finally to your domain.

4. Validation Failures (The Cost of Security):

"Security comes at a price: **Fragility**. In standard DNS, a mistake means *some*

people can't reach you. In DNSSEC, a mistake (like an expired signature) means you disappear from the internet entirely. We'll discuss the dreaded SERVFAIL error.

5. Operational Complexity (Rollovers):

"We will discuss the most dangerous operation in DNS: **The Key Rollover**.

Changing your cryptographic keys while millions of users are actively querying your domain is like changing the tires on a moving car. We'll look at the automation tools required to do this safely.

6. Strategic Deployment Decisions:

"Finally, we ask the business question: 'Is it worth it?' DNSSEC adds latency and complexity. We will discuss which domains *must* have it (Banks, Government) and which might skip it (Static Blogs) based on risk appetite.



DNSSEC Threat Model (What it is, What it isn't)

- **The Problem:** Standard DNS has no authentication. I can spoof the IP of bank.com.
- **The Solution (Integrity):** DNSSEC adds cryptographic signatures to DNS records.
 - It proves the answer came from the real owner.
 - It proves the answer was not modified in transit.
- **The Limitation (Privacy):**
 - DNSSEC does **NOT** encrypt the traffic.
 - An attacker can still see *where* you are going (Privacy is solved by DoH/DoT, not DNSSEC).

Just like an Envelope you used to send a letter. DNSSEC is a wax seal on the envelope (Tamper Proofing). TLS/DoH is a locked briefcase (Privacy). You need both.

1. The Problem: The First Response Wins" Vulnerability To understand the necessity of DNSSEC, we must first recognize the inherent fragility of standard DNS. Traditional DNS resolution is unauthenticated; essentially, the client blindly trusts the first response it receives. This architecture leaves the network wide open to Man-in-the-Middle (MITM) attacks and cache poisoning. If an attacker sits between you and the resolver, they can spoof the IP address of a target, like 'bank.com', and redirect your traffic to a malicious server. Because standard DNS lacks a mechanism to verify the identity of the sender, the resolver accepts this forged data as truth, unaware that the user is being routed to a phishing site,.

2. The Solution: Cryptographic Integrity via DNSSEC DNSSEC (Domain Name System Security Extensions) addresses this specific integrity gap by adding digital signatures to DNS records. It establishes a 'Chain of Trust' that begins at the root zone and extends down to the authoritative server for the specific domain,.

Proof of Origin: DNSSEC validates that the response originated from the actual owner of the zone. It does this by grouping records into Resource Record sets (RRsets) and digitally signing them using a private key (Zone Signing Key). The client then verifies this signature using the public key,.

Proof of Integrity: By verifying the digital signature, the client confirms that the

data was not modified in transit. If an attacker attempts to tamper with the IP address in the response, the cryptographic hash will not match, and the resolver will reject the answer,.

Denial of Existence: DNSSEC even provides authenticated proof that a domain *does not* exist (using NSEC/NSEC3 records), preventing attackers from spoofing 'NXDOMAIN' errors to disrupt connectivity.

3. The Limitation: It is Not Encryption However, there is a critical misconception we must correct: **DNSSEC does not provide privacy.** It does not encrypt the traffic. Even with DNSSEC enabled, the queries and responses are sent in plaintext, meaning an eavesdropper on the wire can still see exactly which domains you are visiting,.

Privacy Solved Elsewhere: To solve the confidentiality problem, hiding *where* you are going, we must look to encrypted DNS protocols like **DNS over HTTPS (DoH)** or **DNS over TLS (DoT)**. These protocols encrypt the transport layer to prevent snooping, but they do not inherently validate the authenticity of the record data itself in the way DNSSEC does,.

4. The Analogy: Wax Seal vs. Locked Briefcase To summarize the threat model, think of a letter sent via the postal service. **DNSSEC is the wax seal** on the envelope. It guarantees that the letter came from the King (Authentication) and that it hasn't been opened or altered (Integrity). However, anyone handling the envelope can still read the address written on the outside. To hide the destination and the contents from prying eyes, you need to put that envelope inside a **locked briefcase**, that is **DoH or DoT**. For a complete security posture, you need both: the briefcase for privacy, and the wax seal for trust.



Key Hierarchy (KSK vs. ZSK)

- **The Design Challenge:** Signing every record with a massive key is slow and makes packets huge.
- **The Solution: Split Keys.**
 - **ZSK (Zone Signing Key):** A smaller, faster key used to sign the actual data (A records, MX records). It changes frequently (e.g. monthly).
 - **KSK (Key Signing Key):** A larger, stronger key used *only* to sign the ZSK. It changes rarely (e.g. yearly).
- **Why?** This reduces the administrative burden. You only need to update the parent (Registrar) when you rotate the KSK, not the ZSK.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Design Challenge and Solution: Split Keys To understand the architecture of DNSSEC, we must address the balance between security and performance. If we used a single, massive cryptographic key to sign every single DNS record, the computational overhead would be prohibitive, and the resulting packet sizes would strain the network. The solution is a **Split Key** architecture that divides duties between two distinct types of keys. We separate the function of signing zone data from the function of protecting the keys themselves, establishing a hierarchy of trust within the zone,.

2. The Zone Signing Key (ZSK) The workhorse of this system is the **Zone Signing Key (ZSK)**. As we see in the protocol mechanics, all resource records of a particular type (such as A records for IP addresses or MX records for mail servers) are grouped into a Resource Record set (RRset) and signed using the ZSK. Because the ZSK is used to sign the bulk of the zone's data, we typically use a smaller key size to ensure that signature generation and verification are fast and efficient. Because a smaller key is theoretically easier to crack, best practice dictates that we rotate the ZSK frequently, for example, monthly or quarterly, to limit the window of vulnerability.

3. The Key Signing Key (KSK) Sitting above the ZSK in the hierarchy is the **Key Signing Key (KSK)**. The KSK has a singular, critical purpose: it is used to sign the

DNSKEY RRset, which contains the public keys for the zone,. By signing the DNSKEY record, the KSK validates the ZSK, allowing resolvers to trust the data signed by that ZSK. Because the KSK is used far less frequently (only when keys change), we can afford to make it much larger and cryptographically stronger without impacting daily query performance.

4. Reducing Administrative Burden The primary operational benefit of this split is the reduction of administrative overhead regarding the 'Chain of Trust.' To link our zone to the global DNS hierarchy, the parent zone (typically the Registrar) publishes a **Delegation Signer (DS)** record, which is a hash of our KSK,. Because the parent only holds a fingerprint of the KSK, we are free to rotate the ZSK locally as often as necessary without ever needing to contact the Registrar or update the parent zone. We only incur the administrative burden of updating the DS record at the parent/Registrar when we rotate the KSK itself, which is why we do so rarely (e.g yearly).



Chain of Trust (The Digital Notary)

- **How Validation Works:** Resolvers trace the signature path up to the top.
 - The **Root (.)** signs the **TLD (.com)**.
 - The **TLD (.com)** signs the **Domain (example.com)** via a **DS Record**.
 - **example.com** signs the specific record (**www**).
- **The DS Record (Delegation Signer):** This is the critical link. It is a fingerprint of your KSK that lives in the Parent zone (e.g Verisign for .com).
- **Trust Anchor:** The Root Key (ICANN) is hardcoded into every DNS server in the world. This is the starting point of trust.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Hierarchy of Validation To understand the 'Chain of Trust,' we must recognize that DNSSEC essentially overlays a hierarchy of digital signatures onto the DNS system to prevent spoofing and Man-in-the-Middle attacks. In this model, a resolver does not simply trust an IP address it receives; it verifies the answer by tracing a signature path from the specific resource record all the way up to the root of the internet. The domain (e.g example.com) signs its own specific records (like www), but the validity of that signature depends on the parent zone (the TLD, such as .com) vouching for the domain, which in turn is vouched for by the Root,.

2. The DS Record: The Critical Link The mechanism that connects these layers is the **Delegation Signer (DS) Record**. This record acts as the cryptographic glue between a parent and a child zone. The DS record resides in the *parent* zone (e.g the .com zone managed by Verisign) and contains a hash of the child zone's **Key Signing Key (KSK)**,. Because the parent signs this DS record with its own private key, it creates an unbreakable chain of evidence. If a resolver checks example.com and finds that the hash of its KSK matches the signed DS record held by .com, it knows the child zone is legitimate and has not been hijacked.

3. The Trust Anchor: The Root of All Trust Finally, every chain must have a starting point. In DNSSEC, this is the **Trust Anchor**, the Root Key managed by

ICANN. Since there is no zone above the root to sign for it, the Root KSK is explicitly trusted and hardcoded into DNS software and resolvers worldwide. This key is generated and managed through a rigorous, audited 'Key Signing Ceremony' involving trusted community representatives and Hardware Security Modules (HSMs),. This single key serves as the ultimate foundation of trust for the entire global DNS ecosystem.



Validation Failures (The Cost of Security)

- **The Risk:** In standard DNS, a misconfiguration usually means *some* users can't reach you. In DNSSEC, a signature failure means **NOBODY** can reach you.
- **SERVFAIL:** This is the error code a validating resolver returns when signatures don't match.
 - To the user, the website looks Down.
 - To the engineer, the website is Up," but the signature is expired.
- **Common Causes:**
 - Clock skew (Time synchronization is critical for crypto).
 - Expired RRSIGs (Signatures have a validity period).
 - Forgot to update the DS record at the Registrar after a key rollover.

1. The Risk: From Best Effort" to Hard Fail" Implementing DNSSEC fundamentally shifts our availability model. In standard DNS, a misconfiguration might result in latency or a failover, but the protocol strives to return an answer. With DNSSEC, we introduce a strict binary state: valid or invalid. Because DNSSEC is designed to prevent data tampering, a validation failure, whether caused by a hacker or a typo, is treated identically: the resolver assumes the data is untrustworthy and drops it. As noted in the sources, if we fail to maintain the cryptographic components, such as renewing signatures, the domain effectively disappears for validating resolvers. Users cannot reach the website, and they cannot even send email to report the outage because the MX records will also fail validation.

2. The Mechanism: SERVFAIL and the Chain of Trust When a validating resolver attempts to resolve a query, it computes the hash of the Resource Record set (RRset) and compares it against the decrypted signature found in the RRSIG record. If these values do not match, or if the chain of trust back to the root cannot be established, the resolver returns a **SERVFAIL** (Server Failure) response. To the end-user, this looks indistinguishable from a crashed server or a network timeout. To the engineer inside the network, however, the web server is running, the firewall is passing traffic, and the DNS service is responding to

queries. The failure exists entirely within the cryptographic layer, creating a difficult troubleshooting scenario where the infrastructure is 'up,' but the service is 'down'.

3. Common Causes: Time and Key Management The three most common culprits for these self-inflicted Denial of Service events are clock skew, expiration, and broken chains:

Clock Skew: Cryptography relies heavily on precise time to enforce validity periods. As stated in the sources, accurate time is essential for validating certificates and signatures. If a server's clock drifts significantly, perhaps due to a failed Network Time Protocol (NTP) synchronization, it may incorrectly interpret a valid signature as 'not yet valid' or 'expired,' causing validation to fail.

Expired RRSIGs: Every DNSSEC signature (RRSIG) has a built-in expiration date. For example, in the Root zone, specific keys are generated to be used for limited 10-day periods. If the signing automation fails and the RRSIG expires, the records become invalid immediately. Administrators are encouraged to use shorter validity periods (e.g 60 days) to force frequent, routine maintenance, ensuring that the renewal process is tested and remembered rather than forgotten over long durations.

Key Rollover (DS Record): The 'Chain of Trust' relies on the parent zone holding a hash of the child's Key Signing Key (KSK) in a Delegation Signer (DS) record. A common failure mode occurs during a key rollover: the administrator generates a new KSK for their domain but fails to upload the new DS record to the registrar. This breaks the link to the parent zone, causing the entire chain of trust to collapse.



Operational Complexity (Rollovers)

- **Key Rollover:** Changing cryptographic keys without breaking the internet.
 - It is like Changing the tires on a moving car.
- **The Process:**
 - Publish the New Key alongside the Old Key (Double Signing).
 - Wait for TTLs to expire (Propagation).
 - Remove the Old Key.
- **The Burden:** If you do this manually, you *will* break your domain. You must use automated tools (like BIND's auto-dnssec or Cloudflare's managed service).

1. The Challenge: Changing Tires on a Moving Car We must address the operational reality of DNSSEC: keys cannot be immortal. Just as we rotate passwords and certificates to limit the window of vulnerability if a key is compromised, we must rotate our DNSSEC keys. However, this presents a unique challenge. Unlike a web server where we can simply swap a certificate, DNS is a globally distributed, cached system. We cannot stop the internet to update our keys; we must perform this maintenance while the system is live and serving millions of queries. This is exactly why we use the analogy of 'changing the tires on a moving car', the vehicle must keep moving, and the service cannot be interrupted during the swap.

2. The Process: Double Signing and the Criticality of TTLs The rollover process relies on a specific sequence to ensure continuity of trust. First, we publish the **New Key** alongside the **Old Key**. In many rollover scenarios, this involves **Double Signing**, where resource records are signed by both keys (or the new key is pre-published). This ensures that whether a resolver has the old or new configuration, it can validate the data. The most critical step is the wait. We must wait for the **Time to Live (TTL)** to expire. Because DNS resolvers cache answers to improve performance, they will hold onto the old records and signatures for the duration of the TTL. If we remove the Old Key immediately, a resolver with a

cached signature created by that old key will try to validate it against the current keys. If the Old Key is gone, validation fails, and the user is blocked. We must wait until every cached instance of the old data has expired from the internet's caches, known as **Propagation**, before we can safely remove the Old Key.

3. The Burden: The Necessity of Automation Finally, we must accept that manual key management is a liability. The sheer number of steps, the precise timing required for TTL expiration, and the catastrophic consequences of a mistake (taking the domain offline) make manual processing nearly impossible to sustain effectively. Just as we use tools like ssh-agent to manage SSH keys or managed services for TLS certificates to avoid outages, we must rely on automated tools for DNSSEC. Whether utilizing BIND's auto-dnssec features or offloading the burden entirely to a managed service like Cloudflare, automation ensures that the rigorous timing of the rollover is executed perfectly, preventing human error from breaking the domain.



Strategic Deployment Decisions

- **When to deploy:**
 - High-Value targets (Banks, Gov, SaaS).
 - If you need DANE (using DNS to bootstrap TLS security).
- **When to wait:**
 - If you lack the operational maturity to manage keys.
 - If "Availability" is more important than "Spoofing Protection" (e.g. a free read-only blog).
- **The Trade-off:** DNSSEC adds latency (larger packets), complexity (signing), and fragility (validation failures). The security gain must outweigh these costs.

1. When to Deploy: The Integrity Imperative We must first evaluate the risk profile of the organization. For high-value targets such as financial institutions, government agencies, or SaaS providers, the integrity of the connection is paramount. Without DNSSEC, the DNS protocol is unauthenticated, meaning any response is accepted as truth if the transaction ID matches, making it trivial for attackers to perform Man-in-the-Middle (MITM) attacks or cache poisoning. Deploying DNSSEC validates that the response comes from the authoritative source. Furthermore, deployment is mandatory if you intend to utilize **DANE (DNS-based Authentication of Named Entities)**. DANE allows administrators to embed public keys or certificates directly into DNS records, effectively bootstrapping TLS security. However, this capability relies entirely on the trust model provided by DNSSEC; without the signed zones, the keys provided by DANE cannot be trusted.

2. When to Wait: The Operational Hazard Conversely, we must acknowledge the operational risks. If your organization lacks the operational maturity to manage cryptographic lifecycles, you should wait. DNSSEC introduces complex requirements, such as the management of Key Signing Keys (KSK) and Zone Signing Keys (ZSK). The penalty for misconfiguration here is severe: unlike a standard expired SSL certificate which might show a browser warning, a lapsed

or misconfigured DNSSEC signature results in a total service outage for validating resolvers. As the sources note, if you forget to renew the certificate/signature, legitimate traffic simply stops, users cannot email you or visit your website. If 'Availability' is your primary metric, for example, on a read-only blog where the impact of spoofing is low, the risk of a self-inflicted Denial of Service due to mismanagement may outweigh the security benefits.

3. The Trade-off: Latency and Fragmentation Finally, we must address the technical trade-offs regarding performance. DNSSEC significantly increases the size of DNS responses because they must include the Resource Record digital signatures (RRSIG) and public keys (DNSKEY),. These records almost always exceed the traditional 512-byte limit of UDP packets. This necessitates the use of **EDNS (Extended DNS)** mechanisms, and frequently forces transmission to switch from UDP to TCP. This switch introduces latency because TCP requires a three-way handshake and connection teardown overhead, unlike the 'fire and forget' nature of UDP,. Therefore, the decision to deploy requires balancing the necessity of cryptographic integrity against the costs of increased latency and protocol complexity.



Monitoring and Detection

1. **BGP anomaly detection:** Identifying unusual path changes.
2. **Route monitoring:** Leveraging external vs. internal visibility tools.
3. **Third-party dependency monitoring:** Watching the providers you rely on.
4. **MTTD vs. MTTR:** Metrics for detection and recovery time.
5. **SOC integration:** Incorporating network signals into security operations.

We have spent the last few modules building walls, RPKI for routing and DNSSEC for naming. But in cyber, walls get climbed. This module is about **Visibility**. If an attacker bypasses your controls, how quickly will you know?"

1. BGP Anomaly Detection:

"First, we look at the 'Heartbeat' of the network. BGP should be boring. We'll discuss how to detect cardiac events, sudden spikes in prefix counts or rapid session resets, that indicate a **Route Leak** or a **Control Plane Attack**.

2. Route Monitoring (External vs. Internal):

"This is a critical concept: **The Blind Spot**. Your internal logs only show traffic leaving. They cannot see a hijack happening in London that redirects your traffic to Russia. We will show you how to use **Looking Glasses** and **BGPStream** to see your network from the outside in.

3. Third-Party Dependency Monitoring:

"You are only as secure as your supply chain. We'll look at why you need to monitor your Cloud Provider and DNS Host as closely as you monitor yourself. If they trip, you fall.

4. MTTD vs. MTTR (The Metrics):

"You cannot manage what you cannot measure. We will define **Mean Time To Detect** and **Mean Time To Respond**. We'll discuss why 'Coverage' (how much of

the network you see) is often more important than 'Depth' (how deep you look).

5. SOC Integration:

"Finally, we bring it back to the SOC. We will show how to integrate network signals, like **Zeek** logs and **RITA** beacon analysis, into your SIEM to create an automated feedback loop. If Zeek sees a threat, the Firewall should block it automatically.



BGP Anomaly Detection

- **The Concept:** Identifying deviations from the "Normal" routing state.
- **Key Metrics:**
 - Prefix Count: Did my neighbor suddenly send 5,000 routes instead of 50? This indicates a Route Leak.
 - AS Path Changes: Did my upstream provider change from Level3 to an unknown ASN in Russia? This indicates a Hijack or traffic detour.
 - Session Flaps: Is the BGP session resetting every 60 seconds? This could indicate an MTU mismatch or a volumetric attack on the Control Plane.

Treat BGP like a heartbeat. It should be steady. Any spike in routes or rapid change in neighbors is a cardiac event for the network.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Concept: Baselines and Deviation We begin with the premise that BGP should be a stable, predictable protocol. To detect anomalies, we must first establish a baseline of what 'Normal' looks like for our routing state. BGP is the backbone of internet routing, designed to help systems find one another, and relies heavily on trust between neighbors,. Because BGP prefers the shortest Autonomous System (AS) path and the most specific route, it is inherently vulnerable to manipulation. Therefore, detection relies on identifying deviations, unexpected changes in volume, pathing, or stability, that signal an error or an attack.

2. Prefix Count: Detecting Route Leaks The first critical metric is the **Prefix Count**. If a neighbor that typically sends 50 routes suddenly advertises 5,000, we are likely witnessing a **Route Leak**. This occurs when an ISP or enterprise accidentally announces prefixes they learned from one peer to another, effectively becoming a transit provider for the world. The impact can be devastating: a massive spike in inbound traffic that saturates links. A real-world example occurred when Vodafone Idea Ltd. mistakenly announced over 30,000 BGP prefixes, causing a 13x spike in inbound traffic and widespread disruption,. Monitoring for these sudden spikes allows us to shut down sessions before our network is overwhelmed.

3. AS Path Changes: Identifying Hijacks Next, we scrutinize **AS Path Changes** to detect **Hijacks**. If your traffic usually flows through a known Tier-1 provider like Level3, but suddenly reroutes through an unknown ASN in a different geopolitical region (e.g Russia), this is a high-fidelity alert. Attackers exploit BGP by advertising a more specific prefix (e.g a /24 inside your /19) or a shorter path to redirect your traffic through their infrastructure. This technique was notably used by Rostelecom to intercept traffic destined for financial subnets,. By monitoring the AS Path, we can detect these unauthorized detours that facilitate Man-in-the-Middle attacks.

4. Session Flaps: The Heartbeat of the Control Plane Finally, we treat BGP stability as a vital sign. **Session Flaps**, where a BGP session resets periodically (e.g every 60 seconds), indicate a failure in the Control Plane. BGP uses TCP (port 179) and relies on **Keepalive** messages (typically sent every 60 seconds by default) to maintain the session,. If these keepalives fail due to an **MTU mismatch** dropping large packets, or a **volumetric attack** exhausting the router's CPU, the hold timer expires, and the session drops,. We must treat BGP like a heartbeat; it should be steady. Any arrhythmia, rapid route changes or flapping neighbors, is a cardiac event that threatens the life of the network.



Route Monitoring (External vs. Internal)

- **Internal Visibility (The Blind Spot):** Your internal logs only show traffic leaving your network. They cannot see if your traffic is being hijacked after it leaves your border.
- **External Visibility (The Solution):**
 - **Looking Glasses:** Servers around the world that let you run traceroute back to your IP to verify the path.
 - **BGPStream:** A public monitoring service that detects hijacks and outages in real-time.
- **Analyst Action:** You must ping in from the outside to see what the rest of the world sees.

1. The Internal Blind Spot: Why Egress Logs Are Not Enough We must recognize that relying solely on internal logs creates a dangerous blind spot. Your internal firewalls and routers can confirm that traffic successfully left your border, but they cannot verify what happens to that traffic once it enters the public internet. In a BGP hijack scenario, an attacker manipulates the global routing table, often by advertising a more specific route (a longer prefix) than the legitimate owner. Because routers prefer the most specific path, the internet routes your traffic to the attacker. Your internal logs will show a 'successful' connection to the next hop, leaving you completely unaware that your data is being intercepted or blackholed downstream.

2. External Visibility: The Necessity of Global Monitoring To solve this, we must invert our perspective and monitor from the outside in. We utilize public monitoring services (referenced in the industry as BGPStream or BGPMON) to track how our routes are propagated globally. These services act as an external sensor grid, detecting when unauthorized autonomous systems (AS) suddenly start advertising our IP space. This allows us to detect hijacks in real-time, rather than waiting for customer complaints. Furthermore, analysts must validate these paths using tools like traceroute from external vantage points (Looking Glasses). By 'pinging in' from the outside, we can verify that the path traffic takes to reach

us is the path we expect, rather than a detour through a malicious network.



Third-Party Dependency Monitoring

- **The Reality:** You rely on upstream ISPs, Cloud Providers, and DNS hosts. If they break, you break.
- **Case Study:** [The Facebook Outage \(2021\)](#).
 - A BGP config error withdrew routes to Facebook's DNS servers.
 - Because the DNS servers were unreachable, nobody could resolve facebook.com.
 - Lesson: BGP is the Kill Switch" for the entire enterprise.
- **Defense:** Monitor your providers' ASNs as closely as you monitor your own.

1. The Reality: Outsourced Infrastructure, Retained Risk We must face the reality that modern network architecture is heavily reliant on third-party dependencies, ISPs, Cloud Providers, and DNS hosts. While we outsource the responsibility for maintaining these critical assets, we can never outsource the liability or the impact on our business availability. As noted in our sources, cloud service providers (CSPs) and ISPs function as the backbone of our operations, yet outages are inevitable. Whether it's a data center failure, a region-wide power outage, or a configuration error, when these upstream providers break, we break. We must recognize that availability is a shared fate; if the 'phone book' (DNS) or the 'road' (BGP) to your front door vanishes, you effectively cease to exist on the internet, regardless of whether your internal servers are running perfectly.

2. The Mechanism: BGP as the Kill Switch" The slide highlights the 2021 Facebook outage as a prime example of the fragility of the Border Gateway Protocol (BGP). While the specific details of that event are not in our text, the mechanism described, a BGP configuration error withdrawing routes, is a fundamental vulnerability of the internet's routing architecture. The public internet relies on BGP to determine reachability. BGP routers exchange UPDATE messages to advertise availability or WITHDRAW messages to remove routes. If a misconfiguration causes your border routers to send withdrawal messages for

your DNS nameservers' IP prefixes, the global routing table updates to say 'this destination no longer exists.' Consequently, valid DNS queries for your domain cannot reach your nameservers. Without DNS resolution, no one can resolve your domain name to an IP address, rendering your entire digital presence unreachable. In this way, BGP acts as a 'Kill Switch' for the enterprise; a single routing change can blackhole all ingress traffic.

3. Defense: ASN and Route Monitoring To defend against this, we cannot simply trust that our providers will stay up; we must monitor them as rigorously as we monitor our own infrastructure. The most effective defense is to actively monitor the Autonomous System Numbers (ASNs) and specific route prefixes associated with your critical services. An ASN identifies a group of networks controlled by a single entity, such as an ISP or a large enterprise. Tools and services like BGPMON allow us to monitor routing tables in real-time. By alerting on unexpected changes, such as the withdrawal of critical prefixes or the appearance of a new, unauthorized ASN in the path (which could indicate hijacking), we can detect connectivity issues immediately. We must track not just our own ASNs, but also those of our upstream dependencies, ensuring we are the first to know when the 'road' to our infrastructure has been cut.



MTTD vs. MTTR (Metrics)

- **Mean Time to Detect (MTTD):** The time between the initial infection (Patient Zero) and the first alert.
 - Goal: Drive this down by tuning baselines for Rare Domains".
 - Failure: [SolarWinds \(SUNBURST\)](#) had an MTTD of months because the beaconing was slow and blended in.
- **Mean Time to Respond (MTTR):** The time between the alert and the containment (e.g pushing a block to the RPZ).

You cannot manage what you cannot measure. Coverage" is a critical metric - what % of your network is actually monitored by Zeek?.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. Mean Time to Detect (MTTD): The Race Against Dwell Time We begin with **Mean Time to Detect (MTTD)**, which defines the critical window between the initial compromise, 'Patient Zero', and the moment our security systems generate an alert,. Industry reports, such as Mandiant's M-Trends, indicate that median dwell times have historically spanned weeks or even months, giving attackers ample time to entrench themselves,. Our goal is to drive this metric down aggressively. One effective technical strategy is tuning our baselines to identify 'Rare Domains.' By analyzing DNS logs for statistical anomalies, such as high-entropy hostnames associated with Domain Generation Algorithms (DGAs), we can identify Command and Control (C2) beaconing that deviates from standard business traffic,. Tools like freq.py or RITA can automate this analysis to highlight these outliers.

2. The SolarWinds Failure: The Consequence of High MTTD The **SolarWinds (SUNBURST)** incident serves as a stark example of a detection failure where the MTTD stretched into months. In this supply chain attack, the adversaries remained undetected because their tradecraft was designed to evade traditional baselines. The malware utilized 'slow' beaconing and masqueraded as legitimate protocol traffic, allowing it to blend seamlessly with normal network noise,. Because the beaconing did not trigger standard volumetric or signature-based

alerts, the adversaries maintained persistent access, moving laterally across on-premises and cloud environments before they were discovered.

3. Mean Time to Respond (MTTR) and Containment Once detection occurs, the clock starts on **Mean Time to Respond (MTTR)**. In this context, MTTR measures the velocity of our containment, the time elapsed between the alert generation and the neutralization of the immediate threat, such as isolating a host or pushing a block to a Response Policy Zone (RPZ),,. While some definitions focus on 'Repair' or 'Remediation,' from an active defense perspective, we prioritize the speed of **Containment** to stop the bleeding,. Rapid containment prevents the adversary from expanding their foothold or exfiltrating data while a longer-term remediation plan is developed.

4. The Criticality of Coverage Finally, we must acknowledge that metrics are meaningless without context: **You cannot manage what you cannot measure.** A low MTTR is deceptive if it only applies to a small fraction of your estate. We must track **Coverage** as a primary metric, specifically, what percentage of our network traffic is actually being monitored by sensors like **Zeek**?. If you monitor only 70% of your network, the remaining 30% is a blind spot where the dwell time is effectively infinite,. Ensuring comprehensive horizontal visibility across all subnets and vertical visibility across all layers is a prerequisite for any valid metric program.



SOC Integration

- **The Feedback Loop:** Connecting the Eyes" (Zeek) to the Hands" (DNS/Firewall).
- **The Workflow:**
 - Zeek detects a new C2 domain (e.g a DGA).
 - Analyst/Automation verifies the threat.
 - MISP/SOAR pushes the domain to the DNS RPZ.
 - DNS Server blocks the domain for all employees instantly.
- Tools: Security Onion (built-in Zeek & Suricata) and RITA (Beacon analysis).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Feedback Loop: From Observation to Action To mature our Security Operations Center (SOC), we must close the gap between detection and enforcement. We connect our 'Eyes', deep inspection tools like **Zeek**, to our 'Hands', enforcement points like DNS servers and firewalls. Zeek provides protocol-level visibility and transaction logging, allowing us to characterize traffic beyond simple signatures,. However, visibility alone is insufficient; we need a mechanism to act on that intelligence instantly. By integrating these components, we transition from a passive monitoring stance to an active defense, where a detection in one part of the network automatically triggers a defensive posture change across the infrastructure, such as updating a blocklist or terminating a connection,.

2. The Workflow: Automating the Response to C2 Let's walk through the specific workflow for a Command and Control (C2) threat. First, Zeek detects a high-fidelity indicator, such as a domain utilizing a **Domain Generation Algorithm (DGA)**. Zeek is particularly adept at this because it logs DNS transactions and can identify queries with high entropy or unusual structures typical of DGAs. Once detected, this intelligence is passed to an analyst or an automated platform. Tools like **MISP** (Malware Information Sharing Platform) allow us to store and correlate these Indicators of Compromise (IOCs),. Through

SOAR (Security Orchestration, Automation, and Response) integrations, we can push this confirmed threat directly to our **DNS Response Policy Zone (RPZ)**,. The DNS server then updates its configuration to 'lie' to any internal client requesting that domain, effectively sinkholing the traffic and neutralizing the threat instantly for the entire organization,.

3. The Toolset: Security Onion and RITA To execute this, we rely on robust open-source platforms. **Security Onion** acts as our central hub, bundling **Zeek** for metadata generation, **Suricata** for signature-based alerting, and the **Elastic Stack** for log retention and visualization,. This consolidation provides the necessary visibility to feed our automation pipelines. Furthermore, we utilize **RITA (Real Intelligence Threat Analytics)** to detect the most subtle C2 channels. RITA ingests Zeek logs to perform statistical anomaly analysis, specifically looking for **beaconing behavior**,. It identifies C2 channels not by signatures, but by analyzing the timing consistency (intervals) and connection duration of traffic, catching threats that evade traditional defenses,. Together, these tools provide the data fidelity required to automate blocking with confidence.



DNS and Routing Interdependencies

1. **Routing dependencies of DNS:** How reachability issues take down resolution.
2. **Routing attacks enabling DNS compromise:** Hijacking IP space to spoof nameservers.
3. **Anycast manipulation:** Specific risks to distributed DNS via routing.
4. **Cloud and shared infrastructure risks:** The impact of multi-tenancy.
5. **Combined failure scenarios:** Analyzing total outages where both systems fail.

We often troubleshoot BGP and DNS separately. But in reality, they are inextricably linked. DNS is an application that rides on top of the routing table. If the road (Routing) is broken, the ambulance (DNS) cannot reach the hospital.

1. Routing Dependencies of DNS:

"First, we will establish the baseline dependency. If your BGP routes are withdrawn, your DNS servers effectively vanish. We will discuss why 'Layer 3 availability' is the prerequisite for 'Layer 7 resolution.'"

2. Routing Attacks Enabling DNS Compromise:

"This is where it gets malicious. Attackers know that if they hijack the *IP prefix* of your nameserver, they can impersonate your domain. We'll show how a BGP hijack can be used to defeat standard DNS trust models.

3. Anycast Manipulation:

"Anycast relies on BGP to route users to the 'closest' server. But what if an attacker manipulates BGP to route *all* global traffic to your smallest server? We'll look at how routing policy can be weaponized to cause a DDoS.

4. Cloud and Shared Infrastructure Risks:

"In the cloud, you are a tenant in a shared building. We will discuss the risks of multi-tenancy, if your cloud provider's BGP is compromised, your DNS is compromised, and you have zero control over the fix.

5. Combined Failure Scenarios (The Perfect Storm):

"Finally, we will analyze the 'Perfect Storm.' We'll look at scenarios where a BGP hijack happens *during* a DNSSEC key rollover, leaving you unable to reach the management plane to fix a global validation failure. This is how multi-day outages happen.



Routing Dependencies of DNS

- **The Foundation:** DNS is an application that rides on top of the routing layer.
- **The Failure Mode:** If BGP breaks, DNS breaks.
 - **Scenario:** A BGP route leak causes your authoritative nameserver's IP prefix to be misrouted to a black hole.
 - **Result:** Users cannot resolve your domain, even though the DNS server software is running perfectly.

Layer 7 (DNS) cannot function if Layer 3 (Routing) is compromised. Availability is a stack-wide dependency.

1. The Foundation: DNS as an Application Layer Protocol To understand the fragility of DNS availability, we must look at the OSI model. As we see in the sources, DNS is defined as a Layer 7 **Application** protocol. However, for any application layer protocol to function, it relies entirely on the successful operation of the layers beneath it, specifically Layer 3, the **Network** layer where routing and IP addressing occur. We often troubleshoot DNS by looking at the server process itself, but we must remember that DNS packets are ultimately just data riding on top of the routing infrastructure. If the underlying transport mechanisms fail, the application is unreachable, regardless of its health.

2. The Failure Mode: BGP Route Leaks The vulnerability here lies in **BGP (Border Gateway Protocol)**, the system that determines the path traffic takes across the internet. BGP operates on a 'trust your neighbor' model, meaning routers generally accept the path advertisements sent to them by peers. In a **Route Leak** scenario, an ISP or peer might accidentally advertise a path to your authoritative nameserver's IP prefix. Because BGP routers prefer the longest prefix (most specific route) or the shortest Autonomous System (AS) path, the rest of the internet may suddenly decide that the 'best' path to your DNS server is through a misconfigured ISP. This can route your traffic into a **black hole** (or null route), where the traffic is silently discarded.

3. The Result and Pedagogical Takeaway The result of a BGP hijack or leak is that users cannot resolve your domain. Crucially, your DNS server software is likely running perfectly with no errors in its local logs because the requests simply never reach it. The server is 'up,' but the service is 'unavailable.' This reinforces the reality of the OSI model: Layer 7 (DNS) cannot function if Layer 3 (Routing) is compromised. Availability is a stack-wide dependency; you cannot secure the application without securing the path it travels.



Routing Attacks Enabling DNS Compromise

- **The Attack Vector:** Using BGP to hijack DNS traffic.
 - **Goal:** Impersonation.
 - **Method:** Attacker hijacks the IP prefix of your DNS server. They spin up a **fake DNS** server on that IP.
 - **Outcome:** They answer queries for bank.com with their own phishing IP.
- **Why it works:** Standard DNS clients trust the IP address they are talking to. If BGP says "This is the right place," the client believes it.
- **Defense:** DNSSEC is the only defense here, as it validates the *data*, not the *path*.

1. The Attack Vector: The Fragility of BGP Trust We must recognize that the integrity of DNS relies heavily on the stability of the underlying routing protocol, BGP. As we discussed in previous sections regarding BGP anomaly detection, the internet relies on Autonomous Systems (AS) trusting the routes advertised by their neighbors,. In this specific attack vector, an adversary exploits this trust by performing a **BGP Hijack**. They advertise a more specific IP prefix (a longer prefix) than the legitimate owner effectively overriding the global routing table,. By hijacking the IP prefix that belongs to your authoritative nameserver, the attacker physically redirects all DNS queries destined for your domain to a server they control.

2. Method and Outcome: Impersonation via Rogue Infrastructure Once the routing table is manipulated, the attacker spins up a fake DNS server on that hijacked IP address. When a victim queries bank.com, the request is routed through the internet infrastructure directly to the attacker's machine because BGP says, 'This is the correct path.' The attacker's server then replies with a spoofed A-record, directing the user to a phishing site instead of the real bank. Because standard DNS lacks built-in authentication, the client has no way to distinguish this rogue server from the legitimate one; it simply trusts the IP address it is talking to.

3. Why It Works: Validating the Path vs. Validating the Data This attack succeeds because of a fundamental flaw in the traditional internet stack: **Standard DNS validates the path, not the data.** In a standard DNS transaction, the resolver blindly accepts the response if the source IP and transaction ID match the request. If BGP says the attacker's server *is* the legitimate destination, the resolver trusts it. This is similar to the 'First Response Wins' vulnerability we see in local network spoofing, but executed at a global routing scale. The client believes it is talking to the right server because the network layer confirms it, leaving the application layer vulnerable to deception.

4. The Defense: The Necessity of DNSSEC This brings us to the critical role of **DNSSEC**. It is the only defense mechanism that effectively neutralizes this specific attack vector because it decouples trust from the network path. DNSSEC adds cryptographic signatures to the DNS records, allowing the resolver to verify the authenticity and integrity of the data regardless of where it came from,. Even if an attacker successfully hijacks the BGP route and answers the query, they do not possess the domain's private signing keys (ZSK or KSK). Therefore, they cannot generate a valid digital signature for their fake record. When the resolver checks the signature against the public key in the chain of trust, validation fails, and the malicious response is discarded,. In this scenario, DNSSEC turns a potential compromise into a denial of service, which is a far safer outcome.



Anycast Manipulation

- **Context:** Anycast depends entirely on BGP to route users to the closest" node.
- **The Attack:** Traffic Engineering Attacks.
 - An attacker manipulates BGP attributes (AS-Path Prepending or Community tags) to force global traffic to a single, small Anycast node.
- **The Impact:**
 - *Overload:* That small node gets crushed by global traffic (DDoS via routing policy).
 - *Latency:* Users in New York are routed to a server in Sydney, destroying performance.

1. Context: The Fragility of Closest" To understand this vulnerability, we must first look at how Anycast functions. In an Anycast architecture, multiple servers or data centers advertise the exact same IP address to the global internet. The logic of which server a user connects to is not determined by the application, but entirely by **BGP (Border Gateway Protocol)**. BGP routers are designed to select the 'best' path, which defaults to the **shortest Autonomous System (AS) path**,. Under normal conditions, this works perfectly: a user in New York connects to the New York node because it is topologically 'closer' than the node in Sydney, ensuring low latency and high performance.

2. The Attack: Weaponizing BGP Attributes The attack vector here is **Traffic Engineering**. Because BGP operates on a 'trust your neighbor' model, it is susceptible to manipulation. An attacker, or a misconfigured peer, can manipulate BGP attributes such as **AS-Path** to alter how traffic flows globally. By artificially lengthening the AS-Path (prepending) for the robust, high-capacity nodes, or by advertising a 'better' path (shorter AS-Path or more specific prefix) to a smaller, weaker node, the attacker forces the global routing table to converge on that specific target,. This effectively overrides the geographic distribution that Anycast is designed to provide.

3. The Impact: Volumetric Collapse and Latency The consequences of this

manipulation are twofold. First, we face **Overload**. By funneling global traffic, which was meant to be distributed across dozens of data centers, into a single, small node, we create a volumetric bottleneck. The bandwidth and processing capacity of that small node are instantly saturated, resulting in a Denial of Service (DoS) event caused not by a botnet, but by the routing policy itself. Second, we destroy performance through **Latency**. As the slide suggests, users in New York may be routed to Sydney. Because the traffic is taking a topologically inefficient path, the low-latency benefit of the Content Delivery Network (CDN) is negated, leading to slow load times and a degraded user experience.



Cloud and Shared Infrastructure Risks

- **The Reality:** Modern enterprises use Cloud DNS (Route53, Cloudflare, Azure DNS).
- **Multi-Tenancy Risk:** You share the same nameserver IPs with thousands of other customers.
 - **Collateral Damage:** If a fellow customer gets DDoSed, the shared DNS infrastructure might degrade, affecting your resolution times.
- **Trust Boundary:** You rely on the Cloud Provider's BGP security. If *they* get hijacked, *you* get hijacked. You have outsourced the routing risk.

1. The Reality: Migration to Multi-Tenant Architectures We must recognize that the modern enterprise network no longer ends at the traditional perimeter. By adopting cloud DNS services like Route53, Cloudflare, or Azure DNS, organizations effectively move critical infrastructure 'somewhere else,' relying on the cloud provider's resource pooling to handle their traffic. While this offers scalability, it introduces the **Multi-Tenancy Risk**. As noted in the sources, cloud providers achieve cost efficiency and elasticity by pooling resources such as network bandwidth and processing power. This means your authoritative nameservers often share the same underlying infrastructure, and sometimes even the same IP addresses, with thousands of other customers whom you do not know and cannot trust,.

2. Collateral Damage: The Shared Fate of DDoS This shared architecture creates a vulnerability known as **Collateral Damage**. In a massive Distributed Denial of Service (DDoS) attack, the sheer volume of traffic can saturate the provider's bandwidth, affecting not just the intended target but every other tenant sharing that pipe,. We saw this reality during the Mirai botnet attack on DynDNS; the attack targeted specific entities, yet it took down major portions of the internet for everyone using that shared DNS infrastructure. Even robust providers can be challenged by high-bandwidth volumetric attacks, degrading

resolution times for fellow customers sharing the degraded infrastructure.

3. Trust Boundaries: Outsourcing BGP Security Finally, we must address the **Trust Boundary**. When you utilize Cloud DNS, you are effectively outsourcing the defense of your routing layer. You rely entirely on the Cloud Provider's BGP (Border Gateway Protocol) security to ensure traffic actually reaches your services. If the provider suffers a route hijack or a misconfiguration, your traffic is redirected before it ever reaches your virtual network controls,. While we can outsource the responsibility for maintaining these routes, we cannot outsource the liability when they fail. We must therefore treat these external dependencies as untrusted and monitor them rigorously, recognizing that a compromise at the provider level allows adversaries to bypass our internal controls entirely.



Combined Failure Scenarios (The Perfect Storm)

- **The Scenario:** A BGP hijack during a **DNS key rollover**.
 - You are trying to update your DNSSEC keys.
 - An attacker hijacks the route to the Registrar.
 - You cannot upload the new DS record.
 - **Result:** Your domain goes Bogus" globally (**DNSSEC failure**) AND you cannot reach the management plane to fix it.
- **Impact Analysis:** These cascading failures are often what cause multi-day outages for major platforms. **Recovery requires fixing Layer 3 (Routing) before you can even touch Layer 7 (DNS).**

1. The Scenario: Critical Maintenance Meets Critical Failure This slide describes the ultimate nightmare for network reliability engineers: a dependency deadlock caused by simultaneous failures at different layers of the OSI stack. We begin with a routine **DNSSEC Key Rollover**. As we established earlier, DNSSEC relies on a strict chain of trust where a parent zone holds a Delegation Signer (DS) record, which is a hash of the child's Key Signing Key (KSK). To rotate keys, we must upload a new DS record to our registrar to maintain this chain. However, in this scenario, a **BGP Hijack** occurs at the exact moment we need to communicate with the registrar. An attacker, or perhaps a misconfiguration, advertises a more specific prefix for the registrar's network, effectively overriding the global routing table.

2. The Mechanism: The Broken Chain and the Unreachable Control Plane The result is a catastrophic severance of the management plane. We are technically capable of generating the new keys and signing our zones locally, but because the network path (Layer 3) to the registrar is compromised, we cannot publish the DS record to the parent zone. The chain of trust is broken. To the rest of the world, our domain is now 'Bogus.' Because DNSSEC is designed to fail closed when validation fails, validating resolvers will treat our domain as unsafe and drop all traffic,. We effectively suffer a total blackout, indistinguishable from the

domain ceasing to exist.

3. Impact Analysis: The OSI Dependency Trap The recovery from this event is prolonged because it requires solving a Layer 3 problem to fix a Layer 7 issue. As noted in the sources, the Application Layer (Layer 7), where DNS resides, relies entirely on the Network Layer (Layer 3) for path determination and connectivity,. You cannot log in to the registrar to fix the DNSSEC configuration because the BGP route to the registrar is hijacked. You must first identify and mitigate the BGP hijack, perhaps by contacting upstream ISPs to filter the malicious route advertisements, before you can even begin to address the DNS outage. This cascading dependency is why these specific scenarios often result in multi-day outages; the 'road' to the data center must be rebuilt before the 'server' can be fixed.



Module 4 : Cloud Implications



Cloud and Hybrid Environments

1. **Cloud DNS trust boundaries:** Limitations of managed DNS services.
2. **Hybrid name resolution:** Complexity of resolving across on-prem and cloud.
3. **Inter-region and inter-VPC risks:** Lateral movement via routing constructs.
4. **SaaS dependency mapping:** Identifying hidden reliance on third parties.
5. **Shared responsibility gaps:** Where provider security ends and yours begins.
6. **Cloud routing abstractions:** Understanding VPCs, transit gateways, and cloud trust boundaries.
7. **Subdomain Takeover:** Orphaned DNS records pointing to deprovisioned cloud resources enabling attackers to claim the subdomain.
8. **Cloud-Native DNS Logging:** Limited or fragmented DNS telemetry across managed services reducing visibility into attacks.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We are leaving the physical data center. In the cloud, there are no cables to tap and no physical routers to rack. Routing is software defined, and DNS is a managed service. This changes the game completely.

1. Cloud Routing Abstractions (VPC & Direct Connect):

"First, we have to relearn our definitions. A 'Router' is now a VPC Route Table. A 'Cable' is now a Direct Connect or ExpressRoute. We'll discuss how these abstractions extend your internal trust boundary right into the public cloud, often inadvertently.

2. Cloud DNS Trust Boundaries:

"We'll look at the limits of Managed DNS (like Route53). You gain ease of use, but you lose visibility. You often can't see the query logs by default, and you share your nameserver infrastructure with thousands of other tenants.

3. Hybrid Name Resolution (The 'Split Brain' Problem):

"The hardest part of modern networking is making On-Prem talk to Cloud. We will discuss the 'Split Brain' DNS architecture, where your on-prem servers need to resolve cloud names and vice versa, and the fragile 'Resolver Endpoints' required to make it work.

4. Inter-Region and Inter-VPC Risks:

"We will bust the myth that 'Private IP means Trusted.' Just because a packet

comes from a peered VPC doesn't mean it's friendly. We'll cover lateral movement across Transit Gateways.

5. Shared Responsibility Gaps:

"The most dangerous slide in this deck: The Gap. AWS secures the BGP path to the region; *you* must secure the DNS records within your zone. We will show exactly where students typically fail this responsibility.

6. Subdomain Takeover (Cloud Specific):

"This is the #1 bug bounty finding in the cloud. We will explain 'Dangling CNAMEs', what happens when you delete an EC2 instance but forget to delete the DNS record pointing to it. (Spoiler: An attacker takes it over).

7. Cloud-Native DNS Logging:

"Finally, since we can't use a physical TAP anymore, we have to use Cloud-Native logs. We'll show you how to turn on and analyze Route53 Query Logs to catch tunneling inside your VPC.



Cloud DNS Trust Boundaries

- **The Concept:** Managed DNS services (Route53, Cloudflare, Azure DNS) abstract the complexity, but also the control.
- **Limitations:**
 - **No Root" Access:** You cannot see the underlying logs (query logs) unless you pay for premium features (e.g Route53 Query Logging).
 - **Shared Infrastructure:** You share the nameserver IP with thousands of other customers. If *their* domain gets DDoSed, *your* resolution might slow down.
- **Trust Boundary:** You trust the provider to secure the BGP routes to the nameservers. If AWS BGP is hijacked, your Route53 zones are hijacked.

1. The Concept: Abstracted Complexity and Loss of Control When we shift to managed DNS services like Amazon Route 53, Cloudflare, or Azure DNS, we are effectively purchasing abstraction. We gain the benefit of the provider's massive scale and ease of use, but we must acknowledge that we are trading away direct control over the underlying infrastructure. In this model, the cloud provider creates a 'black box' around the service; they manage the patching, the hardware, and the availability, but they also define the limits of what we can see and touch. We no longer own the server; we simply rent a slot in their multi-tenant environment.

2. Limitations on Visibility: The Cost of Root" Access A critical operational limitation is the lack of default visibility. On a traditional on-premise name server, you have root access to watch query logs in real-time. In the cloud, this visibility is often an optional, paid feature. For example, AWS Route 53 query logs, which track the queries made to your public zones, are not enabled by default. To gain the visibility required for threat hunting or debugging, you must explicitly configure logging and pay for the resulting storage and data processing,. Without enabling and paying for these 'premium' features, you are effectively flying blind regarding the traffic hitting your authoritative zones.

3. Shared Infrastructure: The Noisy Neighbor" and DDoS Risk Furthermore, we

must address the risks inherent to **Shared Infrastructure**. Cloud services rely on resource pooling to achieve efficiency. This means your authoritative zone likely shares the same nameserver IP addresses with thousands of other customers. If a fellow tenant becomes the target of a massive volumetric DDoS attack, the shared infrastructure supporting your domain can become saturated, leading to collateral damage. As we saw with the Mirai botnet attack on Dyn, a DDoS attack targeting specific customers can degrade service availability for the entire platform, slowing down or blocking resolution for everyone sharing that infrastructure.

4. The Ultimate Trust Boundary: Outsourced BGP Security Finally, we arrive at the hard reality of the **Trust Boundary**. By using a managed DNS provider, you are outsourcing the defense of the routing layer to them. You trust the provider to secure the Border Gateway Protocol (BGP) routes that direct traffic to their nameservers. If the provider fails to secure their ASNs and suffers a BGP hijack, similar to the incidents we've seen with ISPs like Vodafone or Rostelecom, adversaries can redirect traffic destined for your Route 53 or Azure DNS zones before it ever reaches the cloud provider's network,. In this scenario, your zone files are intact, but the 'road' to your DNS server has been stolen.



Hybrid Name Resolution (The Split Brain")

- **The Scenario:** Your servers are in AWS, but your Active Directory is On-Prem.
- **The Problem:**
 - **AWS** instances need to resolve intranet.corp (On-Prem).
 - **On-Prem servers** need to resolve app.aws.corp (Cloud).
- **The Solution: Resolver Endpoints.**
 - **Inbound Endpoint:** Allows On-Prem to query Cloud DNS.
 - **Outbound Endpoint:** Allows Cloud to query On-Prem DNS via VPN/Direct Connect.
- **Complexity:** Managing conditional forwarders across two different environments creates a fragile Split Brain" DNS that is hard to debug.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Scenario: Hybrid Infrastructure Reality We must address the most common architectural reality for large enterprises: the hybrid cloud. As organizations migrate to the cloud, they rarely move everything at once. Consequently, we frequently see a split architecture where compute workloads run in AWS, but the authoritative identity source, Active Directory, remains on-premises. This creates a fundamental disconnection in name resolution. By default, AWS instances utilize the Amazon Route 53 Resolver for DNS within the VPC, while on-premises servers rely on internal domain controllers. Without specific intervention, these two environments are blind to one another's private hostnames.

2. The Solution: Resolver Endpoints and Forwarding To bridge this gap, we implement **Resolver Endpoints**. This solution creates a 'hybrid private hosted zone' by allowing DNS queries to traverse the boundary between the cloud and the data center.

Inbound Endpoints: These allow on-premises resources to query cloud DNS. The on-premises DNS server forwards queries for cloud domains (e.g. app.aws.corp) to the Inbound Endpoint, which passes them to the VPC resolver.

Outbound Endpoints: These allow cloud resources to resolve on-premises domains. You configure forwarding rules in AWS to send queries for specific

domains (e.g intranet.corp) through the Outbound Endpoint to your on-premises name servers. This traffic typically rides over private, secure connections such as **AWS Direct Connect** or **Site-to-Site VPNs**, ensuring these queries never traverse the public internet.

3. Complexity: Managing the Split Brain" While effective, this architecture introduces significant complexity, often referred to as a 'Split Brain' or Split-DNS scenario. We are effectively splitting the knowledge of our DNS zones between two distinct infrastructures. This creates fragility because the availability of DNS, a critical utility, now depends entirely on the stability of the underlying network transport (VPN or Direct Connect). If the VPN tunnel drops, the cloud application cannot resolve the database string for the on-prem SQL server, causing an outage. Furthermore, debugging becomes difficult because logs are decentralized; you must correlate Route 53 Resolver logs in the cloud with Windows DNS logs on-premises to trace a single resolution failure.



Inter-Region and Inter-VPC Risks

- **The Trust Assumption:** VPC Peering.
 - "If I peer VPC A with VPC B, they trust each other."
- **The Attack:** Lateral Movement via Routing.
 - If an attacker compromises a Dev VPC, and it is peered with Prod, they can route packets directly to Prod databases.
 - **Transit Gateways (TGW):** Centralized hubs that can become a single point of failure or a hub" for spreading malware if not segmented with Route Tables.
- **Defense:** strict Security Groups (Firewalls) even between peered VPCs. Never assume a Private IP" means Trusted Device.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Trust Assumption and Lateral Movement We must first address the dangerous assumption that connectivity equals trust. When we establish **VPC Peering**, we are creating a direct network route between two environments, effectively joining them into a private network where they 'trust' each other's routing capability. This enables a specific attack vector: **Lateral Movement**. If an attacker compromises a lower-security environment, such as a Development VPC, and that VPC is peered with Production, the attacker can leverage this existing route to pivot directly to sensitive production databases,. In many default configurations, once this peering is established, the network traffic is routable by default, allowing adversaries to move 'behind enemy lines' without ever traversing the public internet or passing through a perimeter firewall,.

2. The Transit Gateway (TGW) Risk While **Transit Gateways (TGW)** solve the management complexity of connecting hundreds of VPCs by acting as a central hub, they introduce their own risks. If not architected correctly with isolated Route Tables, a TGW can flatten your network topology, allowing all attached VPCs to communicate with one another,. This turns the TGW into a potential 'hub' for spreading malware; if one spoke is infected, the malware can traverse the hub to infect other spokes. To prevent this, we must utilize TGW route tables to strictly control which attachments can route to one another, essentially

treating the TGW as a centralized cloud router that enforces segmentation rather than just facilitating connectivity,.

3. Defense: Zero Trust and Security Groups The primary defense against these risks is the application of strict **Security Groups** and a move toward **Zero Trust**. We must stop assuming that a 'Private IP' equals a 'Trusted Device.' Just because traffic arrives from a private RFC 1918 address (e.g 10.x.x.x) does not mean it is benign; it simply means it is internal,. We must enforce micro-segmentation by implementing 'default deny' policies for all east-west traffic, only allowing specific ports and protocols required for business logic. Furthermore, it is critical to note that for **Inter-Region** peering, you typically cannot reference the Security Group ID of the peer; you must manage rules based on CIDR blocks, which requires rigorous discipline to ensure you are not inadvertently allow-listing an attacker's IP.



SaaS Dependency Mapping

- **The Hidden Risk:** You don't just rely on your cloud; you rely on your SaaS providers' cloud.
- **Scenario:** Your app uses an external API (e.g Twilio or Stripe).
 - If *their* DNS provider (e.g Dyn) goes down (like in 2016), your app crashes.
- **Mapping:** You must identify **every external hostname your app queries**.
- **Mitigation:**
 - **Hardening:** Can you cache the IP? (Risky).
 - **Failover:** Do you have a backup provider if the primary API is unreachable?

1. The Hidden Risk: Recursive Cloud Dependency We must recognise that in a modern microservices architecture, our perimeter effectively extends to the third-party APIs we consume. As the sources highlight, application programming interfaces (APIs) allow disparate applications to communicate, meaning your application likely relies on external data sources or functionality, such as payment processors or messaging services. However, this creates a hidden risk: you are not just relying on *your* cloud provider's uptime, but also on the cloud infrastructure that *your SaaS providers* use. While we can outsource the responsibility for maintaining these services, we can never outsource the liability when they fail. If a critical dependency goes offline, your application fails just as surely as if your own servers had crashed.

2. The Scenario: The Fragility of DNS The 2016 DynDNS outage serves as the definitive case study for this vulnerability. In that incident, the Mirai botnet launched a massive distributed denial-of-service (DDoS) attack, reaching throughputs of up to 1 Tbps, against the DNS provider Dyn. This attack did not necessarily target the specific applications using Dyn, but rather the infrastructure they relied on. Because the DNS servers were overwhelmed by traffic (including DNS error messages generated by the attack itself), legitimate users could not resolve the domain names of major services. If your app relies on

an API like Twilio or Stripe, and their DNS provider suffers a similar fate, your app will be unable to resolve the API endpoint, resulting in a total service failure.

3. Mapping and Mitigation: Visibility and Resilience To defend against this, you must first map your territory. You cannot manage a risk you do not know exists. This involves identifying every external hostname your app queries, a process similar to generating a Software Bill of Materials (SBOM) where you decompose the application to identify all remote library and API dependencies. Regarding mitigation, the slide asks if we can simply cache the IP address to bypass DNS. We must be extremely cautious here. As noted in our sources regarding Content Delivery Networks (CDNs) and cloud services, IP ranges for services are dynamic and can change frequently. Hard-coding or aggressively caching an IP address is risky; if the provider updates their infrastructure or rotates IPs for security, your application will break because it is holding onto a stale address. A more robust strategy is architectural redundancy and failover planning, ensuring that if a primary service or availability zone becomes unreachable, your system can degrade gracefully or switch to an alternative provider.



Shared Responsibility Gaps

- **The Model:** Cloud Providers secure the *Cloud* (Physical datacenter, Hypervisor). You secure *in* the Cloud (OS, Data, Traffic).
- **The Gap in Routing/DNS:**
 - **Provider:** Secures the BGP advertisement of the Region.
 - **You:** Secure the Route Tables in your VPC (preventing traffic from routing to a rogue Internet Gateway).
 - **You:** Secure the DNS records in your hosted zone (preventing Dangling CNAMEs" that lead to Subdomain Takeover).
- **Key Takeaway:** AWS will not delete your CNAME record when you delete the EC2 instance it points to. That hygiene is *your* job.

1. The Model: Security of" vs. Security in" the Cloud We must begin by explicitly defining the Shared Responsibility Model, as it is the source of many critical vulnerabilities. The cloud provider is responsible for the **Security of the Cloud**, this includes the physical data centers, the hardware, the global infrastructure, and the hypervisor layer. They ensure the building is secure and the servers have power. However, the customer is responsible for **Security in the Cloud**. This encompasses your data, your operating system configuration, your application logic, and crucially, your network traffic management. A common failure mode occurs when teams assume the provider's infrastructure security automatically extends to their specific configurations, which is simply not true.

2. The Gap in Routing: BGP vs. VPC Route Tables This gap becomes dangerous in the context of routing. The cloud provider secures the **BGP advertisements** for the Region, ensuring that traffic destined for us-east-1 actually arrives at their data centers. However, once that packet enters your Virtual Private Cloud (VPC), the provider hands the steering wheel to you. You must secure the **Route Tables**. If a route table is misconfigured to send internal traffic to a rogue Internet Gateway or a malicious instance acting as a gateway, the provider will honor that route because you defined it. The provider ensures the road exists; you are responsible for ensuring your car doesn't drive off a cliff.

3. The Gap in DNS: The Dangling CNAME" Risk The most prevalent manifestation of this gap is the **Dangling CNAME**. In a cloud environment, resources like EC2 instances or S3 buckets are ephemeral, they are created and destroyed frequently. However, DNS records are often static and manual. If you delete an EC2 instance or deprovision an S3 bucket but fail to delete the **CNAME record** pointing to it, you create a 'dangling' record. The DNS record still points to a specific domain name (e.g feature-branch.s3.amazonaws.com) that no longer exists in your account.

4. Subdomain Takeover and Hygiene This leads to **Subdomain Takeover**. Because the resource name is now available in the provider's pool, an attacker can scan for these dangling records, identify the missing resource, and register a new S3 bucket or EC2 instance with that exact name. Your valid corporate domain now resolves to the attacker's infrastructure, allowing them to serve malicious content, steal cookies, or conduct phishing campaigns under your brand. **Key Takeaway:** AWS will not clean up your DNS records for you. When you terminate an instance, the CNAME remains until you explicitly remove it. This operational hygiene is entirely your responsibility and requires rigorous auditing to prevent attackers like the 'Hazy Hawk' group from exploiting these dormant records.



Cloud Routing Abstractions (VPC & Direct Connect)

- **The Abstraction:** In the cloud, Routers" are virtual software constructs, not physical boxes.
- **VPC Routing Tables:**
 - **Public Subnet:** Has a route to the Internet Gateway (IGW).
 - **Private Subnet:** Has a route *only* to a NAT Gateway or Virtual Private Gateway (VGW).
- **Direct Connect / ExpressRoute:**
 - A physical cable connecting your On-Prem datacenter to the Cloud Provider.
 - **Risk:** This extends your Internal Network" trust boundary into the cloud. Malware in the cloud can traverse this link to infect on-prem servers if not firewalled.

1. The Abstraction: Software-Defined Networking When we operate in the cloud, we must abandon the mental model of physical cables and hardware routers. The cloud utilizes **Software-Defined Networking (SDN)**, where the 'router' is a virtual software construct provided by the cloud vendor,. We customize routing tables to override default behaviors, but the underlying physical switching and routing are entirely abstracted away from us. This abstraction allows for dynamic reconfiguration and scalability, but it also means we rely on the provider's logic to direct our traffic rather than physical wiring,.

2. VPC Routing Tables: Defining Public vs. Private The distinction between a 'Public' and 'Private' subnet is defined entirely by the **Routing Table**. A **Public Subnet** is public simply because it has a route table entry that sends traffic destined for the internet (0.0.0.0/0) to an **Internet Gateway (IGW)**. This gateway performs a 1:1 NAT, allowing the instance to communicate directly with the outside world,. In contrast, a **Private Subnet** lacks this direct route. If a server in a private subnet needs to patch or update, it cannot go straight to the IGW. Instead, its routing table points to a **NAT Gateway** (which allows only outbound traffic) or a **Virtual Private Gateway (VGW)** for internal VPN traffic,. This isolation is a critical security control, ensuring that backend resources like databases are not directly reachable from the internet.

3. Direct Connect & ExpressRoute: The Physical Trust Bridge For hybrid environments, we move beyond simple VPNs to dedicated physical circuits like **AWS Direct Connect** or **Azure ExpressRoute**. These are dedicated fiber connections providing high bandwidth and low latency between your on-premise data center and the cloud provider, bypassing the public internet entirely,,. However, this creates a critical **Trust Boundary Risk**. By establishing this link, you are effectively extending your internal network perimeter into the cloud. If an attacker compromises a cloud instance, they can potentially utilize this 'trusted' private link to pivot laterally back into your on-premise data center,. Therefore, you must treat these links as untrusted interfaces, placing firewalls at the termination points to inspect traffic and prevent malware from traversing the link to infect your core internal servers.



Subdomain Takeover (Cloud Specific)

- **The Mechanism:**

- You create a DNS record app.corp.com -> bucket-123.s3.amazonaws.com.
- You delete the S3 bucket but *forget* to delete the DNS record.
- Attacker creates a new S3 bucket named bucket-123.
- Now app.corp.com serves the attacker's content (Phishing/Malware).

- **Prevalence:** This is the #1 high-severity finding in cloud bug bounties.

- **Defense:** Continuous monitoring for Dangling CNAMEs" (DNS records pointing to non-existent resources).

1. The Mechanism: The Dangling" DNS Problem We need to discuss a vulnerability born from the elasticity of the cloud: the Subdomain Takeover. In a standard deployment, you create a friendly Canonical Name (CNAME) record, such as app.corp.com, and point it to a cloud resource, like an S3 bucket or a CloudFront distribution. The vulnerability manifests when you decommission that cloud resource, deleting the S3 bucket, but neglect to remove the corresponding DNS record. This creates a 'dangling CNAME' or 'dangling record,' where your organization's verified domain points to a resource that no longer exists in your account but is theoretically available in the cloud provider's pool,.

2. The Attack: Global Namespace Exploitation Because cloud storage namespaces, such as S3 bucket names, are often universal or globally unique, only one customer can hold a specific name at a time. Once you delete your bucket, that name becomes available to the general public. An attacker identifies this dangling record and simply registers a new bucket in their own account with the exact same name you just deleted. Instantly, your legitimate domain, app.corp.com, begins routing traffic to the attacker's infrastructure. This allows them to bypass traditional reputation filters because the content is being served from a trusted, high-reputation domain.

3. Impact and Prevalence: Weaponizing Trust The impact here is severe

because it exploits the trust users place in your brand. Attackers use these hijacked subdomains to host phishing pages, serve malware, or steal session cookies, as the browser treats the attacker's content as originating from your trusted domain,. This vector is actively exploited by threat groups like 'Hazy Hawk,' who target dormant cloud-linked DNS records to impersonate major institutions and distribute malicious payloads via traffic distribution systems,.

4. Defense: Hygiene and Monitoring To defend against this, we must treat DNS cleanup as a mandatory step in the de-provisioning lifecycle. You cannot simply delete a cloud resource; you must also purge the associated CNAME.

Continuous monitoring is essential; we should audit our DNS zones for records pointing to 'NXDOMAIN' or cloud error pages and utilize tools that assess accounts for decoupled configurations, such as active Route 53 aliases pointing to deleted CloudFront distributions.



Cloud-Native DNS Logging

- **Visibility:** Traditional pcap" is hard in the cloud. You rely on logs.
- **AWS Route53 Query Logs / Azure DNS Analytics:**
 - You must explicitly enable these; they are off by default.
 - They show: Timestamp, Source IP (Internal VPC IP), Query Name, Response.
- **Use Case:** Feeding these logs into a SIEM (CloudWatch/Sentinel) to detect DGA and Tunneling within the VPC.

1. Visibility: The Shift from Packets to Logs In a traditional on-premise environment, we rely on SPAN ports and physical taps to capture full packets (pcap). In the public cloud, however, this 'wire' access is abstracted away. The underlying network infrastructure is multi-tenant, meaning we cannot simply plug in a tap to see traffic for our specific VPC without potentially violating the privacy of other customers. While mechanisms like traffic mirroring exist, they can be complex to manage and prohibitively expensive to store at scale. Therefore, our primary visibility strategy must shift from trying to capture every packet to relying on high-fidelity service logs generated by the cloud platform itself, specifically at the DNS layer.

2. Enabling the Logs: A Critical Configuration Step A common critical failure in cloud security is the assumption that logging is automatic. For services like **AWS Route 53 Resolver Query Logs** and **Azure DNS**, logging is **disabled by default**. Cloud providers disable these by default because they generate massive volumes of data and incur additional storage and ingestion costs. You must explicitly configure these services to export logs to a destination like S3, CloudWatch, or a Log Analytics Workspace; otherwise, you are effectively blind to the name resolution activity occurring inside your virtual networks.

3. The Data: What the Logs Reveal Once enabled, these logs provide the

precise context needed for attribution. Key fields include the **Timestamp**, the **Source IP** (which allows you to identify the specific internal interface or EC2 instance making the request), the **Query Name** (the domain being requested), and the **Response** code. For example, Route 53 Resolver logs can show you exactly which internal IP queried a specific domain and whether the resolution was successful or failed (NXDOMAIN). This allows you to trace a malicious request back to a specific compromised asset within your VPC.

4. Use Case: Detection via SIEM Integration The final piece of the workflow is ingestion and analysis. We feed these logs into a SIEM like **Amazon CloudWatch** or **Microsoft Sentinel** to automate threat detection.

DGA Detection: We use these logs to identify **Domain Generation Algorithms (DGA)** by looking for high volumes of 'NXDOMAIN' (non-existent domain) responses or queries with high entropy (randomness), which are characteristic of malware attempting to contact a Command and Control (C2) server.

Tunneling Detection: We can also detect **DNS Tunneling** by analyzing query length and frequency. Since DNS is often allowed outbound by default, attackers use it to exfiltrate data or establish C2 channels. By monitoring for unusually long queries or distinct query patterns to a single domain, we can identify when DNS is being used as a transport protocol rather than a directory service.



Regulatory and Risk Perspective

1. **Availability expectations:** MAS TRM context and regulatory uptime mandates.
2. **NIST and ISO framing:** Mapping technical controls to compliance frameworks.
3. **Critical infrastructure risk:** Systemic risk considerations.
4. **Third-party exposure:** Managing supply chain risk.
5. **Board-level communication:** Translating technical risks into business language.
6. **Risk appetite vs. engineering ideals:** Pragmatic decision making.

We've spent the last hour acting as Engineers. Now, I need you to put on your 'CISO Hat.' We are going to look at how Routing and DNS security are viewed by Regulators, Auditors, and the Board of Directors.

1. Availability Expectations (MAS TRM & Mandates):

"First, we look at the law. Regulators like MAS (Monetary Authority of Singapore) don't care if your RPKI validation failed; they care that the banking service is down. We will discuss how **Availability Mandates** translate into technical requirements for redundancy and resilience.

2. NIST & ISO Framing (The Checklists):

"How do you prove you are secure? You map to a framework. We will look at how controls like **RPKI** and **Sinkholing** map directly to **NIST CSF** and **ISO 27001** requirements for 'Asset Management' and 'Protective Technology.'"

3. Critical Infrastructure Risk (Systemic Fragility):

"We will discuss **Systemic Risk**. The internet is a shared nervous system. We'll analyze how a failure in a neighbor's network (like a major ISP outage) becomes *your* critical risk, and why you must treat the internet as a hostile, contested environment.

4. Third-Party Exposure (Supply Chain):

"You can secure your castle, but you can't secure the road leading to it. We will

examine **Supply Chain Risk**, specifically, the risks of outsourcing your DNS to the cloud or relying on ISPs that route traffic through hostile territories.

5. Board-Level Communication (Translating Tech):

"One of the hardest skills is translation. We will practice converting technical terms like 'BGP Route Leaks' into business terms like 'Revenue Loss' and 'Brand Damage' so you can get the budget you need.

6. Risk Appetite vs. Engineering Ideals:

"Finally, we discuss the reality of trade-offs. We'll debate **Risk Appetite**, why we sometimes choose to 'Fail Open' (allowing traffic despite security warnings) to keep the business running, and how to document that decision.



Availability Expectations

- **The Regulatory View:** Regulators (like MAS in Singapore) don't care about your BGP config; they care about **Uptime**.
 - **High Availability (HA):** It is not just about having two routers. It is about Path Diversity and Dependency Management.
- **The Failure Mode:**
 - If you rely on a single DNS provider and they go down, you are in breach of availability mandates.
 - **Case Evidence:** The Facebook Outage (2021) showed that a single BGP configuration error can act as a "Kill Switch" for the entire enterprise, locking engineers out of the building and removing the company from the internet.
- **Compliance Requirement:** You must demonstrate that a BGP or DNS failure in one provider does **not** cause a total system outage (e.g by using Multi-homing or Anycast).

Connect Availability" to Resilience. Regulators want to know: If L3 breaks, do you break?"

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Regulatory View: Outcomes Over Configuration When we discuss availability with regulators, such as the Monetary Authority of Singapore (MAS) or similar bodies listed in compliance frameworks like Azure's Trust Center, the conversation shifts from *how* we configure our networks to *whether* they work. Regulators prioritize **Availability**, one of the three pillars of the CIA triad, ensuring that systems work promptly and service is not denied to authorized users,. While we might spend hours tuning BGP knobs, the regulator's primary concern is the business impact of downtime, which can be quantified in significant financial loss per minute. They function on the premise that business operations are inherently risky, and management must balance this risk against availability needs; simply having a configuration is not enough if the outcome is an outage.

2. High Availability: Diversity and Dependency True High Availability (HA) is not merely about possessing redundant hardware; it is about architectural resilience. As noted in the sources, redundancy must extend beyond the device level to include WAN and internet access, ensuring there are no single points of failure in the connectivity path. This involves managing dependencies so that a failure in one component does not cascade. For instance, reliance on a single provider for critical infrastructure guarantees disruption if that provider fails. We

must design for resilience by assuming failures are a given and limiting their 'blast radius' to ensure the end-user experience remains unaffected.

3. The Failure Mode: The Fragility of BGP and DNS The slide highlights the Facebook 2021 outage as a case study in BGP fragility. BGP is the routing protocol that powers the internet and data centers, but it is susceptible to misconfigurations that can propagate rapidly,. As we see in our text, an unadorned redistribution of routes or a configuration error can lead to advertising incorrect addresses or black-holing traffic, effectively acting as a 'kill switch' that isolates the enterprise from the internet. If we rely on a single DNS provider or BGP route, and that control plane fails, we breach our availability mandates because the 'road' to our digital doorstep has been erased.

4. Compliance and Resilience: Answering the 'Level3' Question Finally, we must address the hard question: 'If Level3 breaks, do you break?' Compliance requires us to demonstrate that a failure in one upstream provider does not cause a total system outage. To achieve this, we employ strategies like **Anycast** addresses, where a single IP address is announced by multiple endpoints, allowing routing logic to automatically direct traffic to the nearest healthy node. By utilizing multiple cloud providers or distinct availability zones, we ensure that a disruption in one provider's infrastructure does not result in a total loss of service for our organization. This connects 'Availability' directly to 'Resilience', we must prove that our architecture can survive the inevitable failure of a dependency.



NIST & ISO Framing (Mapping Controls)

- **The Frameworks:** Mapping technical controls to standards like NIST CSF or ISO 27001.
- **Mapping Exercise:**
 - **Identify (Asset Management):** Do you know every external IP and Domain you own? (Preventing Subdomain Hijacking/Shadowing) .
 - **Protect (Access Control):** Implementing RPKI (Route Validation) and MFA for Registrar accounts.
 - **Detect (Continuous Monitoring):** Using Looking Glasses and BGPStream to detect external hijacks.
 - **Respond (Mitigation):** Having a playbook for Sinkholing infected hosts.
- **Industry Standards:** Adherence to MANRS (**Mutually Agreed Norms for Routing Security**) is now often considered **Reasonable Security**" by auditors.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Frameworks: Bridging Technical and Governance To translate technical routing and DNS security into language that management and auditors understand, we map our actions to established frameworks like NIST Cybersecurity Framework (CSF) or ISO 27001. As the sources note, these standards provide a consensus-driven benchmark for security. While compliance with a standard does not guarantee security, a lack of compliance almost certainly indicates insecurity,. By mapping technical controls, like RPKI or sinkholing, to these governance frameworks, we demonstrate that our network defense capability is mature and auditable.

2. Identify: Asset Management as the Root of Trust Under the **Identify** function, we focus on Asset Management. The sources highlight that you cannot secure what you do not know you have; specifically, CIS Control 1 mandates a comprehensive inventory of all enterprise assets,. In the context of DNS, a failure here leads to **Subdomain Hijacking**. If we fail to identify and remove 'dangling' CNAME records, DNS entries pointing to cloud resources that have been decommissioned, attackers can register those resources and hijack our domain trust,. Effective asset management means knowing every external IP and domain to ensure these 'zombie' records are purged.

3. Protect: Validating Routes and Access For the **Protect** function, we

implement rigorous Access Controls. This includes securing the BGP routing layer using **Resource Public Key Infrastructure (RPKI)**. RPKI uses a chain of trust and digital certificates to cryptographically validate that an Autonomous System (AS) is authorized to announce specific IP prefixes, preventing route hijacking,. Furthermore, we must protect the management plane of our registrars. Since compromised credentials can lead to total domain takeovers, we must enforce Multi-Factor Authentication (MFA) on these accounts, ensuring that access requires 'something you have' and 'something you know'.

4. Detect: Continuous Monitoring of the Routing Plane In the **Detect** function, we move from static checks to **Continuous Monitoring**. As noted in the texts, BGP is vulnerable to hijacking where attackers advertise specific prefixes to redirect traffic,. To detect this, we cannot rely on internal logs alone; we must utilize external monitoring tools (like BGPStream or Looking Glasses) that observe how the rest of the internet routes to us. This aligns with the concept of continuous diagnostics to validate that security measures are effective in real-time.

5. Respond and Standards: Mitigation via Sinkholing Finally, for **Respond**, we must have playbooks ready for immediate mitigation. A primary technique for containing malware or stopping communication with hostile infrastructure is **Sinkholing**. This involves configuring internal DNS servers to return a non-routable address (such as 0.0.0.0) for known malicious domains, effectively severing the connection between infected internal hosts and the attacker's command and control,. By adhering to these practices, often codified in industry norms like MANRS, we provide what auditors define as 'assurance,' proving that our controls are not just designed, but are operating effectively.



Critical Infrastructure Risk (Systemic Fragility)

- **The Concept:** The Internet is a shared nervous system. Your security depends on the hygiene of your neighbors.
- **Systemic Risk:**
 - **BGP Trust:** BGP was designed in 1989 on a handshake trust model. It assumes all operators are friendly.
 - **Cascading Failure:** A mistake by a small ISP (like the YouTube/Pakistan incident) can ripple globally, taking down critical infrastructure in other countries.
- **Risk Assessment:** You must treat the Internet not as a stable utility (like electricity), but as a hostile, contested environment where routes can change instantly due to malice or incompetence.

1. The Concept: A Legacy of Implicit Trust To understand the systemic risk of the internet, we must look at its foundations. As noted in the sources, the initial design of most routing and switching architectures, specifically the Border Gateway Protocol (BGP), was established decades ago in an era of implicit trust between devices. The internet functions as a shared nervous system where your availability is inextricably linked to the hygiene of your neighbors. Routing protocols were written with a 'trust your neighbor' point of view, which works well in a closed, trusted Wide Area Network (WAN) but creates significant peril on the public internet where participants may be malicious or incompetent.

2. Systemic Risk: The Fragility of BGP The core systemic risk lies in BGP's handshake trust model. While BGP peers are explicitly defined, the protocol essentially believes what it is told. If a neighbor advertises a route, BGP generally assumes it is valid. This mechanism makes it relatively simple for an actor to abuse routing protocols to redirect or change traffic flows. Because the internet relies on this chain of trust, a compromise or misconfiguration in one autonomous system (AS) can allow an adversary to offer a 'better' path for traffic, effectively hijacking the route. The protocol was not designed to verify the veracity of these advertisements by default.

3. Cascading Failure: The Mechanics of a Hijack The slide mentions the

YouTube incident, and the sources detail exactly how such a cascading failure occurs mechanically. BGP routing prefers the 'shortest AS path,' but crucially, the 'longest prefix' (most specific route) overrides that preference. In the YouTube case, while the valid route might have been a broad /19 subnet (e.g 104.237.160.0/19), an attacker or misconfigured ISP can advertise a more specific /24 subnet (e.g 104.237.171.0/24). The global routing table updates instantly, directing traffic to the hijacker. We have seen this ripple effect in other incidents, such as when Vodafone Idea Ltd. mistakenly announced over 30,000 BGP prefixes, causing a massive 13x spike in inbound traffic and disrupting connectivity globally. A single error by a small operator can physically reroute traffic destined for critical infrastructure.

4. Risk Assessment: The Internet as Hostile Terrain Consequently, we must fundamentally alter our risk assessment. We cannot treat the internet as a stable utility like electricity. It is a hostile, contested environment where everything that crosses the network is liable to be intercepted, modified, or dropped,. Routes can change instantly due to malice, as seen in the Rostelecom hijacking of financial subnets, or incompetence. Since we cannot rely on the 'trust your neighbor' model in a hostile WAN, our defense must focus on resilience, ensuring that even if the 'road' to our data center changes, our data remains protected.



Third-Party Exposure (Supply Chain)

- **The Blind Spot:** You can secure your perimeter, but you cannot secure the path to your perimeter.
- **Vendor Risk:**
 - **Cloud DNS:** If you outsource DNS to a cloud provider, you share their risk surface. If their BGP is hijacked, your domain is hijacked.
 - **ISP Trust:** Traffic from your users might be detoured through hostile nations (e.g the China Telecom detour) before reaching you, allowing for espionage without service interruption.
- **Mitigation:** Continuous monitoring of AS Paths using external sensors to verify that your traffic is taking the expected physical route.

1. The Blind Spot: Insecurity of the Path We must recognize that our security perimeter effectively ends where our ISP connection begins. While we spend significant resources hardening our internal 'castle,' we have zero control over the 'roads', the internet infrastructure, that our users traverse to reach us. This introduces a critical supply chain risk: we rely on upstream providers for connectivity, but we cannot secure the path to our perimeter,. As noted in our texts, everything that crosses a network can be intercepted, copied, or modified by an adversary and then forwarded to its destination. Because the latency introduced by this interception is often only milliseconds, we may never realize that our traffic is being detoured or sniffed until it is too late.

2. Vendor Risk and BGP Hijacking When we outsource DNS to a cloud provider, we are not just buying a service; we are sharing their risk surface. The fundamental routing protocol of the internet, BGP, relies on a web of trust between Autonomous Systems (AS). If an attacker targets your cloud DNS provider by advertising a more specific prefix or a 'better' path, they can hijack the traffic destined for your domain,. We have seen real-world examples of this fragility, such as the incident involving the Russian ISP Rostelecom, where financial subnets were hijacked and traffic was redirected. If your provider's BGP routes are compromised, your zone files might be secure, but the traffic never

reaches them.

3. ISP Trust and Espionage This vulnerability extends to ISP trust. Attacks like the China Telecom detour (or similar BGP redirection events) exploit the fact that the 'shortest AS path' is usually the preferred route. By manipulating these advertisements, hostile actors can force traffic from your users to detour through their infrastructure before forwarding it to you. This allows for espionage and data theft without causing a service interruption, making it incredibly difficult to detect from inside your network,. The adversary essentially becomes a 'man-in-the-middle' on a global scale.

4. Mitigation: External Monitoring Because this activity happens outside your firewall, internal logs will show normal traffic arrival. To defend against this, you must look outward. The most effective mitigation is continuous monitoring of AS Paths using external sensors. Services like BGPMON can monitor specific subnets and alert you when the path your traffic takes changes unexpectedly or passes through a suspicious Autonomous System,. You must verify that the physical route your traffic takes matches your expectations, rather than blindly trusting the internet's routing tables.



Board-Level Communication (Translating Tech)

- **The Challenge:** The Board does not understand BGP Hijack" or DNS Tunneling. They understand Revenue, Reputation, and Regulation.
- **Translation Guide:**
 - **Technical:** We need to deploy RPKI and DNSSEC.
 - **Business:** We need to digitally sign our network address to prevent fraudsters from impersonating our bank to customers".
 - **Technical:** We need to block DoH and implement RPZ.
 - **Business:** We need to ensure our security cameras (logs) aren't blinded by encrypted traffic so we can stop ransomware before it encrypts data".

1. The Challenge: Bridging the Gap When presenting to the Board, we must recognize that our technical vocabulary, terms like 'BGP Hijacking' or 'DNS Tunneling', often fails to land. As noted in our threat detection materials, influencing leadership is less about dry spreadsheets and more about storytelling that answers the critical question: 'So what?'. The Board operates on the axes of **Revenue, Reputation, and Regulation**. Therefore, our job is not to teach them networking protocols, but to translate technical risks into business impacts, demonstrating how security controls directly protect the organization's bottom line and brand integrity.

2. Translating RPKI and DNSSEC: Protecting Brand Identity When we request budget for **RPKI and DNSSEC**, we are effectively asking to digitally sign our network's street address. Technically, these protocols prevent route hijacking and cache poisoning, where attackers redirect our traffic to their own servers,. To the Board, however, we frame this as fraud prevention. Without these controls, criminals can impersonate our bank, intercepting customer traffic to steal credentials and funds. By implementing these signatures, we are preventing wholesale identity theft of our organization, thereby protecting our **Reputation** and preventing direct **Revenue** loss associated with fraud recovery and customer churn.

3. Translating DoH and RPZ: Maintaining Observability Similarly, the conversation around **blocking DoH (DNS over HTTPS)** and implementing **RPZ (Response Policy Zones)** must be framed around visibility and risk management. Technical staff know that DoH encrypts DNS queries inside HTTPS traffic, effectively bypassing our monitoring tools and creating a blind spot for security operations,. To the Board, we explain this using the 'Security Camera' analogy: Allowing unmanaged encrypted DNS is equivalent to letting burglars spray paint over our security cameras during a heist. We need to block DoH to ensure our 'cameras' (logs) can see the early warning signs of a ransomware attack, specifically the command and control signaling, before it encrypts our data,. Implementing RPZ acts as a firewall for these requests, actively blocking connections to known malicious locations and reducing the risk of a catastrophic regulatory or operational breach.



Risk Appetite vs. Engineering Ideals

- **The Trade-off:** Security often conflicts with Usability and Maintenance.
- **Example 1: The Allow List" (Whitelisting):**
 - **Ideal:** Block everything, allow only known good domains.
 - **Reality:** The web is too interconnected (CDNs, Ads, Analytics). Maintaining an Allow List for a user LAN is impossible and breaks business operations.
 - **Risk Decision:** We accept the risk of a Deny List" (RPZ) approach because the operational cost of Allow Listing" is too high.
- **Example 2: Failing Closed vs. Open:**
 - If RPKI validation fails, do you drop the traffic (Security) or allow it (Availability)? Most organizations **prioritize Availability** until they reach high maturity.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Trade-off: Security vs. Usability We must acknowledge that security architecture involves a constant negotiation between protection and operational reality. While we strive for maximum security, we must balance this against user comfort and business performance. As noted in the sources, if security introduces excessive blocking or friction to the organization, it is often ignored rather than embraced. Therefore, our risk appetite is defined by how much operational friction we are willing to accept to achieve our engineering ideals.

2. Example 1: The Allow List" (Whitelisting) Dilemma The engineering ideal is the **Allow List** (or whitelist). This is the most stringent security method because it explicitly defines valid inputs or destinations and rejects everything else, theoretically preventing any unknown attacks,. However, the reality of the modern web makes this nearly impossible to maintain. As the sources highlight, outbound IP whitelisting has become less feasible due to the reliance on Content Delivery Networks (CDNs) and Global Server Load Balancers (GSLBs), where IP addresses change rapidly. Strict allow-listing is operationally cumbersome and risks a 'self-inflicted denial-of-service' where legitimate business traffic is blocked because the list wasn't updated. Consequently, we make a **Risk Decision** to utilize **Deny Lists** (blacklists). While blacklisting is conceptually flawed because it attempts to 'enumerate evil' and cannot stop

unknown threats, it is often the only practical option. Technologies like **Response Policy Zones (RPZ)** allow us to implement these deny lists at the DNS layer to block known malicious domains without breaking dynamic web connectivity.

3. Example 2: Failing Closed vs. Open Finally, we must address failure modes. When a security control, such as an RPKI validator or an Intrusion Prevention System (IPS), fails, we face a binary choice: **Fail Closed** (block traffic) or **Fail Open** (allow traffic). Failing closed prioritizes security but ensures a service outage. Failing open prioritizes **Availability**, enabling business continuity even when the security control is down. Because availability is often the most critical requirement for business operations (e.g e-commerce), most organizations prioritize uptime and choose to fail open until they reach a high level of operational maturity.



Incident Response and Recovery

1. **Routing incident triage:** Rapid assessment and containment steps.
2. **DNS response playbooks:** Standard procedures for DNS attacks.
3. **External coordination:** Working with ISPs, registrars, and cloud providers.
4. **Crisis communication:** Managing stakeholders during outages.
5. **Validation and trust restoration:** Ensuring the fix is permanent.
6. **Control improvement loop:** Learning from the incident.

"We have spent the last few hours building walls and installing cameras. But in security, we operate under the assumption of failure. Eventually, a route *will* leak, or a cache *will* be poisoned. This section is your 'Firefighting Manual'.

1. Routing Incident Triage (The Outside-In" View):

"First, we tackle **Triage**. When connectivity drops, your internal dashboard will just show 'Red.' To know *why*, you have to look from the outside. We'll discuss using **Looking Glasses** to confirm if the internet is rejecting your routes or if you just have a broken cable.

2. DNS Response Playbooks (Specific Actions):

"Next, we go into the specific plays. If users are hitting a phishing site, you need to **Flush the Cache**. If internal hosts are beaconing to a C2 server, you need to **Sinkhole** the domain. We will walk through the exact commands to execute these defenses.

3. External Coordination (BGP is a Team Sport):

"You cannot fix a BGP hijack alone. You don't own the router that is hijacking you. We will discuss **External Coordination**, how to find the right NOC (Network Operations Center) contact at your ISP or the offending AS using **PeeringDB**.

4. Crisis Communication (The Facebook Lesson):

"We'll cover a critical lesson learned from the 2021 Facebook outage: **Don't rely**

on the network you are trying to fix to communicate. We will discuss the importance of **Out-of-Band (OOB)** management and communication channels.

5. Validation and Trust Restoration:

"Once the fire is out, how do you know it's safe to go back inside? We'll look at **Validation**, verifying the global routing table has converged, and **Trust Restoration**, which involves rotating every secret (passwords, keys) that might have been exposed during the breach.

6. Control Improvement Loop:

"Finally, we close the loop. Every incident is a lesson. We will discuss how to feed the data from the incident back into your RPKI policies and Zeek filters so you don't fight the same fire twice.



Routing Incident Triage

- **The First 15 Minutes:** Rapid assessment is critical. Is this a local hardware failure, or is the internet rejecting us?
- **Verification:** Do not trust your internal logs alone.
 - Use Looking Glasses (Hurricane Electric, Level3) to see your network from the outside.
 - **Question:** Is the path valid?"
- **Containment (The More Specific" Defense):**
 - If you are being hijacked (e.g someone announces your /22), you cannot stop" them directly.
 - **Counter-Move:** Announce More Specific prefixes (e.g four /24s).
 - **Result:** Global routers will prefer your specific /24s over the hijacker's /22, forcing traffic back to you immediately.
- In BGP, you fight routing with routing. Use the Longest Prefix Match" rule as a weapon of defense!

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The First 15 Minutes: External Verification In the initial moments of an outage, typically the 'Observe' phase of your incident response OODA loop, you must rapidly distinguish between an internal hardware failure and a systemic routing issue where the internet is rejecting your traffic. Internal logs are insufficient here because they only report what reaches your perimeter, not what is being diverted elsewhere. To verify the path, you must look outward using Looking Glasses or monitoring services like BGPMON. These tools allow you to view your network from the outside, helping you confirm if your prefixes are being advertised correctly or if an unauthorized Autonomous System (AS) has hijacked your route.

2. The Mechanics of a Hijack: Longest Prefix Wins To understand the mitigation, we must understand the attack. As noted in the sources, BGP prefers the 'shortest AS path,' but crucially, the 'longest prefix' (the most specific route) overrides that preference. This mechanism is what makes hijacking possible. A famous example occurred when Pakistan Telecom attempted to block YouTube by advertising a specific /24 subnet that fell within YouTube's broader /19 allocation. Because the /24 was more specific, global traffic destined for YouTube was re-routed to Pakistan Telecom, resulting in a global outage for the service.

3. Containment: Fighting Routing with Routing If you confirm a hijack, you cannot stop the attacker's advertisement directly. Instead, you must use the 'Longest Prefix Match' rule as a weapon of defense. If an attacker hijacks your /22 block, your counter-move is to de-aggregate that block and announce more specific prefixes, specifically, four /24 subnets. Because these routes are more specific than the attacker's /22 (or equally specific if they are also announcing /24s), the global routing tables will update to prefer your legitimate path. In BGP incident response, we do not just filter; we fight routing with routing to force traffic back to our infrastructure.



DNS Response Playbooks

- **Scenario A:** Cache Poisoning:
 - Symptom: Users are being redirected to a fake bank site.
 - Immediate Action: **Flush the Cache.** Run `rndc flush` (Bind) or `ipconfig /flushdns` (Windows).
 - **Hardening:** Enable **DNSSEC validation** immediately to prevent re-poisoning.
- **Scenario B:** Malware C2 (Beaconing):
 - Symptom: Zeek detects DGA domains.
 - **Action:** **Sinkhole** the domain. Update the RPZ to redirect the malicious domain to your internal sinkhole IP (e.g 10.10.10.10).
 - **Result:** This severs the C2 link and identifies the infected host for remediation.

1. Scenario A: The Cache Poisoning Crisis In this scenario, an adversary has successfully exploited a vulnerability to insert malicious records into your resolver's cache, a technique known as DNS cache poisoning or spoofing. The symptom is immediate and dangerous: legitimate user traffic, such as requests for a banking portal, is seamlessly redirected to an actor-controlled server. Because the corrupted record resides in the cache, the redirection happens without the attacker needing to intercept every packet continuously. **Immediate Action:** Your first move is containment via erasure. You must purge the corrupted data to force the resolver to query authoritative sources again. On Windows endpoints, this is executed via the `ipconfig /flushdns` command, which clears the local resolver cache. For BIND servers, an administrative command (like `rndc`) allows you to dump or flush the database to remove poisoned entries. **Hardening:** To prevent recurrence, you must deploy **DNSSEC** (Domain Name System Security Extensions). DNSSEC adds a layer of trust by digitally signing resource records. This allows your resolvers to cryptographically validate the origin and integrity of the data, ensuring that if an attacker attempts to inject a spoofed record without a valid signature, the server rejects it automatically.

2. Scenario B: Intercepting Malware C2 In the second scenario, we face malware attempting to phone home using **Domain Generation Algorithms**

(DGAs). These algorithms generate random domain names to evade static blocklists,. **Symptom:** Detection often comes from Network Security Monitoring tools like **Zeek**, which can analyze dns.log files to spot high-entropy domains or unusual query volumes that deviate from the baseline,. **Action:** We respond by **Sinkholing** the domain, often using **Response Policy Zones (RPZ)**. RPZ allows the DNS administrator to essentially 'lie' to the requesting client by overlaying custom information on top of global DNS data,. Instead of resolving the real C2 IP, we configure the nameserver to return a specific internal 'sinkhole' IP address (e.g 10.10.10.10),. **Result:** This action serves a dual purpose. First, it effectively severs the Command and Control (C2) link, containing the threat. Second, because the infected host is now trying to connect to your internal sinkhole IP, you can monitor traffic to that specific IP to identify exactly which machine on your network is compromised and requires remediation.



External Coordination

- **The Reality:** The internet is a shared ecosystem; you cannot fix a BGP hijack alone.
- **The Upstream Call:**
 - Contact the NOC (Network Operations Center) of your ISP.
 - Data needed: The Hijacker's ASN (found via BGPStream).
- **Registrar Lockdown:**
 - If the attack is "Domain Shadowing" (stolen credentials), contact the Registrar (e.g. GoDaddy) to lock the account and revert NS records.
- **PeeringDB:** Ensure your own contact information is up to date in PeeringDB so others can call you.

1. The Reality: A Shared Trust Model We must accept that we cannot remediate a BGP hijack in isolation. The internet's routing architecture was designed in an era of implicit trust, meaning that once traffic leaves our physical perimeter, we lose direct control over its path. Because the internet functions as a shared ecosystem, a routing error or malicious advertisement by a neighbor can instantly redirect our traffic globally. Therefore, our defense strategy cannot be purely internal; it relies heavily on **External Coordination** with the entities that control the backbone of the internet.

2. The Upstream Call: Leveraging Relationships When a hijack occurs, similar to the Rostelecom incident where financial subnets were redirected,, your immediate counter-move is the **Upstream Call**. You must contact the Network Operations Center (NOC) of your ISP. As noted in the sources, effective defense requires having an ISP that is 'well-placed' on the internet and capable of requesting speedy takedowns from their upstream peers. To make this call effective, you need precise data. You must provide the **Hijacker's ASN** (Autonomous System Number). You obtain this data not from internal logs, but from external monitoring services like BGPMON (BGPStream), which track changes to your specific route advertisements and alert you when your prefixes are announced by an unauthorized entity.

3. Registrar Lockdown: Regaining Control If the attack vector is **Domain Shadowing**, where an attacker uses stolen credentials to create malicious subdomains pointing to their own infrastructure, you face a crisis of trust. Attack groups like Hazy Hawk utilize these tactics to hijack dormant records for malware distribution. In this scenario, you must contact your **Registrar** (e.g. GoDaddy or Network Solutions) immediately to lock the account and revert the Name Server (NS) records. This effectively 'seizes' authoritative control back from the adversary, stopping the resolution of the malicious subdomains. The goal is to cut off the adversary's ability to leverage your domain's reputation for their campaigns.

4. Contact Information: The Importance of Reachability Finally, the slide mentions **PeeringDB**, which serves as the de facto directory for network operators. While our text focuses on **WHOIS** protocols, the underlying security principle is identical: accurate contact information is non-negotiable. As the sources emphasize, if a network asset is hacked or starts attacking others, the internet community needs a valid way to contact the owner to fix the issue,. If your contact information is stale or redacted without a forwarding mechanism, you isolate yourself from the very community that can warn you that your network has been compromised.



Crisis Communication

- **The Facebook Lesson" (2021):**
 - When Facebook's BGP routes were withdrawn, their internal tools (DNS, email, badge readers) also failed.
 - Lesson: Do not rely on the network you are trying to fix to communicate.
- **Out-of-Band (OOB) Management:**
 - Maintain a separate communication channel (e.g cellular, Signal group) that does not depend on corporate DNS or routing.
- **Stakeholder Messaging:** Translate BGP Route Leak" to The map of the internet is broken, and traffic cannot find our building".

1. The Facebook Lesson": The Peril of In-Band Dependency We must analyze the Facebook 2021 outage not just as a technical BGP failure, but as a catastrophic failure of **dependency management**. As we discussed regarding availability, BGP errors can act as a 'kill switch' for the entire enterprise. When Facebook's BGP routes were withdrawn, they didn't just lose their public website; they lost the internal nervous system required to fix it. Their DNS resolvers couldn't query, their email servers couldn't send alerts, and physical badge readers failed because they relied on the same network backbone that was down. This reinforces a critical incident response lesson: You cannot rely on the network you are trying to fix to communicate about the fix. If your incident response plan relies on corporate VoIP or email, and that infrastructure is part of the blast radius, your response capability effectively drops to zero.

2. Out-of-Band (OOB) Management: Essential Redundancy To prevent this paralysis, we must implement rigorous **Out-of-Band (OOB)** management. In the context of infrastructure, this often involves a physically isolated management network or 'Access Ethernet' that bypasses the production network, allowing administrators to reach router consoles even during a Denial-of-Service (DoS) attack or routing failure,. However, for human communication, OOB means maintaining channels completely independent of corporate infrastructure. As

noted in the sources, advanced adversaries often eavesdrop on security teams, or the network itself may be unavailable. Therefore, we must establish separate, secure communication paths, such as LTE hotspots, Signal groups, or dedicated cloud-based email resources, that function independently of the corporate domain,,. This ensures that when the 'lines are cut,' the command structure remains intact.

3. Stakeholder Messaging: Storytelling Over Spec Finally, we must bridge the gap between technical reality and executive understanding. As our sources highlight, influencing leadership is about storytelling, not spreadsheets. When a BGP route leak occurs, telling the Board that 'an autonomous system is advertising more specific prefixes' is ineffective. Instead, we must translate this into business impact: 'The map of the internet is broken, and traffic cannot find our building.' This aligns with the concept of **Value Chain** protection; stakeholders need to know *what* is impacting liquidity and operations, not the technical minutiae,. By framing technical failures as tangible business risks, like the inability of customers to reach our digital doorstep, we enable faster, more decisive leadership action during a crisis.



Validation and Trust Restoration

- **Post-Incident Verification:**

- After the fix, use **Looking Glasses** to run traceroutes from multiple continents (London, Tokyo, NY).
- Confirm the AS_PATH is clean and no longer detours through the hijacker.

- **Trust Restoration:**

- If the incident was a breach (e.g Registrar compromise), rotate all secrets:
 - Registrar passwords.
 - DNSSEC Keys (KSK/ZSK rollover).
 - BGP MD5/TCP-AO passwords.

1. Post-Incident Verification: The External View Once we have applied mitigations, such as announcing more specific prefixes to counter a hijack, we must verify that the fix has actually propagated globally. We cannot rely on internal logs for this, because if traffic is being diverted, it never reaches our sensors to generate a log entry. To validate the fix, we utilize **Looking Glasses**, which are diagnostic tools allowing us to view the network from the perspective of external carriers like Hurricane Electric or Level3. By running **traceroutes** from diverse geographic locations (e.g London, Tokyo, NY), we can map the hop-by-hop path packets take to reach us,. We are specifically auditing the **AS_PATH** to confirm it is 'clean', meaning the traffic flows directly to our Autonomous System (AS) and does not detour through an unauthorized AS, which would indicate the hijack is still active.

2. Trust Restoration: Rotating the Keys to the Kingdom If the routing or DNS incident was caused by a credential compromise rather than a simple misconfiguration, we must enter a phase of **Trust Restoration**. This requires a comprehensive rotation of all secrets that establish identity and trust.

Registrar Passwords: First, we must rotate the credentials for our registrar accounts. If an attacker retains access here, they can simply point our nameservers back to their infrastructure. Best practice dictates resetting these

passwords and enforcing multi-factor authentication immediately following a breach,.

DNSSEC Keys (KSK/ZSK): If there is any suspicion that the DNS infrastructure was compromised, we must perform a key rollover for our **DNSSEC** keys, specifically the Key Signing Key (KSK) and Zone Signing Key (ZSK). These keys form the chain of trust from the root zone down to our records,. If an attacker possesses the private KSK/ZSK, they can sign malicious records that appear valid to the rest of the world. Rotating them invalidates the compromised keys.

BGP MD5/TCP-AO: Finally, we must rotate the authentication secrets used for our routing sessions. BGP sessions often rely on **MD5** passwords (or the newer TCP-AO) to authenticate peers. If an attacker has captured this shared secret, they can theoretically establish a rogue peering session or inject false routing information. Changing this password forces a reset of the trust relationship between our routers and our upstream providers, locking out unauthorized peers.



Control Improvement Loop

- **The Feedback Cycle:** Every incident must update the defense logic.
- **Inputs from the Incident:**
 - If **BGP Hijack**: Did we have RPKI set to Drop Invalid"? If not, enable it.
 - If **DNS Tunneling**: Why didn't Zeek catch the large TXT records? Tune the query length threshold.
 - If **Malware**: Update the RPZ feeds with the new C2 domains found during the hunt.

1. The Feedback Cycle: Closing the Loop This slide represents the critical transition from 'reactive' firefighting to 'proactive' resilience. Incident response is not a linear process with a finite end; it is a continuous cycle where the output of one incident becomes the input for better defense. As detailed in the SANS incident response models, the **Lessons Learned** (or Debrief) phase is arguably the most vital yet often neglected step. The goal here is not just to close a ticket, but to identify the root cause, whether it was a policy failure or a technology gap, and implement corrective controls to prevent reoccurrence. We must use the intelligence gained during the containment and eradication phases to improve our preparation for the next event.

2. BGP Hijack and RPKI Enforcement If we experience a routing incident, we must interrogate our BGP configuration. **Resource Public Key Infrastructure (RPKI)** uses digital certificates to cryptographically validate that a specific Autonomous System (AS) is authorized to announce a specific IP prefix. If a hijack occurred, it implies our routers accepted a bogus advertisement. The control improvement here is moving from merely 'monitoring' RPKI validity to enforcing a '**Drop Invalid**' policy. This ensures that if an invalid route is presented, such as an attacker trying to redirect our traffic, the router automatically rejects it, preventing the hijack from propagating.

3. DNS Tunneling and Zeek Tuning regarding DNS Tunneling, if our monitoring failed to catch data exfiltration, we must examine our detection thresholds. Attackers often use **TXT records** for tunneling because they can hold larger payloads than other record types, allowing for faster data transfer. If Zeek (formerly Bro) failed to alert on this, our query length threshold was likely too permissive. Zeek provides deep visibility into DNS transaction data, including query types and response lengths. The improvement action here is to tune our Zeek scripts or SIEM alerts to flag unusually large TXT records or high-entropy domains that deviate from our established baseline of normal traffic.

4. Malware and Response Policy Zones (RPZ) Finally, when we identify malware C2 infrastructure during an active hunt, that intelligence must be operationalized immediately. We do this by updating our **Response Policy Zones (RPZ)**. RPZ functions as a 'DNS firewall,' allowing us to override normal DNS resolution for known malicious domains and return a 'policy' response, such as an NXDOMAIN or a redirect to a sinkhole. By feeding the new C2 domains found during the investigation directly into our RPZ feeds, we effectively inoculate the rest of the network, preventing other infected hosts from successfully phoning home.



Emerging Trends and Future Risks

1. **RPKI adoption challenges:** The friction of global deployment.
2. **State-level routing interference:** Geopolitical use of routing attacks.
3. **DoH and DoT trade-offs:** Encryption vs. enterprise visibility.
4. **DNS provider centralization:** Risks of relying on a few major players.
5. **Geopolitical fragmentation:** The potential for a Splinternet.
6. **Automation and AI impacts:** The future of network operations.

"The internet is no longer just a technical utility; it is a contested domain. The threats we face tomorrow won't just be misconfigurations; they will be structural shifts in how the internet works.

1. RPKI Adoption Challenges: We know RPKI is the gold standard, but why isn't it 100% deployed? We're going to discuss the friction, the fear of 'Failing Closed' and dropping legitimate traffic, and why valid routes are currently only at ~50% global adoption.

2. State-Level Routing Interference: BGP is no longer just about connectivity; it's about espionage. We will revisit the **China Telecom** case study, where traffic wasn't just dropped (blackholed) but silently detoured for inspection. This is the new normal of cyber warfare.

3. DoH and DoT Trade-offs: Privacy is good for users, but hard for defenders. We will discuss the **DoH Blindfold**. When DNS moves to HTTPS (Port 443), our Zeek logs go dark. You need to decide: Do you prioritize user privacy or SOC visibility?"

4. DNS Provider Centralization: We will look at the 'Too Big to Fail' problem. As we saw with the **Facebook Outage**, when massive platforms rely on centralized DNS infrastructure, a single BGP error can wipe out the digital existence of billions of users instantly.

5. Geopolitical Fragmentation: The global internet is splintering. Nations are

building 'Walled Gardens' and sovereign routing tables. We will discuss how this fragmentation affects your ability to route traffic globally and compliant.

6. Automation and AI: Finally, we look at the speed of the fight. Attackers use **DGAs** (Domain Generation Algorithms) to generate thousands of domains in seconds. You cannot fight this manually. We will discuss using AI-driven tools like **RITA** to automate the hunt and match machine speed with machine defense.

"By the end of this section, you will be able to answer the CISO's question: 'What is the existential risk to our connectivity in the next 5 years?' Let's dive in.



RPKI Adoption Challenges (The Friction of Trust)

- The Current State: **RPKI** is the Gold Standard" for routing security, yet global adoption is not 100%.
- **The Friction points:**
 - "Fail Open" vs. Fail Closed": Most operators set RPKI to Drop Invalid", but some fear dropping legitimate traffic due to misconfiguration.
 - Legacy Gear: Older routers lack the cryptographic hardware to perform validation at line rate.
 - Repository Reliability: If the RPKI repository (the authority) goes offline, do you drop all routes? (The answer is usually to fallback to Not Found").

Compare this to the early days of HTTPS. It took 20 years to move from HTTP is default" to HTTPS is mandatory. We are in the middle of that transition for BGP.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Current State: Moving Beyond the Trust" Model RPKI (Resource Public Key Infrastructure) represents the shift from the internet's original 'handshake trust' model to one of cryptographic verification. As noted in our discussions on BGP, the protocol was originally designed in an era of implicit trust between devices, which makes it inherently vulnerable to hijacking and misconfiguration. While BGPSEC and RPKI aim to extend a Certificate Authority (CA) trust model to routing, effectively digitally signing routes, global adoption is not yet at 100%. We are currently in a transitional phase where the technology exists, but the operational implementation is lagging behind the threat landscape.

2. The Friction of 'Fail Closed': A Risk to Availability The most significant friction point is the operational decision between 'Fail Open' and 'Fail Closed.' In a 'Fail Closed' model (setting RPKI to 'Drop Invalid'), the router rejects any route that fails validation. While this secures the network, it introduces the risk of a self-inflicted Denial of Service. This is analogous to 'Strong CRL Checking' in standard PKI, where if a Certificate Revocation List (CRL) cannot be downloaded, the certificate is considered invalid and the connection fails, potentially causing massive outages,. Network operators fear that a simple misconfiguration, like the Vodafone incident where 30,000 prefixes were mistakenly announced, could result in dropping legitimate traffic if their validation logic is too rigid.

Consequently, most operators default to 'Fail Open' to prioritize uptime over absolute security.

3. Legacy Gear and Computational Cost Secondly, we face the 'Legacy Gear' problem. Cryptographic validation is computationally expensive. As we see in other contexts, like TLS inspection, decryption and validation can swamp standard processors, often requiring specialized ASICs to handle the load at scale. Many older routers currently in the field lack the cryptographic hardware acceleration necessary to validate signatures on thousands of BGP updates at line rate. Just as we see with IoT and legacy mobile devices, constraints on CPU power and memory make it difficult to support modern security protocols without degrading performance.

4. Repository Reliability and the HTTPS Analogy Finally, we must consider the reliability of the RPKI repositories, the 'Trust Anchors.' If the repository goes offline and a router cannot retrieve the latest ROAs (Route Origin Authorizations), the system faces a critical decision: drop all routes (disaster) or fallback to 'Not Found' (insecure). This mirrors the evolution of the web from HTTP to HTTPS. It took over two decades to move from SSL 3.0 (1996) to the modern, mandatory TLS 1.3 (2018) standards we see today. Early on, HTTPS was optional and 'failed open' to HTTP. We are currently in that same 'messy middle' for BGP security; we are moving toward mandatory verification, but until the infrastructure is ubiquitous and robust, we are forced to accept 'HTTP-level' insecurity to keep the lights on.



State-Level Routing Interference (The New Cold War)

- The Threat: Nation-states using BGP not for denial of service, but for Espionage and Traffic Shaping.
- Case Study: China Telecom:
 - Traffic from US government sites was routed through China Telecom infrastructure before reaching its destination.
 - Mechanism: Route Leaks and optimized routing announcements, rather than simple Blackholing".
 - The Impact: Users had connectivity (the page loaded), but the data was inspected en route.
- Future Risk: "Sovereign Internet" initiatives where countries mandate traffic stays within borders, fragmenting the global routing table.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Threat: From Disruption to Espionage While we traditionally associate cyberattacks with disruption, such as Denial of Service (DoS) attacks designed to crash networks, state-level actors have shifted focus toward **Espionage** and **Traffic Shaping**. As defined in our text, espionage involves obtaining unauthorized access to sensitive data to benefit an adversary, often a nation-state seeking military, political, or industrial advantages. In this context, BGP (Border Gateway Protocol) is weaponized not to break connectivity, but to redirect it. The goal is no longer to stop the traffic flow but to control the *path* it takes, allowing the adversary to copy or analyze sensitive communications while they are in transit. This represents a sophisticated form of 'Sabotage' where the strategic infrastructure is manipulated covertly rather than destroyed.

2. The Mechanism: Route Leaks and Longest Prefix Match The slide mentions the China Telecom incident, and we can understand the mechanics of such an event by looking at similar documented cases, such as the Rostelecom hijacking of financial subnets. The mechanism relied upon is the BGP **Longest Prefix Match** rule. Normally, BGP prefers the shortest path (fewest hops). However, a more specific route (a longer prefix) always overrides a shorter path. For example, if a legitimate route is advertised as a broad /19 block, a hostile state actor can advertise a specific /24 subnet within that block. Global routers will

automatically prefer this 'optimized' specific route, diverting traffic to the adversary's infrastructure. Unlike 'Blackholing,' where traffic is simply dropped to cause an outage, this technique keeps the traffic alive, allowing the attacker to become a 'Man-in-the-Middle'.

3. The Impact: Invisible Inspection The most dangerous aspect of this attack is that the user experience often remains unchanged. As noted in the sources, everything crossing a network can be intercepted, copied, or modified, and then forwarded to its original destination. If the adversary successfully forwards the traffic after inspection, the end-user sees the page load and assumes the connection is secure. Because the latency introduced by this detour is often only milliseconds, it is imperceptible to humans, allowing the espionage to continue undetected for extended periods. This allows nation-states to harvest credentials or inject 'lookalike' login pages to steal data without raising alarms.

4. Future Risk: Sovereignty and Fragmentation Finally, this behavior drives the push toward 'Sovereign Internet' initiatives. Because public internet traffic is subject to these routing manipulations, organizations are increasingly bypassing the public internet entirely for critical data, opting for direct, private connections to cloud providers to avoid 'machine-in-the-middle' attacks. The risk of data being stored or routed through countries where it is not permitted violates compliance and regulatory requirements, forcing a fragmentation of the global routing table as nations mandate that traffic stays within trusted borders to ensure data sovereignty.



DoH and DoT Trade-offs (Privacy vs. Security)

- **The Conflict:**

- Privacy: DoH (DNS over HTTPS) encrypts queries so ISPs and Governments cannot spy on users.
- Security: DoH blinds the SOC. It hides the dns.log data needed for Threat Hunting (Module 5).

- **Enterprise Reality:**

- If malware uses Google's DoH (8.8.8.8 via HTTPS), the firewall just sees HTTPS traffic to Google.
- Defense Strategy: Enterprises must block known DoH resolvers at the edge to force clients to fallback to internal, monitored DNS.

This is a rare instance where Privacy" tools hurt Security" operations. You must decide who you trust more: Your ISP or the Malware 😊

1. The Conflict: Privacy for Users vs. Blindness for Defenders We are currently witnessing a fundamental clash between privacy protocols and security operations. Protocols like **DoH (DNS over HTTPS)** and **DoT (DNS over TLS)** were designed to protect user privacy by encrypting DNS queries, ensuring that ISPs and governments cannot spy on browsing habits or manipulate responses. However, for the enterprise SOC, this privacy comes at a steep cost: it effectively 'blinds' our monitoring tools. As we will discuss in Module 5, DNS logs are a critical source of threat intelligence used to identify malware Command and Control (C2), Domain Generation Algorithms (DGA), and data exfiltration. When DoH is employed, the dns.log data we rely on for threat hunting is hidden inside an encrypted tunnel, rendering traditional network security monitoring (NSM) tools ineffective for those streams.

2. Enterprise Reality: Hiding in Plain Sight In a practical enterprise scenario, DoH presents a severe visibility gap because it utilizes **TCP port 443**, the same port used for standard HTTPS web traffic. If malware is programmed to use a public DoH resolver, such as Google's **8.8.8.8**, your perimeter firewall will only see a generic encrypted connection to Google. It will not see the actual query for a malicious domain inside that packet. Unlike DoT, which uses a dedicated port (853) that is easily identified, DoH blends in with the noise of regular web

browsing, allowing adversaries to bypass local DNS controls and 'hide in plain sight'. This allows malicious actors to maintain C2 channels that are invisible to the organization's logging infrastructure.

3. Defense Strategy: Forcing the Fallback To regain visibility, enterprises must implement a strategy of containment and redirection. Since we cannot easily inspect encrypted DoH traffic without complex TLS interception, the most effective defense is to **block known external DoH resolvers** at the network edge. We should also block **TCP port 853** to prevent DoT usage. By cutting off access to these external encrypted resolvers, we force legitimate clients and some malware to 'fall back' to the standard, unencrypted internal DNS servers provided by DHCP. This restores our ability to log, monitor, and sinkhole malicious queries. Ultimately, this is a rare instance where privacy tools hurt security operations, forcing us to decide that trusting our internal infrastructure is safer than allowing blind, encrypted tunnels to the outside world.



DNS Provider Centralization (Systemic Risk)

- **The Too Big to Fail" Problem:** The internet relies on a handful of massive DNS providers (Cloudflare, AWS Route53, Akamai).
- **The Risk:**
 - If one provider has a bad software update (like the Facebook BGP error), huge swaths of the internet vanish simultaneously.
 - **Shared Infrastructure:** An attack on one customer (e.g a gaming site) can degrade performance for all customers on that Anycast node.
- **Mitigation:** Multi-provider DNS strategies (Primary on AWS, Secondary on Cloudflare) to ensure diversity.

1. The Too Big to Fail" Problem: Concentration of Risk We must recognize that while the cloud offers immense scalability, it creates a dangerous centralization of risk. As we see in the slide, relying on a single massive DNS provider creates a single point of failure for your digital presence. Historical precedents confirm this fragility; for instance, the 2016 DDoS attack on the DNS provider **Dyn** (now part of Oracle) didn't just take down one site, it took down multiple major organizations and companies that relied on Dyn for name resolution simultaneously. When a provider of this magnitude falters, it isn't a local service disruption; it is a systemic event where the general public loses the ability to reach vast swathes of the internet, regardless of whether the backend servers are actually operational.

2. The Risk: Fragility of the Control Plane The slide references the Facebook BGP error, but our data confirms this pattern of 'self-inflicted' outages across all major providers. For example, in 2017, an **AWS S3** outage caused by a mistyped command during maintenance cascaded into a four-hour event that cost S&P 500 companies an estimated \$150 million,. Similarly, in 2021, an impairment of network devices in AWS's *us-east-1* region rendered APIs for critical services like EC2 and DynamoDB unavailable for hours,. These incidents demonstrate that even with robust infrastructure, a bad software update or configuration error at

the provider level can instantly vanish your digital footprint. As noted in the sources, every few months, a major cloud vendor suffers a significant outage due to misconfigurations or infrastructure failures.

3. Shared Infrastructure and the 'Noisy Neighbor' Furthermore, we must address the risks inherent in **Shared Infrastructure**. When you utilize a major DNS or CDN provider, you are effectively in a multi-tenant environment. While providers use technologies like **Anycast** to distribute requests to the nearest nodes,, an overwhelming attack on one customer can degrade performance for everyone else on that infrastructure. We saw this when the Mirai botnet launched a massive attack against **KrebsOnSecurity**, challenging even Akamai's massive bandwidth capabilities. If your neighbor on the Anycast node is targeted by a volumetric DDoS, your legitimate traffic may become collateral damage as the provider scrubs or blackholes traffic to protect their network integrity.

4. Mitigation: Multi-Cloud Diversity To mitigate this systemic risk, we must move beyond simple redundancy to true diversity. The most effective strategy is a **Multi-provider** or **Multi-cloud** approach. This involves architecting your environment so that if your primary provider (e.g AWS) fails, you can failover to a secondary provider (e.g Azure or Cloudflare). As detailed in our texts, this might look like an 'Active-Active' architecture where a global DNS service routes traffic to both AWS and Azure in a round-robin fashion,. While this introduces complexity in management and cost, it is the only way to ensure business continuity when, not if, a major provider experiences a catastrophic failure.



Geopolitical Fragmentation (The Splinternet)

- **The Trend:** Moving away from a single global internet toward "Walled Gardens" based on national borders.
- **Routing Implications:**
 - Countries mandating specific BGP paths to avoid "hostile" nations.
 - **Data Sovereignty:** Laws requiring DNS resolution and data storage to occur physically within the country.
- **Operational Cost:** Network Architects must now design routing policies based on Legal compliance, not just Technical efficiency.

1. The Trend: The Rise of Digital Borders We are witnessing a fundamental shift from a borderless, global internet toward a fragmented landscape often referred to as the 'Splinternet.' As highlighted in our cloud security discussions, the concept of **Data Residency** has moved to the forefront of architecture. We are moving away from a single shared pool of resources toward 'Walled Gardens' where data storage and processing must occur within specific geographic regions to comply with local laws and regulations. This fragmentation forces us to treat the internet not as a uniform utility, but as a collection of distinct national networks with strict boundaries.

2. Routing Implications: Navigating Hostile Territory The routing implications of this fracture are profound. We can no longer assume that the shortest path is the safe path. As we discussed regarding BGP vulnerabilities, nation-states have exploited the 'trust your neighbor' model of routing protocols to redirect traffic. We have seen real-world examples, such as the **Rostelecom incident**, where financial subnets were hijacked and traffic was intercepted. To avoid having traffic detoured through 'hostile' nations for espionage purposes, countries and organizations are increasingly mandating specific BGP paths or utilizing direct connections like **Equinix Cloud Exchange** to bypass the public internet entirely, ensuring data travels only through secure, controlled conduits.

3. Data Sovereignty and Local Resolution Data Sovereignty laws are now dictating network architecture. Regulations such as the **EU's GDPR**, **Saudi Arabia's PDPL**, and **Germany's Data Protection Act** explicitly require that data on their citizens be stored and processed within their physical borders,,. This extends to DNS resolution as well; organizations must ensure that DNS queries and the resulting data flows do not inadvertently cross borders. For example, architects may need to implement **geo-fencing** or geography-based blocking at the CDN layer to ensure content is only delivered to and from permitted regions, effectively creating a localized internet experience for users in those jurisdictions.

4. Operational Cost: Compliance over Efficiency Consequently, the operational cost for Network Architects has shifted. We can no longer design routing policies based solely on technical efficiency or lowest latency. We must now design for **Legal Compliance**. Architects are forced to deploy complex multi-region environments where a user's transaction data is strictly routed to their home region to meet data sovereignty requirements, even if a data center in a neighboring country offers better performance. This requires sophisticated identity management, such as using **wrapped tokens** to inspect region claims and re-route users accordingly, ensuring that technical routing decisions align perfectly with geopolitical legal frameworks.



Automation and AI Impacts

- **The Defense:**
 - **Automated Hunting:** Using tools like RITA (Real Intelligence Threat Analytics) to mathematically identify C2 beacons without human intervention.
 - **Automated Response:** Connecting Zeek detection directly to BGP/RPZ injection to block threats in seconds.
- **The Offense:**
 - Attackers using AI to generate Human-like" domains that bypass Entropy checks (defeating DGA detection).
- **Future Model:** Machine vs. Machine" warfare where algorithms fight over routing updates and DNS queries faster than humans can react.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

1. The Defense: Mathematical Hunting and Instant Response On the defensive side, we must evolve beyond manual log review toward **Automated Hunting**. Tools like **RITA (Real Intelligence Threat Analytics)** exemplify this shift. RITA ingests Zeek logs not to look for signatures, but to perform **statistical anomaly analysis** on network behavior. It mathematically models the 'heartbeat' of Command and Control (C2) traffic, identifying beacons based on the consistency of packet intervals, data sizes, and connection durations, even when attackers attempt to hide with jitter. Furthermore, we must close the gap between detection and action through **Automated Response**. We can program detection engines like Zeek to do more than just alert; we can script them to act. Upon detecting a threat, Zeek can instantly trigger a remediation workflow, such as injecting a rule into a **Response Policy Zone (RPZ)** to sinkhole a malicious domain or updating **BGP** routing tables to divert attack traffic. This capability allows us to block threats in seconds, drastically reducing the 'Mean Time to Resolution' and preventing lateral movement before a human analyst even opens the ticket.

2. The Offense: AI-Driven Evasion However, the adversary is leveraging these same advancements. Historically, we detected **Domain Generation Algorithms (DGAs)** by analyzing the 'entropy' or randomness of a domain name (e.g

xyqz123.com), using tools that flag uncommon character pairs. Now, attackers use **Generative AI** and dictionary-based algorithms to produce 'Human-like' domains composed of legitimate words. These domains possess low entropy and blend in with normal traffic, effectively bypassing our traditional mathematical detection filters. This escalation leads us to a future model of **'Machine vs. Machine' warfare**. As noted in our discussions on Moving Target Defense and AI, human reaction times are insufficient against automated attacks that adapt in real-time. We are moving toward a state where defensive algorithms must autonomously fight offensive algorithms over routing updates and DNS queries, executing decisions at wire speed to maintain network integrity.